



SAPIENZA
UNIVERSITÀ DI ROMA

Modellazione

MIP

Problems with a symbolic model

Twenty-three problems written first in symbolic form and then instantiated: assignment and scheduling, location and covering, production planning, mixed models. For each one: statement, variables, model, model of the instance, constructive heuristic with its bound, dual of the relaxation with a dual solution built by hand, optimal solution, additional modelling questions and one variant worked out in full. At the end, how the course is organised and how it is assessed.

Fabio Furini

sites.google.com/view/fabiofurini

Department of Computer, Control and Management Engineering — Sapienza University of Rome

Online version and downloads:

fabiofurini.github.io/modellazione-mip/
github.com/fabiofurini/modellazione-mip

Academic year 2026–2027

Contents

7	Assignment and scheduling	3
	The family	3
7.1	Minimum-cost assignment with availability	4
7.2	Machines with a fixed usage cost	10
7.3	Job selection with revenues and fixed-cost machines	17
7.4	Parallel jobs: the processing time as a maximum	23
7.5	One machine, job classes with setup	28
7.6	Classes with completion bonus and “if and only if” reduction	34
7.7	Total tardiness on one machine: sequencing with big-M	40
8	Location and coverage	47
	The family	47
8.1	Capacitated facility location	48
8.2	p-median: at most k locations	53
8.3	Signal coverage with interference	59
8.4	Hub location with maximum cost	64
9	Production planning	71
	The family	71
9.1	Lot sizing with a fixed set-up cost	72
9.2	Production and workforce: two equivalent formulations	79
9.3	Vehicles: minimum lot and a bonus for variety	87
10	Mixed problems	94
	Why a chapter of mixed problems	94
10.1	Prizes obtainable in two ways	95
10.2	Combinatorial auction	100
10.3	Diet with a count of the foods and a minimum lot	105
10.4	Christmas trees and boxes of lights	111
10.5	Shipments in boxes	117
10.6	Summer camps	122
10.7	Antitrust split	128
10.8	Songs on CDs	134

10.9 Books on shelves	139
Index	145

Assignment and scheduling

Class: BIP / MILP

Script: one per problem, `python/fam07_1_assignment.py` ...

[Online page](#)

The family

Seven problems with the same skeleton: some *jobs* must be assigned to some *machines* with limited availability. What changes from problem to problem is what is paid and what is decided: the cost of the assignment (problem 1), the fixed cost of every machine switched on (2), the revenue of the jobs one chooses to execute (3), the processing time when the jobs on the same machine run in parallel (4), the setup of a class of jobs (5), a bonus collected *if and only if* a class is complete (6), the tardiness with respect to the due dates when the jobs follow one another on a single machine (7).

The links between variables revisited here:

- **activation** (problems 2, 3, 5): the binary variable “machine used” or “class activated” that drives the assignments, with the aggregated constraint $\sum_j t_{jm} x_{jm} \leq a_m y_m$ or the disaggregated one $x_j \leq y_c$;
- **maximum variable** (problem 4): the processing time $y_m \geq t_{jm} x_{jm}$ for every job, which at the optimum equals exactly the maximum;
- **if and only if** (problem 6): one direction is imposed by the constraint, the other by the objective;
- **big-M and disjunctions** (problem 7): “either j before i or i before j ”, with M equal to the sum of the times.

For each problem: model, proof of the links, instance, constructive heuristic (a bound from the side of feasible solutions), dual of the LP relaxation with a hand-built dual solution (a bound from the other side), solver solution, additional considerations and two modelling questions with solutions. All the numbers come from the scripts `fam07_1_assignment.py` ... `fam07_7_tardiness.py` in the `python/` folder, one per problem; each with its own notebook.

Notation of the family

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, indexed by $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, indexed by $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time (minutes) of job j on machine m ; t_j if it does not depend on the machine
c_{jm}	$\in \mathbb{Q}_{>0}$	cost (euros) of executing job j on machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost (euros) if machine m is used
a_m	$\in \mathbb{Q}_{>0}$	availability (minutes) of machine m ; a if there is a single machine
p_m	$\in \mathbb{Z}_{\geq 1}$	maximum number of jobs machine m can execute
r_j	$\in \mathbb{Q}_{>0}$	revenue (euros) if job j is executed
d_j	$\in \mathbb{Q}_{>0}$	due date (minutes) of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of job classes, indexed by $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c ; the classes partition the jobs
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	setup cost (euros) and setup time (minutes) of class c
v_c	$\in \mathbb{Q}_{>0}$	bonus (euros) if all the jobs of class c are executed
u	$\in \mathbb{Q}_{>0}$	reduction (minutes) of the availability if jobs of at least two classes are executed

7.1 Minimum-cost assignment with availability

Script: `python/fam07_1_assignment.py`

[Notebook in Colab](#)

Problem 7.1 — minimum-cost assignment

A company has to process $n \in \mathbb{Z}_{\geq 1}$ jobs on $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes and the value $c_{jm} \in \mathbb{Q}_{>0}$ is the cost in euros of processing job j on machine m . For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the available processing time in minutes. Each machine handles one job at a time (no parallel processing). The company seeks a minimum-cost assignment of all jobs to the machines.

The problem in words. We decide on which machine each job is processed. The objective: minimum total cost. The constraints: every job is processed by exactly one machine; the total time of the jobs assigned to a machine does not exceed its availability. It is the *generalised assignment* problem: an assignment with a knapsack constraint per machine.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
c_{jm}	$\in \mathbb{Q}_{>0}$	cost of processing job j on machine m
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m

Decision variables

We introduce the following $n k$ binary variables:

$$x_{jm} = \begin{cases} 1 & \text{if job } j \text{ is processed by machine } m, \\ 0 & \text{otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Using these variables, a BIP model for the problem reads as follows:

Model 7.1 — minimum-cost assignment

$$\min \sum_{j=1}^n \sum_{m=1}^k c_{jm} x_{jm} \quad (7.1a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.1b)$$

$$\sum_{j=1}^n t_{jm} x_{jm} \leq a_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.1c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.1d)$$

Description of the objective function and constraints

- the linear objective function (7.1a) minimises the total processing cost, the sum of the costs of the chosen assignments;
- the linear constraints (7.1b) ensure that each job is assigned to exactly one machine, so that all jobs are processed (n linear constraints);
- the linear constraints (7.1c) guarantee that the total processing time of the jobs assigned to each machine does not exceed its availability (k linear constraints);
- the constraints (7.1d) define the variables of the model.

The model has a single family of variables: there are no links to prove. The constraints (7.1c) are of the “capacity/resource” kind: two quantities of the same nature (minutes required and minutes available) compared, no logical implication to write. The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

Every family of constraints is one `addConstrs` named after its label; the matrix t_{jm} is a list of lists, a_m a list.

```
1 m = gp.Model("assignment"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 m.setObjective(gp.quicksum(c[j][mm] * x[j, mm] for j in range(n)
4                      for mm in range(k)), GRB.MINIMIZE)
```

```

5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
6 m.addConstrs((gp.quicksum(t[j][mm] * x[j, mm] for j in range(n)) <= a[
    mm]
7                 for mm in range(k)), name="availability")
8 m.optimize()

```

The instance

$n = 3$ jobs, $k = 3$ machines:

t_{jm}	$m = 1$	$m = 2$	$m = 3$	c_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	2	1	3	$j = 1$	5	10	2
$j = 2$	3	4	2	$j = 2$	5	4	6
$j = 3$	4	5	3	$j = 3$	5	4	6
					$m = 1$	$m = 2$	$m = 3$
				a_m	5	6	7

Model (7.1) for the instance:

The model of the instance

$$\begin{aligned}
 & \min 5x_{11} + 10x_{12} + 2x_{13} + 5x_{21} + 4x_{22} + 6x_{23} + 5x_{31} + 4x_{32} + 6x_{33} \\
 & \text{subject to } x_{11} + x_{12} + x_{13} = 1 \\
 & \quad \quad \quad x_{21} + x_{22} + x_{23} = 1 \\
 & \quad \quad \quad x_{31} + x_{32} + x_{33} = 1 \\
 & \quad \quad 2x_{11} + 3x_{21} + 4x_{31} \leq 5 \\
 & \quad \quad \quad x_{12} + 4x_{22} + 5x_{32} \leq 6 \\
 & \quad \quad \quad 3x_{13} + 2x_{23} + 3x_{33} \leq 7 \\
 & \quad x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\}
 \end{aligned}$$

Constructive heuristic: the primal bound

Three heuristics inspired by bin packing, all linear in the number of job-machine pairs. *Next-fit*: one machine is loaded at a time and the next one is opened when a job no longer fits. *First-fit*: each job goes on the first machine with enough residual availability. *Best-fit*: among the machines with enough availability, the one with minimum cost is chosen. We report the best-fit, the most natural here since the objective is a cost.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

Run on the instance (output of the script):

- **Step 1.** Job 1: residual availabilities $ra = (5, 6, 7)$; every machine is enough ($2 \leq 5$, $1 \leq 6$, $3 \leq 7$); costs 5, 10, 2: the minimum is machine 3, hence $x[1][3] = 1$ and $ra[3] = 7 - 3 = 4$.
- **Step 2.** Job 2: $ra = (5, 6, 4)$; every machine is enough; costs 5, 4, 6: the minimum is machine 2, hence $x[2][2] = 1$ and $ra[2] = 6 - 4 = 2$.

- **Step 3.** Job 3: $ra = (5, 2, 4)$; machine 2 is not enough ($5 > 2$); among the others, costs 5 and 6: the minimum is machine 1, hence $x[3][1] = 1$ and $ra[1] = 5 - 4 = 1$.

The feasible solution found is

$$\bar{x}_{13} = 1, \quad \bar{x}_{22} = 1, \quad \bar{x}_{31} = 1, \quad \text{all the other } \bar{x}_{jm} = 0,$$

of value $2 + 4 + 5 = 11$. Hence $UB = 11$ is an upper bound on the optimum:

$$z(MILP) \leq 11.$$

Next-fit and first-fit both find the assignment $x_{11} = x_{21} = x_{32} = 1$, of value 14: a worse bound.

LP relaxation and dual: the dual bound

The LP relaxation of model (7.1) replaces constraints (7.1d) with

$$x_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.2)$$

(The constraint $x_{jm} \leq 1$ is implied by constraints (7.1b): adding it would change nothing.) We associate a free dual variable μ_j with each constraint (7.1b) (equality) and a non-positive dual variable π_m with each constraint (7.1c) ($\leq$ in a minimisation problem). The dual of the LP relaxation is:

Dual of the LP relaxation of model 7.1

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k a_m \pi_m \quad (7.3a)$$

$$\text{subject to } \mu_j + t_{jm} \pi_m \leq c_{jm}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.3b)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.3c)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.3d)$$

Description. The variable μ_j is the value attached to job j , which has to be assigned anyway, and π_m the (non-positive) price of one unit of capacity of machine m . The objective (7.3a) prices the jobs and the available capacity. Constraints (7.3b) are the columns of the x_{jm} : putting job j on machine m satisfies its assignment constraint, and so is worth μ_j , and consumes t_{jm} of capacity at price π_m ; the balance cannot exceed the cost c_{jm} of that pairing. The μ_j are free because the assignment constraints are equalities, the π_m non-positive because the capacity ones are of type $\leq$ in a minimisation.

For the instance:

The model of the instance

$$\begin{aligned}
& \max \mu_1 + \mu_2 + \mu_3 + 5\pi_1 + 6\pi_2 + 7\pi_3 \\
& \text{subject to } \mu_1 + 2\pi_1 \leq 5 \\
& \mu_1 + \pi_2 \leq 10 \\
& \mu_1 + 3\pi_3 \leq 2 \\
& \mu_2 + 3\pi_1 \leq 5 \\
& \mu_2 + 4\pi_2 \leq 4 \\
& \mu_2 + 2\pi_3 \leq 6 \\
& \mu_3 + 4\pi_1 \leq 5 \\
& \mu_3 + 5\pi_2 \leq 4 \\
& \mu_3 + 3\pi_3 \leq 6 \\
& \mu_1, \mu_2, \mu_3 \geq 0 \\
& \pi_1, \pi_2, \pi_3 \geq 0
\end{aligned}$$

A hand-built dual solution. With $\bar{\pi}_m = 0$ for every machine, constraints (7.3b) become $\mu_j \leq c_{jm}$ for every m : the largest feasible value is

$$\bar{\mu}_1 = \min\{5, 10, 2\} = 2, \quad \bar{\mu}_2 = \min\{5, 4, 6\} = 4, \quad \bar{\mu}_3 = \min\{5, 4, 6\} = 4,$$

with value $2 + 4 + 4 + 0 = 10$. By weak duality every feasible dual solution is worth at most the LP relaxation, which in turn is worth at most the MILP:

$$10 \leq z(LP) \leq z(MILP) \leq 11.$$

The recipe has a meaning: “every job costs at least its minimum cost” is a lower bound anyone would write down; the dual formalises it and says how to improve it, with $\pi_m < 0$ where the availability is tight.

What the solver says. The optimum of the LP relaxation is $z(LP) = 53/5 = 10.6$ (equal to the optimum of the dual: strong duality), with duals $\tilde{\mu} = (2, 4.8, 5)$ and $\tilde{\pi} = (0, -0.2, 0)$: machine 2 is the tight resource. The MILP has optimum $z(MILP) = 11$ with $\tilde{x}_{13} = \tilde{x}_{22} = \tilde{x}_{31} = 1$: the best-fit heuristic had found the optimal solution, but only the solver certifies it, because the dual bound stopped at 10 (and the relaxation at 10.6: no integer between 10.6 and 11, so in fact $\lceil 53/5 \rceil = 11$ closes the gap — an argument that works because the costs are integer).

UB (best-fit)	LB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
11	10	53/5	11	0.0%

Additional considerations

- For every job j and machine m , $x_{jm} \in \{0, 1\}$ implies $x_{jm} \leq 1$: the $n k$ inequalities $x_{jm} \leq 1$ are valid for model (7.1), but they are implied by constraints (7.1b) and do not strengthen the relaxation (indeed $z(LP) = z(LP^+)$).
- If a job j does not fit on a machine m ($t_{jm} > a_m$), the variable x_{jm} can be fixed to zero before solving: constraint (7.1c) would force it anyway, but the model is smaller and the relaxation does not get worse.

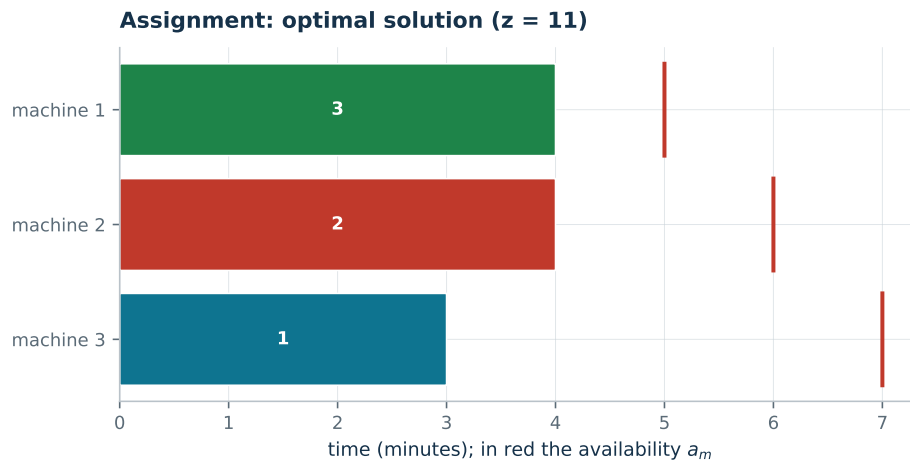


Figure 7.1: The optimal solution of the instance: the jobs assigned to each machine and, in red, the availability a_m .

Additional modelling questions

Modelling question 7.1 — jobs 1 and 3 on the same machine

Jobs 1 and 3 use the same tool and must be processed by the same machine. How does the model change? What is the new optimum for the instance?

Modelling question 7.2 — fixed cost per used machine

Every machine that processes at least one job costs an extra $g_m = 3$ euros to switch on. Model the fixed cost and find the new optimum. Which link of Chapter “Links between variables” comes into play?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	11	heuristic solution (UB)
LB	10	certified bound (LB)
$z(MILP)$	11	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 9.1\%.$$

The sandwich on variant 1a

The constraint "jobs 1 and 3 on the same machine" adds one free variable σ_m per machine to the dual, and with it the two jobs can be priced *together*: the constraints of their columns give $\mu_1 + \mu_3 \leq \min_m(c_{1m} + c_{3m})$, which is more than pricing them separately. The heuristic tries the pair on every machine that can hold it and places job 2 last; the two bounds meet and the problem closes without the solver.

	value	what it is
UB	12	heuristic solution
LB	12	dual certificate built by hand
$z(LP)$	12	relaxation without the bounds
$z(LP^+)$	12	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

7.2 Machines with a fixed usage cost

Script: `python/fam07_2_fixedcost.py`

[Notebook in Colab](#)

Problem 7.2 — machines with fixed cost

A company needs to process $n \in \mathbb{Z}_{\geq 1}$ jobs and has $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes of job j on machine m . For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the availability in minutes and the value $c_m \in \mathbb{Q}_{>0}$ is the cost in euros if the machine is used. Each machine handles one job at a time. The company wants to assign all jobs to the machines minimising the cost of the machines used.

The problem in words. We decide which machines to switch on and on which machine each job is processed. *The objective*: total cost of the machines switched on. *The constraints*: every job on exactly one machine; a machine switched off processes no job; a machine switched on does not exceed its availability. Compared with problem 7.1 the cost is no longer on the assignments but on the machines: *activation* variables are needed.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost if machine m is used

Decision variables

We introduce the following $n \cdot k + k$ binary variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is processed by machine } m, \text{ 0 otherwise,} \\ y_m = 1 \text{ if machine } m \text{ is used, 0 otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Using these variables, a BIP model for the problem reads as follows:

Model 7.2 — machines with fixed cost

$$\min \sum_{m=1}^k c_m y_m \quad (7.4a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.4b)$$

$$-\sum_{j=1}^n t_{jm} x_{jm} + a_m y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.4c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.4d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.4e)$$

Description of the objective function and constraints

- the linear objective function (7.4a) minimises the total cost of the machines used;
- the linear constraints (7.4b) ensure that each job is assigned to exactly one machine (n linear constraints);
- the linear constraints (7.4c) link the assignment variables with the usage variables and impose the capacity restrictions. In particular, for each machine, they ensure that if at least one job is assigned to it then the machine is used, that no job is assigned to an unused machine and that, if the machine is used, the total processing time of the assigned jobs does not exceed its availability (k linear constraints);
- the constraints (7.4d) and (7.4e) define the variables of the model.

Link between the variables

- For each machine m , if the total processing time of the jobs assigned to the machine is positive, then the machine must be used. This logical relation can be written as the implication

$$\sum_{j=1}^n t_{jm} x_{jm} > 0 \implies y_m = 1.$$

From the contrapositive we have

$$y_m = 0 \implies \sum_{j=1}^n t_{jm} x_{jm} = 0 :$$

if machine m is not used, the total processing time of the jobs assigned to it must be zero — that is, since $t_{jm} > 0$ for every j , no job can be assigned to it.

The implication is imposed directly by constraints (7.4c) of model (7.4). Indeed, for each machine m , constraint (7.4c) gives $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m y_m$; if $\sum_{j=1}^n t_{jm} x_{jm} > 0$,

then $a_m y_m \geq \sum_{j=1}^n t_{jm} x_{jm} > 0$, hence $y_m > 0$ and, y_m being binary, $y_m = 1$. Conversely, if $y_m = 0$, the constraint gives $\sum_{j=1}^n t_{jm} x_{jm} \leq a_m \cdot 0 = 0$ and, since $t_{jm} > 0$ and $x_{jm} \geq 0$ for every j , this forces $x_{jm} = 0$ for every j , that is $\sum_{j=1}^n t_{jm} x_{jm} = 0$.

- Conversely, for each machine m , if the total processing time of the jobs assigned to the machine is zero, then the machine is not used:

$$\sum_{j=1}^n t_{jm} x_{jm} = 0 \implies y_m = 0,$$

and by contraposition

$$y_m = 1 \implies \sum_{j=1}^n t_{jm} x_{jm} > 0 :$$

if the machine is used, at least one job is assigned to it.

This implication is *not* imposed by the constraints of model (7.4) (the solution $y_m = 1$ with no jobs is feasible), but it follows from the objective function in every optimal solution. Indeed, since $c_m > 0$, if $x_{jm} = 0$ for every j and $y_m = 1$, setting $y_m = 0$ keeps all constraints satisfied (constraint (7.4c) becomes $0 \geq 0$) and decreases the objective by the strictly positive amount c_m : the starting solution was not optimal. Therefore, in every optimal solution, a machine with no jobs is not used.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("fixed_cost"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(c[mm] * y[mm] for mm in range(k)),
5                 GRB.MINIMIZE)
6 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
7 m.addConstrs((-gp.quicksum(t[j][mm] * x[j, mm] for j in range(n))
8               + a[mm] * y[mm] >= 0 for mm in range(k)),
9               name="link")
10 m.optimize()
```

The instance

$n = 3$ jobs, $k = 3$ machines:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
c_m	8	7	5
a_m	25	20	12

Model (7.4) for the instance:

Model 7.2 on the instance

$$\begin{array}{llllll}
\min & & & & & 8y_1 + 7y_2 + 5y_3 \\
\text{subject to} & x_{11} + x_{12} + x_{13} & & & & = 1 \\
& & x_{21} + x_{22} + x_{23} & & & = 1 \\
& & & x_{31} + x_{32} + x_{33} & & = 1 \\
& -6x_{11} & -5x_{21} & -20x_{31} & +25y_1 & \geq 0 \\
& & -5x_{12} & -10x_{22} & -13x_{32} & +20y_2 \geq 0 \\
& & & -3x_{13} & -2x_{23} & -10x_{33} +12y_3 \geq 0 \\
& x_{11}, & x_{12}, & x_{13}, & x_{21}, & x_{22}, & x_{23}, & x_{31}, & x_{32}, & x_{33} & \in \{0, 1\} \\
& & & & & & & & & & y_1, & y_2, & y_3 \in \{0, 1\}
\end{array}$$

Constructive heuristic: the primal bound

The same three families as in problem 7.1, with the best-fit criterion becoming the *minimum time* (there are no assignment costs: it pays to consume little availability). The best-fit finds $x_{13} = x_{23} = x_{32} = 1$:

- **Step 1.** Job 1: $ra = (25, 20, 12)$; times 6, 5, 3, all feasible; minimum on machine 3: $x[1][3] = 1$, $ra[3] = 12 - 3 = 9$.
- **Step 2.** Job 2: $ra = (25, 20, 9)$; times 5, 10, 2; minimum on machine 3: $x[2][3] = 1$, $ra[3] = 9 - 2 = 7$.
- **Step 3.** Job 3: $ra = (25, 20, 7)$; machine 3 is not enough ($10 > 7$); between 20 and 13 the minimum is machine 2: $x[3][2] = 1$, $ra[2] = 20 - 13 = 7$.

The machines used are 2 and 3: $\bar{y} = (0, 1, 1)$, value $7 + 5 = 12$, hence $z(MILP) \leq 12$. Next-fit and first-fit use machines 1 and 2 (value 15); the “opened machines first” variants also give 15. The minimum-time best-fit is the only one of the five to find 12.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

The LP relaxation replaces (7.4d) and (7.4e) with $x_{jm} \geq 0$ and $y_m \geq 0$. With a free dual variable μ_j for each constraint (7.4b) and a non-negative π_m for each constraint (7.4c) ($\geq$ in a minimisation), the dual is:

Dual of the LP relaxation of model 7.2

$$\max \sum_{j=1}^n \mu_j \quad (7.5a)$$

$$\text{subject to } \mu_j - t_{jm} \pi_m \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.5b)$$

$$a_m \pi_m \leq c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.5c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.5d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.5e)$$

Description. The variable μ_j is the value of job j and π_m the price of one unit of capacity of machine m . The objective (7.5a) prices the jobs only: capacity by itself is not paid for. Constraints (7.5b) are the columns of the x_{jm} : assigning job j to machine m collects μ_j and consumes t_{jm} of capacity at price π_m , and the balance cannot be positive because the x_{jm} do not appear in the primal objective. Constraints (7.5c) are the columns of the y_m : all the capacity a_m that switching on machine m makes available, priced at π_m , cannot be worth more than the fixed cost c_m .

For the instance:

The model of the instance

$$\begin{array}{llll}
 \max & \mu_1 + \mu_2 + \mu_3 \\
 \text{subject to} & \mu_1 & -6\pi_1 & \leq 0 \\
 & \mu_1 & -5\pi_2 & \leq 0 \\
 & \mu_1 & -3\pi_3 & \leq 0 \\
 & \mu_2 & -5\pi_1 & \leq 0 \\
 & \mu_2 & -10\pi_2 & \leq 0 \\
 & \mu_2 & -2\pi_3 & \leq 0 \\
 & \mu_3 - 20\pi_1 & & \leq 0 \\
 & \mu_3 & -13\pi_2 & \leq 0 \\
 & \mu_3 & -10\pi_3 & \leq 0 \\
 & & 25\pi_1 & \leq 8 \\
 & & 20\pi_2 & \leq 7 \\
 & & 12\pi_3 & \leq 5 \\
 & \mu_1, \mu_2, \mu_3 & & \geq 0 \\
 & & \pi_1, \pi_2, \pi_3 & \geq 0
 \end{array}$$

A hand-built dual solution. Constraints (7.5c) say how large each π_m can be at most: take the maximum, $\bar{\pi}_m = c_m/a_m$, that is

$$\bar{\pi}_1 = \frac{8}{25}, \quad \bar{\pi}_2 = \frac{7}{20}, \quad \bar{\pi}_3 = \frac{5}{12}$$

(the cost per minute of each machine). Constraints (7.5b) then give $\mu_j \leq t_{jm}\bar{\pi}_m$ for every m , and the largest feasible value is

$$\begin{aligned}
 \bar{\mu}_1 &= \min\left\{6 \cdot \frac{8}{25}, 5 \cdot \frac{7}{20}, 3 \cdot \frac{5}{12}\right\} = \min\left\{\frac{48}{25}, \frac{7}{4}, \frac{5}{4}\right\} = \frac{5}{4}, \\
 \bar{\mu}_2 &= \min\left\{5 \cdot \frac{8}{25}, 10 \cdot \frac{7}{20}, 2 \cdot \frac{5}{12}\right\} = \min\left\{\frac{8}{5}, \frac{7}{2}, \frac{5}{6}\right\} = \frac{5}{6}, \\
 \bar{\mu}_3 &= \min\left\{20 \cdot \frac{8}{25}, 13 \cdot \frac{7}{20}, 10 \cdot \frac{5}{12}\right\} = \min\left\{\frac{32}{5}, \frac{91}{20}, \frac{25}{6}\right\} = \frac{25}{6}.
 \end{aligned}$$

Value: $\frac{5}{4} + \frac{5}{6} + \frac{25}{6} = \frac{25}{4}$. Hence

$$\frac{25}{4} \leq z(MILP) \leq 12.$$

The reading: every job “pays” at least the cost per minute of the machine that is cheapest for it, times its time. It is a weak bound: the fixed cost of a machine is paid in full as soon as it is used, but the relaxation spreads it over the minutes.

What the solver says. The optimum of the relaxation without the bounds is exactly $z(LP) = 25/4$: the hand-built solution is optimal for the dual. The relaxation strengthened

with $y_m \leq 1$ and $x_{jm} \leq 1$ is worth $z(LP^+) = 1273/200 = 6.365$; adding the disaggregated constraints $x_{jm} \leq y_m$ (valid, see below) it rises to $440/67 = 6.567$. The integer optimum is $z(MILP) = 12$, with machines 2 and 3 on and $\tilde{x}_{12} = \tilde{x}_{23} = \tilde{x}_{33} = 1$: the heuristic had found the optimum, and the gap between 6.25 and 12 measures how far the relaxation is from the integer in this family of models, where the “fractional” fixed costs fool the LP.

LB (best-fit)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
12	25/4	25/4	1273/200	12	0.0%

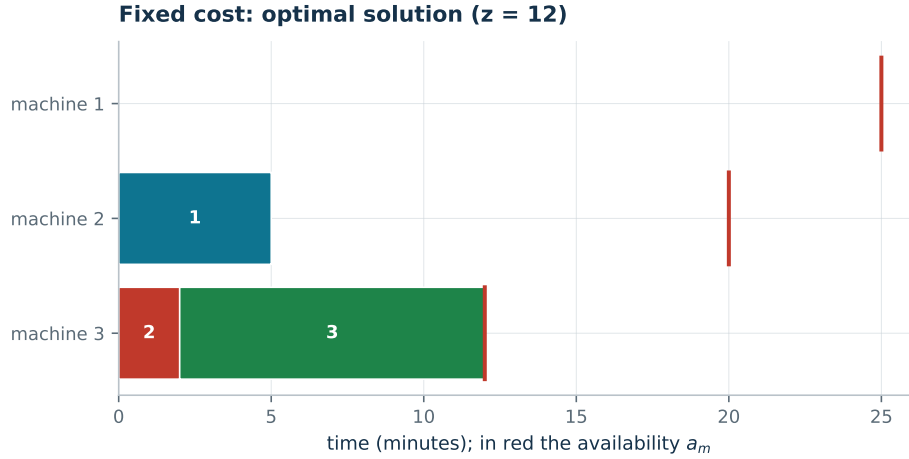


Figure 7.2: The optimal solution of problem 7.2: machines 2 and 3 are switched on.

Additional considerations

- The inequalities $y_m \leq 1$ (k) and $x_{jm} \leq 1$ ($n \cdot k$) are valid; the latter are implied by (7.4b), the former are not: indeed they strengthen the relaxation ($6.25 \rightarrow 6.365$).
- For each machine m , “if at least one job is assigned to m then m is used” is the Boolean implication

$$(x_{1m} \text{ OR } x_{2m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m,$$

whose contrapositive is

$$\text{NOT } y_m \implies (\text{NOT } x_{1m} \text{ AND } \dots \text{ AND } \text{NOT } x_{nm}) :$$

if the machine is not used, no job is assigned to it. As a Boolean expression,

$$\text{NOT } (x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \text{ OR } y_m;$$

by De Morgan and the distributive property it becomes the CNF

$$(\text{NOT } x_{1m} \text{ OR } y_m) \text{ AND } (\text{NOT } x_{2m} \text{ OR } y_m) \text{ AND } \dots \text{ AND } (\text{NOT } x_{nm} \text{ OR } y_m),$$

i.e. the constraints $1 - x_{jm} + y_m \geq 1$:

$$x_{jm} \leq y_m, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}$$

($n k$ valid inequalities, the *disaggregated* form of the link). The model implies them — we proved it above — but the relaxation does not: added, they bring $z(LP^+)$ to 440/67. It is the concrete example of Section “Activation: aggregated and disaggregated form”: same set of integer solutions, tighter relaxation.

- The opposite direction, “if no job is assigned then the machine is not used”, is the clause

$$\text{NOT } y_m \text{ OR } x_{1m} \text{ OR } \cdots \text{ OR } x_{nm},$$

i.e.

$$\sum_{j=1}^n x_{jm} \geq y_m, \quad \forall m \in \{1, 2, \dots, k\} :$$

not a valid inequality (the model allows $y_m = 1$ with no jobs) but it can be added without losing the optimum, because $c_m > 0$.

Additional modelling questions

Modelling question 7.3 — minimum usage of a machine switched on

Switching a machine on for a few minutes makes no sense: every machine used must work at least $\ell = 8$ minutes. Model the constraint and find the new optimum.

Modelling question 7.4 — link between two activations

Machine 3 shares the power supply with machine 1: if machine 1 is used, machine 3 must be used too. Write the constraint and discuss what it imposes and what it does not.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	12	heuristic solution (UB)
LB	$\frac{25}{4}$	certified bound (LB)
$z(MILP)$	12	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 47.9\%.$$

The sandwich on variant 2b

The link "if machine 1 is used then machine 3 is used too" adds a variable $\rho \leq 0$ to the dual, which loosens the column of y_1 and tightens that of y_3 . On the data of the instance it does not pay to move it: machine 3 is the minimum for every job, and lowering its price would lower every μ_j . The certificate stays the one of the base problem — a link between *activations* does not touch the relaxation, which can switch on half a machine. What grows is the integer optimum, and therefore the gap.

	value	what it is
UB	12	heuristic solution
LB	$\frac{25}{4}$	dual certificate built by hand
$z(LP)$	$\frac{25}{4}$	relaxation without the bounds
$z(LP^+)$	$\frac{1273}{200}$	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

7.3 Job selection with revenues and fixed-cost machines

Script: `python/fam07_3_selection.py`

[Notebook in Colab](#)

Problem 7.3 — job selection

A company can execute $n \in \mathbb{Z}_{\geq 1}$ jobs and has $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$, the value $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes (the same on every machine) and the value $r_j \in \mathbb{Q}_{>0}$ is the revenue in euros if the job is executed. For each machine $m \in \{1, 2, \dots, k\}$, the value $a_m \in \mathbb{Q}_{>0}$ is the availability in minutes and the value $c_m \in \mathbb{Q}_{>0}$ is the cost in euros if the machine is used. Each machine handles one job at a time. The company wants to choose which jobs to execute, and on which machines, to maximise the profit: revenues of the executed jobs minus costs of the machines used.

The problem in words. We decide which jobs to execute, on which machines, and which machines to switch on. *The objective:* maximum profit. *The constraints:* every job on at most one machine; no job on a machine switched off; availability respected. It is problem 7.2 where the jobs are no longer compulsory and have a revenue: a *maximisation* problem, and the roles of the bounds swap.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_j	$\in \mathbb{Q}_{>0}$	processing time of job j
r_j	$\in \mathbb{Q}_{>0}$	revenue if job j is executed
a_m	$\in \mathbb{Q}_{>0}$	availability of machine m
c_m	$\in \mathbb{Q}_{>0}$	fixed cost if machine m is used

Decision variables

We introduce the following $n k + k$ binary variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is executed by machine } m, 0 \text{ otherwise,} \\ y_m = 1 \text{ if machine } m \text{ is used, 0 otherwise,} \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Model 7.3 — job selection

$$\max \sum_{j=1}^n \sum_{m=1}^k r_j x_{jm} - \sum_{m=1}^k c_m y_m \quad (7.6a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.6b)$$

$$\sum_{j=1}^n t_j x_{jm} - a_m y_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.6c)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.6d)$$

$$y_m \in \{0, 1\}, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.6e)$$

Description of the objective function and constraints

- the linear objective function (7.6a) maximises the total profit, the difference between the revenue of the executed jobs and the cost of the machines used;
- the linear constraints (7.6b) ensure that each job is assigned to at most one machine, so that no job is executed twice (n linear constraints);
- the linear constraints (7.6c) link the assignment variables with the usage variables and impose the capacity restrictions: if at least one job is assigned to a machine then the machine is used, no job is assigned to an unused machine, and the total time of the jobs of a used machine does not exceed its availability (k linear constraints);
- the constraints (7.6d) and (7.6e) define the variables of the model.

Link between the variables

The link is that of problem 7.2, with t_j in place of t_{jm} : we restate the two implications, because the “optimality” direction changes sign.

- The implication

$$\sum_{j=1}^n t_j x_{jm} > 0 \implies y_m = 1,$$

$$\text{by contraposition: } y_m = 0 \implies \sum_{j=1}^n t_j x_{jm} = 0,$$

says that an unused machine has no jobs). Imposed by constraints (7.6c): if $\sum_{j=1}^n t_j x_{jm} > 0$ then $a_m y_m \geq \sum_{j=1}^n t_j x_{jm} > 0$, hence $y_m = 1$; conversely, if $y_m = 0$ then $\sum_{j=1}^n t_j x_{jm} \leq 0$ and, with $t_j > 0$ and $x_{jm} \geq 0$, all the x_{jm} are zero.

- The opposite implication

$$\sum_{j=1}^n t_j x_{jm} = 0 \implies y_m = 0,$$

$$\text{by contraposition: } y_m = 1 \implies \sum_{j=1}^n t_j x_{jm} > 0,$$

says that a used machine has at least one job). Not imposed by the constraints, it follows from the objective in every optimum: since $c_m > 0$, if $y_m = 1$ with no jobs, setting $y_m = 0$ stays feasible and *increases* the profit by c_m — in a maximisation problem the direction of the improvement is reversed, the structure of the argument is not.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("selection"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j, mm] for j in range(n)
5                        for mm in range(k))
6                - gp.quicksum(c[mm] * y[mm] for mm in range(k)),
7                GRB.MAXIMIZE)
8 m.addConstrs((x.sum(j, "*") <= 1 for j in range(n)), name="at_most_one"
9              )
10 m.addConstrs((gp.quicksum(t[j] * x[j, mm] for j in range(n))
11              - a[mm] * y[mm] <= 0 for mm in range(k)), name="link")
12 m.optimize()

```

The instance

$n = 3$ jobs, $k = 3$ machines:

	$m = 1$	$m = 2$	$m = 3$		$j = 1$	$j = 2$	$j = 3$
a_m	105	110	100	t_j	25	40	75
c_m	20	30	15	r_j	10	15	30

Model (7.6) for the instance:

Model 7.3 on the instance

$$\begin{aligned}
& \max 10x_{11} + 10x_{12} + 10x_{13} + 15x_{21} + 15x_{22} + 15x_{23} + 30x_{31} + 30x_{32} + 30x_{33} - 20y_1 - 30y_2 - 15y_3 \\
\text{subject to } & x_{11} + x_{12} + x_{13} \leq 1 \\
& x_{21} + x_{22} + x_{23} \leq 1 \\
& x_{31} + x_{32} + x_{33} \leq 1 \\
& 25x_{11} + 40x_{21} + 75x_{31} - 105y_1 \leq 0 \\
& 25x_{12} + 40x_{22} + 75x_{32} - 110y_2 \leq 0 \\
& 25x_{13} + 40x_{23} + 75x_{33} - 100y_3 \leq 0 \\
& x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \in \{0, 1\} \\
& y_1, y_2, y_3 \in \{0, 1\}
\end{aligned}$$

Constructive heuristic: the primal bound

In a maximisation problem a feasible solution gives a *lower* bound. The three heuristics of problem 7.1 adapt with one more rule: a job that fits on no machine is *skipped* (it is not compulsory). The best-fit chooses the *fullest* machine among those that are enough (minimum residual availability), to concentrate the jobs on few machines:

- **Step 1.** Job 1 ($t_1 = 25$): $ra = (105, 110, 100)$, all enough; the fullest is machine 3: $x[1][3] = 1$, $ra[3] = 75$.
- **Step 2.** Job 2 ($t_2 = 40$): $ra = (105, 110, 75)$; the fullest is still machine 3: $x[2][3] = 1$, $ra[3] = 35$.
- **Step 3.** Job 3 ($t_3 = 75$): $ra = (105, 110, 35)$; machine 3 is not enough; between 1 and 2 the fullest is machine 1: $x[3][1] = 1$, $ra[1] = 30$.

Solution: $\bar{x}_{13} = \bar{x}_{23} = \bar{x}_{31} = 1$, $\bar{y} = (1, 0, 1)$, profit $10 + 15 + 30 - 20 - 15 = 20$, hence $z(MILP) \geq 20$. Next-fit and first-fit fill machine 1 first (jobs 1 and 2) and put job 3 on machine 2: profit $55 - 50 = 5$.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing $x_{jm} \geq 0$ and $y_m \geq 0$, with non-negative dual variables μ_j for constraints (7.6b) and π_m for constraints (7.6c) ($\leq$ in a maximisation), the dual is:

Dual of the LP relaxation of model 7.3

$$\min \sum_{j=1}^n \mu_j \quad (7.7a)$$

$$\text{subject to } \mu_j + t_j \pi_m \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.7b)$$

$$-a_m \pi_m \geq -c_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.7c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.7d)$$

$$\pi_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.7e)$$

Description. The variable μ_j is the price of the right to run job j and π_m that of one unit of capacity of machine m . The objective (7.7a) adds up the prices of the rights on the jobs. Constraints (7.7b) are the columns of the x_{jm} : running job j on machine m consumes the right, at price μ_j , and t_j of capacity, at price π_m ; together they must cover the revenue r_j that job brings. Constraints (7.7c) are the columns of the y_m : the capacity a_m that switching the machine on makes available, priced at π_m , cannot cost more than the fixed cost c_m paid to switch it on.

For the instance:

The model of the instance

$$\begin{array}{llll}
 \min & \mu_1 + \mu_2 + \mu_3 \\
 \text{subject to} & \mu_1 & +25\pi_1 & \geq 10 \\
 & \mu_1 & +25\pi_2 & \geq 10 \\
 & \mu_1 & +25\pi_3 & \geq 10 \\
 & \mu_2 & +40\pi_1 & \geq 15 \\
 & \mu_2 & +40\pi_2 & \geq 15 \\
 & \mu_2 & +40\pi_3 & \geq 15 \\
 & \mu_3 & +75\pi_1 & \geq 30 \\
 & \mu_3 & +75\pi_2 & \geq 30 \\
 & \mu_3 & +75\pi_3 & \geq 30 \\
 & -105\pi_1 & & \geq -20 \\
 & & -110\pi_2 & \geq -30 \\
 & & & -100\pi_3 \geq -15 \\
 & \mu_1, \mu_2, \mu_3 & & \geq 0 \\
 & & \pi_1, \pi_2, \pi_3 & \geq 0
 \end{array}$$

A hand-built dual solution. Constraints (7.7c) bound each π_m from above: $\bar{\pi}_m = c_m/a_m$, that is $\bar{\pi}_1 = \frac{20}{105} = \frac{4}{21}$, $\bar{\pi}_2 = \frac{30}{110} = \frac{3}{11}$, $\bar{\pi}_3 = \frac{15}{100} = \frac{3}{20}$. Each μ_j must then satisfy $\mu_j \geq r_j - t_j \bar{\pi}_m$ for every m and $\mu_j \geq 0$: the smallest feasible value is the maximum of these,

$$\begin{aligned}
 \bar{\mu}_1 &= \max\left\{10 - 25 \cdot \frac{4}{21}, 10 - 25 \cdot \frac{3}{11}, 10 - 25 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{21}, \frac{35}{11}, \frac{25}{4}, 0\right\} = \frac{25}{4}, \\
 \bar{\mu}_2 &= \max\left\{15 - 40 \cdot \frac{4}{21}, 15 - 40 \cdot \frac{3}{11}, 15 - 40 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{155}{21}, \frac{45}{11}, 9, 0\right\} = 9, \\
 \bar{\mu}_3 &= \max\left\{30 - 75 \cdot \frac{4}{21}, 30 - 75 \cdot \frac{3}{11}, 30 - 75 \cdot \frac{3}{20}, 0\right\} = \max\left\{\frac{110}{7}, \frac{105}{11}, \frac{75}{4}, 0\right\} = \frac{75}{4}.
 \end{aligned}$$

Value: $\frac{25}{4} + 9 + \frac{75}{4} = 34$. By weak duality

$$20 \leq z(MILP) \leq 34.$$

The reading: μ_j is “how much job j is worth at most” once the spread fixed cost of the cheapest machine per minute (machine 3, at $\frac{3}{20}$ euros per minute) has been paid for every minute.

What the solver says. The optimum of the relaxation without the bounds is $z(LP) = 34$: here too the hand-built solution is optimal for the dual. With the bounds $x_{jm} \leq 1$ and $y_m \leq 1$ the relaxation with the bounds drops to $z(LP^+) = 680/21 = 32.38$. The integer optimum is $z(MILP) = 25$: jobs 1 and 3 on machine 3 ($25 + 75 = 100 \leq 100$, exactly the availability), profit $40 - 15$; job 2 is not worth it, since it would require a second machine ($c_1 = 20 > r_2 = 15$). The heuristic stopped at 20: it had put job 2 on machine 3, leaving out job 3, which then opened machine 1. A 20% gap.

LB (best-fit)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
20	34	34	680/21	25	20.0%

Additional considerations

- $y_m \leq 1$ (k) and $x_{jm} \leq 1$ ($n k$) are valid inequalities; the latter are implied by (7.6b), the former strengthen the relaxation ($34 \rightarrow 32.38$).
- The disaggregated constraints $x_{jm} \leq y_m$ (from

$$(x_{1m} \text{ OR } \dots \text{ OR } x_{nm}) \implies y_m,$$

as in problem 7.2) are valid and implied by the model, but strengthen the relaxation.

- If $r_j < \min_m c_m$ and job j is the only one on a machine, executing it never pays: this is what happens to job 2 in the instance. A dominance argument the solver makes by itself, but which helps to guess the optimum by hand.

Additional modelling questions

Modelling question 7.5 — all jobs compulsory

The company has already signed the contracts: all jobs must be executed. How does the model change and how much does the obligation cost on the instance?

Modelling question 7.6 — a job conditional on another

Job 3 requires a semi-finished product made by job 2: job 3 can be executed only if job 2 is executed too (on any machine). Write the constraint and find the new optimum.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	20	heuristic solution (LB)
UB	34	certified bound (UB)
$z(MILP)$	25	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 56.0\%.$$

The sandwich on variant 3b

"Job 3 only if job 2 as well" adds a multiplier $\lambda \geq 0$ to the dual, which discounts the columns of job 3 and loads those of job 2. One raises λ as far as job 2 can bear for free: on this instance there is no such margin, λ stays at zero and the certificate coincides with the one of the base problem.

	value	what it is
UB	34	dual certificate built by hand
LB	20	heuristic solution
$z(LP)$	34	relaxation without the bounds
$z(LP^+)$	$\frac{680}{21}$	relaxation with the bounds
$z(MILP)$	20	optimum of the MILP

7.4 Parallel jobs: the processing time as a maximum

Script: `python/fam07_4_parallel.py`

[Notebook in Colab](#)

Problem 7.4 — parallel jobs

A company needs to execute $n \in \mathbb{Z}_{\geq 1}$ jobs using $k \in \mathbb{Z}_{\geq 1}$ machines. For each job $j \in \{1, 2, \dots, n\}$ and each machine $m \in \{1, 2, \dots, k\}$, the value $t_{jm} \in \mathbb{Q}_{>0}$ is the processing time in minutes of job j on machine m . Each machine $m \in \{1, 2, \dots, k\}$ can execute at most $p_m \in \mathbb{Z}_{\geq 1}$ jobs, and executes them in parallel: all the jobs assigned to the same machine start together. The company wants to assign the jobs minimising the sum of the processing times of the machines, where the processing time of a machine is the time of the longest job among those assigned to it.

The problem in words. We decide on which machine each job goes. The objective: the sum of the processing times of the machines, each equal to the *maximum* of the times of the assigned jobs. The constraints: every job on one machine; at most p_m jobs on machine m . The novelty is the objective: a maximum is not linear, but it is linearised with a continuous variable and n " $\geq$ " constraints per machine.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of machines, $m \in \{1, 2, \dots, k\}$
t_{jm}	$\in \mathbb{Q}_{>0}$	processing time of job j on machine m
p_m	$\in \mathbb{Z}_{\geq 1}$	maximum number of (parallel) jobs on machine m

Decision variables

We introduce the following nk binary variables and k non-negative variables:

$$\begin{cases} x_{jm} = 1 \text{ if job } j \text{ is executed by machine } m, 0 \text{ otherwise,} \\ y_m = \text{processing time of machine } m, \end{cases}$$

$$\forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}.$$

Model 7.4 — parallel jobs

$$\min \sum_{m=1}^k y_m \quad (7.8a)$$

$$\text{subject to } \sum_{m=1}^k x_{jm} = 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.8b)$$

$$\sum_{j=1}^n x_{jm} \leq p_m, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.8c)$$

$$-t_{jm} x_{jm} + y_m \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8d)$$

$$x_{jm} \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.8e)$$

$$y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}. \quad (7.8f)$$

Description of the objective function and constraints

- the linear objective function (7.8a) minimises the sum of the processing times of the machines;
- the linear constraints (7.8b) ensure that each job is assigned to exactly one machine (n linear constraints);
- the linear constraints (7.8c) limit the number of jobs assigned to each machine (k linear constraints);
- the linear constraints (7.8d) link the assignment variables with the processing-time variables. In particular, for each job and each machine, they guarantee that if the job is assigned to the machine then the processing time of the machine is at least the time required by that job on that machine ($n k$ linear constraints);
- the constraints (7.8e) and (7.8f) define the variables of the model.

Link between the variables

It is the pattern of the *variable defined as a maximum* (Section “The maximum auxiliary variable”), in three steps.

- For each machine m and each job j , if the job is assigned to the machine then the processing time of the machine is at least the time of the job:

$$x_{jm} = 1 \implies y_m \geq t_{jm},$$

$$\text{and by contraposition: } y_m < t_{jm} \implies x_{jm} = 0 :$$

if the time of the machine is strictly smaller than t_{jm} , job j cannot be assigned to it. The implication is imposed directly by constraints (7.8d), which give $y_m \geq t_{jm} x_{jm}$: if $x_{jm} = 1$, $y_m \geq t_{jm}$. Conversely, if $y_m < t_{jm}$, then x_{jm} cannot equal 1, since the constraint would force $y_m \geq t_{jm}$; being binary, $x_{jm} = 0$. Since it holds for every j , the family (7.8d) imposes, for every machine, $y_m \geq \max\{t_{jm} x_{jm} : j \in \{1, 2, \dots, n\}\}$.

- Conversely, if no job is assigned to machine m , its processing time is zero:

$$\sum_{j=1}^n x_{jm} = 0 \implies y_m = 0,$$

$$\text{and by contraposition: } y_m > 0 \implies \sum_{j=1}^n x_{jm} > 0.$$

It is not imposed by the constraints (with $x_{jm} = 0$ for every j constraints (7.8d) reduce to $y_m \geq 0$), but it follows from the objective in every optimum: y_m has coefficient $1 > 0$ in the objective; if no job is on the machine and $y_m > 0$, setting $y_m = 0$ keeps the constraints ($0 \geq 0$) and decreases the objective by the strictly positive amount y_m .

- *Synthesis*: in every optimal solution

$$y_m = \max\{t_{jm}x_{jm} : j \in \{1, 2, \dots, n\}\},$$

i.e. y_m is exactly the processing time of the machine. Indeed the first point gives $y_m \geq$ maximum in every feasible solution; if in some solution $y_m >$ maximum, setting y_m equal to the maximum would keep all constraints (7.8d) (by definition of maximum) and decrease the objective by the difference, strictly positive: the solution was not optimal. With no jobs the maximum is 0, and the second point is recovered.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("parallel"); m.Params.OutputFlag = 0
2 x = m.addVars(n, k, vtype=GRB.BINARY, name="x")
3 y = m.addVars(k, name="y") #
   continuous, >= 0
4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(j, "*") == 1 for j in range(n)), name="assign")
6 m.addConstrs((x.sum("*, mm) <= p[mm] for mm in range(k)),
7               name="cardinality")
8 m.addConstrs((-t[j][mm] * x[j, mm] + y[mm] >= 0
9               for j in range(n) for mm in range(k)), name="maximum")
10 m.optimize()
```

The instance

$n = 3$ jobs, $k = 3$ machines, with the times of problem 7.2:

t_{jm}	$m = 1$	$m = 2$	$m = 3$
$j = 1$	6	5	3
$j = 2$	5	10	2
$j = 3$	20	13	10

	$m = 1$	$m = 2$	$m = 3$
p_m	1	2	2

Model (7.8) for the instance has 9 constraints (7.8d), one per pair: $-6x_{11} + y_1 \geq 0$, $-5x_{12} + y_2 \geq 0$, $-3x_{13} + y_3 \geq 0$, $-5x_{21} + y_1 \geq 0$, $-10x_{22} + y_2 \geq 0$, $-2x_{23} + y_3 \geq 0$, $-20x_{31} + y_1 \geq 0$, $-13x_{32} + y_2 \geq 0$, $-10x_{33} + y_3 \geq 0$; the three assignment constraints as in problem 7.2; the cardinalities $x_{11} + x_{21} + x_{31} \leq 1$, $x_{12} + x_{22} + x_{32} \leq 2$, $x_{13} + x_{23} + x_{33} \leq 2$; objective $\min y_1 + y_2 + y_3$.

Constructive heuristic: the primal bound

Next-fit on the cardinalities: machine 1 is filled up to p_1 jobs, then machine 2, and so on; the time of every machine is the maximum of the assigned times.

- **Step 1.** Job 1 on machine 1 (one job $\leq p_1 = 1$): $y[1] = \max(0, 6) = 6$.
- **Step 2.** Machine 1 is full: job 2 on machine 2: $y[2] = \max(0, 10) = 10$.
- **Step 3.** Job 3 on machine 2 (two jobs $\leq p_2 = 2$): $y[2] = \max(10, 13) = 13$.

Solution $\bar{x}_{11} = \bar{x}_{22} = \bar{x}_{32} = 1$, $\bar{y} = (6, 13, 0)$, value 19: $z(MILP) \leq 19$.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing (7.8e) to $x_{jm} \geq 0$, with duals μ_j free (7.8b), $\pi_m \leq 0$ (7.8c, $\leq$ in a minimisation) and $\lambda_{jm} \geq 0$ (7.8d, $\geq$):

Dual of the LP relaxation of model 7.4

$$\max \sum_{j=1}^n \mu_j + \sum_{m=1}^k p_m \pi_m \quad (7.9a)$$

$$\text{subject to } \mu_j + \pi_m - t_{jm} \lambda_{jm} \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}, \quad (7.9b)$$

$$\sum_{j=1}^n \lambda_{jm} \leq 1, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9c)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.9d)$$

$$\pi_m \leq 0, \quad \forall m \in \{1, 2, \dots, k\}, \quad (7.9e)$$

$$\lambda_{jm} \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \forall m \in \{1, 2, \dots, k\}. \quad (7.9f)$$

Description. The variable μ_j is the value of job j , π_m the (non-positive) price of one of the p_m slots of machine m , and λ_{jm} the price of the link that pushes the maximum y_m above t_{jm} . The objective (7.9a) prices the jobs and the available slots. Constraints (7.9b) are the columns of the x_{jm} : putting job j on machine m satisfies the assignment and takes one slot, but forces the maximum of the machine up to t_{jm} ; the balance cannot be positive, because the x_{jm} do not appear in the objective. Constraints (7.9c) are the columns of the y_m : the maximum of machine m costs 1 in the primal objective, and the prices of the links that push it up cannot be worth more.

Constraint (7.9c) is the reduced cost of y_m : the coefficient 1 in the primal objective limits the sum of the λ_{jm} . **A hand-built dual solution.** Split that unit in equal parts, $\bar{\lambda}_{jm} = 1/n = 1/3$ for every pair, and set $\bar{\pi}_m = 0$; constraints (7.9b) become $\mu_j \leq t_{jm}/3$ for every m :

$$\bar{\mu}_1 = \min\{6, 5, 3\}/3 = 1, \quad \bar{\mu}_2 = \min\{5, 10, 2\}/3 = \frac{2}{3}, \quad \bar{\mu}_3 = \min\{20, 13, 10\}/3 = \frac{10}{3},$$

value $1 + \frac{2}{3} + \frac{10}{3} = 5$. Hence $5 \leq z(MILP) \leq 19$.

What the solver says. $z(LP) = 520/49 = 10.61$ (equal to the optimum of the dual; the relaxation with the bounds coincides, because $x_{jm} \leq 1$ is implied). The integer optimum is $z(MILP) = 15$: job 1 on machine 2, jobs 2 and 3 on machine 3, $\tilde{y} = (0, 5, 10)$. The hand bound (5) is far from the relaxation (10.61): the uniform split $\lambda_{jm} = 1/3$ is the first that comes to mind, not the best — the optimal dual distributes the unit of every machine differently.

LB (next-fit)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
19	5	520/49	15	26.7%

Additional considerations

- $x_{jm} \leq 1$ ($n k$ valid inequalities) is implied by (7.8b).
- The direction “no job $\Rightarrow y_m = 0$ ” can be imposed explicitly with an upper bound: with $\bar{t}_m = \max_j t_{jm}$, the constraints

$$\bar{t}_m \sum_{j=1}^n x_{jm} - y_m \geq 0, \quad \forall m \in \{1, 2, \dots, k\}$$

(k constraints) force $y_m = 0$ if the machine is empty and leave y_m room up to $\bar{t}_m$ otherwise. They are *not* valid inequalities (the model allows $y_m > 0$ on an empty machine) but they are *optimality-preserving constraints*: they can be added because the objective minimises $\sum_{m=1}^k y_m$. The distinction between “valid” and “optimality-preserving” is the same as between the two bullets of the link.

Additional modelling questions

Modelling question 7.7 — minimising the time of the slowest machine

The machines all work together and the company wants to finish as early as possible: minimise the *maximum* of the processing times of the machines (the makespan), not the sum. Model and solve.

Modelling question 7.8 — fixed cost if the machine works

Switching a machine on costs $g_m = 4$ euros (once) and one minute of processing costs 1 euro: minimise the total cost. Which link is needed and what is the smallest big-M?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	19	heuristic solution (UB)
LB	5	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 26.7\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 93.3\%.$$

The sandwich on variant 4a

Minimising the *maximum* of the times instead of the sum removes the cost from the y_m and adds a family $\nu_m \geq 0$ for the constraints $w \geq y_m$, with the column of w imposing $\sum_m \nu_m \leq 1$. It pays to price a single job, the longest one: the ν maximising $\min_m t_{jm} \nu_m$ makes that product constant, that is ν_m proportional to $1/t_{jm}$, and the bound is the harmonic mean of the times of that job. The recipe turns out to be *optimal* for the relaxation.

	value	what it is
UB	13	heuristic solution
LB	$\frac{260}{59}$	dual certificate built by hand
$z(LP)$	$\frac{260}{59}$	relaxation without the bounds
$z(LP^+)$	$\frac{260}{59}$	relaxation with the bounds
$z(MILP)$	10	optimum of the MILP

7.5 One machine, job classes with setup

Script: `python/fam07_5_classessetup.py`

[Notebook in Colab](#)

Problem 7.5 — classes with setup

A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes. For each job $j \in \{1, 2, \dots, n\}$, the value $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and the value $r_j \in \mathbb{Q}_{>0}$ is the revenue in euros if the job is executed. The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes: for each class $c \in \{1, 2, \dots, q\}$, the set $\mathcal{J}_c \subseteq \{1, 2, \dots, n\}$ contains the jobs of the class. If the machine executes jobs of a class c , a setup cost of $f_c \in \mathbb{Q}_{\geq 0}$ euros is incurred and a setup time of $s_c \in \mathbb{Q}_{\geq 0}$ minutes is needed to prepare the machine. The machine cannot execute jobs in parallel. The company wants to choose the jobs to execute maximising the profit.

The problem in words. We decide which jobs to execute and, consequently, which classes to activate. *The objective:* revenues minus setup costs. *The constraint:* times of the jobs plus setup times of the activated classes within the availability. A knapsack with fixed costs per group: the activation link, this time *disaggregated* from the start.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c (partition)
f_c, s_c	$\in \mathbb{Q}_{\geq 0}$	setup cost and setup time of class c

Decision variables

We introduce the following $n + q$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed, } 0 \text{ otherwise,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ if at least one job of class } c \text{ is executed, } 0 \text{ otherwise,} & \forall c \in \{1, 2, \dots, q\}. \end{cases}$$

Model 7.5 — classes with setup

$$\max \sum_{j=1}^n r_j x_j - \sum_{c=1}^q f_c y_c \quad (7.10a)$$

$$\text{subject to } \sum_{j=1}^n t_j x_j + \sum_{c=1}^q s_c y_c \leq a, \quad (7.10b)$$

$$x_j - y_c \leq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.10c)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.10d)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}. \quad (7.10e)$$

Description of the objective function and constraints

- the linear objective function (7.10a) maximises the profit, the difference between the revenue of the executed jobs and the setup cost of the activated classes;
- the linear constraint (7.10b) ensures that the time of the executed jobs, plus the setup times of the activated classes, does not exceed the availability of the machine (1 linear constraint);
- the linear constraints (7.10c) link the execution variables with the class-activation variables: if a job of a class is executed, the class is activated (n linear constraints, one per job, because the classes partition the jobs);
- the constraints (7.10d) and (7.10e) define the variables of the model.

Link between the variables

Both families are binary: the link constraint is the direct linearisation of a Boolean implication (Section “From CNF to linear constraints”).

- For each class c , if at least one job of the class is executed, the class is activated:

$$\underbrace{(x_j \text{ OR } x_s \text{ OR } \dots)}_{j,s,\dots \in \mathcal{J}_c} \implies y_c,$$

and by contraposition (De Morgan)

$$\text{NOT } y_c \implies \underbrace{(\text{NOT } x_j \text{ AND } \text{NOT } x_s \text{ AND } \dots)}_{j,s,\dots \in \mathcal{J}_c} :$$

if the class is not activated, none of its jobs is executed.

The implication is equivalent to the Boolean expression

$$\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \text{ OR } y_c;$$

by De Morgan and the distributive property it becomes the CNF

$$(\text{NOT } x_j \text{ OR } y_c) \text{ AND } (\text{NOT } x_s \text{ OR } y_c) \text{ AND } \dots,$$

which translates into the $|\mathcal{J}_c|$ linear constraints $1 - x_j + y_c \geq 1$, i.e. $x_j - y_c \leq 0$ for every $j \in \mathcal{J}_c$: exactly constraints (7.10c). Check in both directions: if $x_j = 1$ for some $j \in \mathcal{J}_c$, the constraint $x_j \leq y_c$ forces $y_c = 1$; if $y_c = 0$, the constraints $x_j \leq y_c$ force $x_j = 0$ for every $j \in \mathcal{J}_c$.

- Conversely, if no job of the class is executed, the class is not activated:

$$\text{NOT}(x_j \text{ OR } x_s \text{ OR } \dots) \implies \text{NOT } y_c,$$

$$\text{by contraposition: } y_c \implies (x_j \text{ OR } x_s \text{ OR } \dots).$$

An activated class has at least one executed job.

It is not imposed by the constraints, but it follows *without loss of optimality* from the objective and the capacity constraint: if $x_j = 0$ for every $j \in \mathcal{J}_c$ and $y_c = 1$, setting $y_c = 0$ stays feasible, frees s_c minutes in constraint (7.10b) and *does not decrease* the objective, because $f_c \geq 0$. Since f_c may be zero, the improvement is not necessarily strict: the correct conclusion is “there always exists an optimum in which classes with no jobs are not activated”, not “in every optimum”. The sign of the data decides the strength of the conclusion.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

The classes are lists of indices: $J = [[0, 1], [2, 3], [4, 5, 6]]$.

```
1 m = gp.Model("classes_setup"); m.Params.OutputFlag = 0
2 x = m.addVars(n, vtype=GRB.BINARY, name="x")
3 y = m.addVars(q, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
5                 - gp.quicksum(f[c] * y[c] for c in range(q)), GRB.
                   MAXIMIZE)
```

```

6 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n))
7             + gp.quicksum(s[c] * y[c] for c in range(q)) <= a,
8             name="availability")
9 m.addConstrs((x[j] - y[c] <= 0 for c in range(q) for j in J[c]),
10             name="link")
11 m.optimize()

```

The instance

$n = 7$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6, 7\}$, availability $a = 50$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$	$j = 7$
r_j	10	6	8	6	7	9	5
t_j	5	10	8	6	9	5	6

	$c = 1$	$c = 2$	$c = 3$
f_c	10	5	4
s_c	10	12	6

Model (7.10) for the instance:

The model of the instance

$$\begin{aligned}
 & \max 10x_1 + 6x_2 + 8x_3 + 6x_4 + 7x_5 + 9x_6 + 5x_7 - 10y_1 - 5y_2 - 4y_3 \\
 & \text{subject to } 5x_1 + 10x_2 + 8x_3 + 6x_4 + 9x_5 + 5x_6 + 6x_7 + 10y_1 + 12y_2 + 6y_3 \leq 50 \\
 & \quad x_1 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 & \quad \quad x_2 \qquad \qquad \qquad -y_1 \qquad \qquad \leq 0 \\
 & \quad \quad \quad x_3 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 & \quad \quad \quad \quad x_4 \qquad \qquad \qquad -y_2 \qquad \leq 0 \\
 & \quad \quad \quad \quad \quad x_5 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad \quad \quad \quad \quad \quad x_6 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad \quad \quad \quad \quad \quad \quad x_7 \qquad \qquad \qquad -y_3 \leq 0 \\
 & \quad x_1, \quad x_2, \quad x_3, \quad x_4, \quad x_5, \quad x_6, \quad x_7 \qquad \in \{0, 1\} \\
 & \quad \quad \quad \quad \quad \quad y_1, \quad y_2, \quad y_3 \in \{0, 1\}
 \end{aligned}$$

Constructive heuristic: the primal bound

Class by class, job by job: the first job of a class is executed if its time and the setup of the class fit; the following ones if their time fits.

- **Step 1.** Class 1 not active: $s_1 + t_1 = 10 + 5 = 15 \leq 50$; $y[1] = 1$, $x[1] = 1$, $ra = 35$.
- **Step 2.** Class 1 active: $t_2 = 10 \leq 35$; $x[2] = 1$, $ra = 25$.
- **Step 3.** Class 2 not active: $s_2 + t_3 = 12 + 8 = 20 \leq 25$; $y[2] = 1$, $x[3] = 1$, $ra = 5$.
- **Step 4.** Class 2 active: $t_4 = 6 > 5$; job 4 is skipped.
- **Steps 5–7.** Class 3 not active: $s_3 + t_j \in \{15, 11, 12\}$, all > 5 ; jobs 5, 6, 7 are skipped.

Solution $\bar{x} = (1, 1, 1, 0, 0, 0, 0)$, $\bar{y} = (1, 1, 0)$, profit $10 + 6 + 8 - 10 - 5 = 9$: $z(MILP) \geq 9$.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing $x_j \geq 0$ and $y_c \geq 0$, with duals $\pi \geq 0$ for constraint (7.10b) and $\lambda_j \geq 0$ for each constraint (7.10c):

Dual of the LP relaxation of model 7.5

$$\min \quad a\pi \quad (7.11a)$$

$$\text{subject to} \quad t_j\pi + \lambda_j \geq r_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.11b)$$

$$s_c\pi - \sum_{j \in \mathcal{J}_c} \lambda_j \geq -f_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.11c)$$

$$\pi \geq 0, \quad (7.11d)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.11e)$$

Description. The variable π is the price of one unit of time and λ_j the price of the link between job j and the setup of its class. The objective (7.11a) is the value of all the available time a . Constraints (7.11b) are the columns of the x_j : the time t_j that job j consumes, priced at π , plus the price of its link, must cover the revenue r_j . Constraints (7.11c) are the columns of the y_c : the setup of class c consumes s_c of time and releases the links of all the jobs in the class; the balance cannot fall below $-f_c$, that is, below the cost of that setup.

For the instance: $\min 50\pi$ subject to $5\pi + \lambda_1 \geq 10$, $10\pi + \lambda_2 \geq 6$, $8\pi + \lambda_3 \geq 8$, $6\pi + \lambda_4 \geq 6$, $9\pi + \lambda_5 \geq 7$, $5\pi + \lambda_6 \geq 9$, $6\pi + \lambda_7 \geq 5$, $10\pi - \lambda_1 - \lambda_2 \geq -10$, $12\pi - \lambda_3 - \lambda_4 \geq -5$, $6\pi - \lambda_5 - \lambda_6 - \lambda_7 \geq -4$, $\pi, \lambda \geq 0$.

A hand-built dual solution. With $\bar{\lambda}_j = 0$ constraints (7.11b) ask $\pi \geq r_j/t_j$ for every job, and constraints (7.11c) are satisfied for every $\pi \geq 0$:

$$\bar{\pi} = \max_j \frac{r_j}{t_j} = \max\left\{\frac{10}{5}, \frac{6}{10}, \frac{8}{8}, \frac{6}{6}, \frac{7}{9}, \frac{9}{5}, \frac{5}{6}\right\} = 2,$$

value $50 \cdot 2 = 100$: $9 \leq z(MILP) \leq 100$. The reading: π is the value of one minute of machine; if every minute were worth as much as the most profitable job per minute, the whole machine would be worth 100 euros. A coarse bound, as “knapsack” bounds often are: it ignores setups and costs.

What the solver says. $z(LP) = 425/13 = 32.7$ (optimum of the dual, reached with $\pi = \frac{17}{26}$ and some $\lambda_j > 0$: the solver does not settle for $\lambda = 0$); with the upper bounds, $z(LP^+) = 329/13 = 25.3$. The integer optimum is $z(MILP) = 21$: classes 2 and 3 activated, jobs 3, 4, 5, 6 executed ($8 + 6 + 9 + 5 + 12 + 6 = 46 \leq 50$), profit $30 - 9$. The heuristic, which opens the classes in the given order and spends 10 + 10 minutes on class 1, stays at 9: a 57% gap, the worst of the family — the scanning order matters, and a constructive heuristic that orders the classes by potential profit would do better.

LB (constructive heuristic)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
9	100	425/13	329/13	21	57.1%

Additional considerations

- $y_c \leq 1$ (q) are valid inequalities that strengthen the relaxation; $x_j \leq 1$ (n) are implied by $y_c \leq 1$ and (7.10c).
- The direction “no job $\Rightarrow$ class not activated” is the clause

$$\text{NOT } y_c \text{ OR } x_j \text{ OR } x_s \text{ OR } \dots,$$

i.e. $\sum_{j \in \mathcal{J}_c} x_j \geq y_c$ (q constraints): not valid, but it can be added without losing the optimum (for $f_c \geq 0$ and $s_c \geq 0$).

- The aggregated form of the link, $\sum_{j \in \mathcal{J}_c} x_j \leq |\mathcal{J}_c| y_c$, has the same set of integer solutions and a weaker relaxation: here the model is born already in the disaggregated form, the right one.

Additional modelling questions

Modelling question 7.9 — a single class

Changing class requires a cleaning of the machine the company wants to avoid altogether: at most one class can be activated. Model and solve.

Modelling question 7.10 — a class subordinate to another

The jobs of class 3 use a tool that is mounted during the setup of class 1: class 3 can be activated only if class 1 is activated too. Write the constraint and find the new optimum.

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	9	heuristic solution (LB)
UB	100	certified bound (UB)
$z(MILP)$	21	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 57.1\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 433.3\%.$$

The sandwich on variant 5a

"At most one class activated" adds a variable $\theta \geq 0$ with right-hand side 1 to the dual: it enters the objective and loosens the columns of the y_c . Once the price of time π is fixed, the λ_j and θ are forced by the dual constraints, so the recipe reduces to searching π among the ratios r_j/t_j and keeping the lowest value. The bound improves a great deal over the recipe of the base problem.

	value	what it is
UB	$\frac{157}{5}$	dual certificate built by hand
LB	17	heuristic solution
$z(LP)$	17	relaxation without the bounds
$z(LP^+)$	17	relaxation with the bounds
$z(MILP)$	17	optimum of the MILP

7.6 Classes with completion bonus and “if and only if” reduction

Script: `python/fam07_6_classesbonus.py`

[Notebook in Colab](#)

Problem 7.6 — completion bonus

A company has $n \in \mathbb{Z}_{\geq 1}$ jobs that can be executed on a machine with an available processing time of $a \in \mathbb{Q}_{>0}$ minutes. For each job $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and $r_j \in \mathbb{Q}_{>0}$ the revenue in euros if executed. The jobs are partitioned into $q \in \mathbb{Z}_{\geq 2}$ classes $\mathcal{J}_1, \dots, \mathcal{J}_q$. The availability of the machine is reduced by $u \in \mathbb{Q}_{>0}$ minutes *if and only if* the machine executes jobs of at least two different classes. For each class $c \in \{1, 2, \dots, q\}$, an extra revenue of $v_c \in \mathbb{Q}_{>0}$ euros is obtained *if and only if* all the jobs of the class are executed. The machine does not execute jobs in parallel. The company wants to choose the jobs to execute maximising the total revenue.

The problem in words. We decide which jobs to execute. *The objective:* revenues of the jobs plus bonuses of the complete classes. *The constraint:* times of the jobs, plus the reduction u if the classes involved are at least two, within the availability. Two “if and only if”s: one rewards (completion), one penalises (mixing). We shall see that for each it suffices to impose one direction: the objective imposes the other.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
a	$\in \mathbb{Q}_{>0}$	availability of the machine
t_j, r_j	$\in \mathbb{Q}_{>0}$	time and revenue of job j
q	$\in \mathbb{Z}_{\geq 2}$	number of classes, $c \in \{1, 2, \dots, q\}$
$\mathcal{J}_c$	$\subseteq \{1, 2, \dots, n\}$	jobs of class c (partition)
v_c	$\in \mathbb{Q}_{>0}$	bonus if all the jobs of class c are executed
u	$\in \mathbb{Q}_{>0}$	reduction of the availability if jobs of at least two classes are executed

Decision variables

We introduce the following $n + q + 1$ binary variables:

$$\begin{cases} x_j = 1 \text{ if job } j \text{ is executed, } 0 \text{ otherwise,} & \forall j \in \{1, 2, \dots, n\}, \\ y_c = 1 \text{ if all the jobs of class } c \text{ are executed, } 0 \text{ otherwise,} & \forall c \in \{1, 2, \dots, q\}, \\ z = 1 \text{ if jobs of at least two different classes are executed, } 0 \text{ otherwise.} \end{cases}$$

Model 7.6 — completion bonus

$$\max \sum_{j=1}^n r_j x_j + \sum_{c=1}^q v_c y_c \quad (7.12a)$$

$$\text{subject to } x_j - y_c \geq 0, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.12b)$$

$$x_j + x_i - z \leq 1, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.12c)$$

$$\sum_{j=1}^n t_j x_j + u z \leq a, \quad (7.12d)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.12e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.12f)$$

$$z \in \{0, 1\}. \quad (7.12g)$$

Description of the objective function and constraints

- the linear objective function (7.12a) maximises the total revenue, the sum of the revenues of the executed jobs and of the bonuses of the complete classes;
- the linear constraints (7.12b) link the execution variables with the completion variables: if a class is declared complete, all its jobs are executed (n linear constraints);
- the linear constraints (7.12c) link the execution variables with the mixing variable: if two jobs of different classes are executed, then $z = 1$ ($\sum_{c < g} |\mathcal{J}_c| |\mathcal{J}_g|$ linear constraints, one per pair of jobs of different classes);
- the linear constraint (7.12d) ensures that the time of the executed jobs, plus the reduction u when classes are mixed, does not exceed the availability (1 linear constraint);
- the constraints (7.12e)–(7.12g) define the variables of the model.

Link between the variables

Four implications: two for y_c , two for z . In each pair one is imposed by the constraint, the other by the optimum — and that is how an “if and only if” is modelled (Section “If and only if”).

- For each class c , if not all the jobs of the class are executed, the class is not complete:

$$\text{NOT } (x_j \text{ AND } x_i \text{ AND } \dots) \implies \text{NOT } y_c,$$

$$\text{by contraposition: } y_c \implies (x_j \text{ AND } x_i \text{ AND } \dots),$$

with $j, i, \dots \in \mathcal{J}_c$. The Boolean expression becomes, by distributivity, the CNF

$$(x_j \text{ AND } x_i \text{ AND } \dots) \text{ OR NOT } y_c \\ \equiv (x_j \text{ OR NOT } y_c) \text{ AND } (x_i \text{ OR NOT } y_c) \text{ AND } \dots,$$

i.e. $x_j + (1 - y_c) \geq 1$: these are constraints (7.12b), $x_j - y_c \geq 0$ for every $j \in \mathcal{J}_c$. Check: if $y_c = 1$, every constraint $x_j \geq y_c$ forces $x_j = 1$; if some $x_j = 0$ with $j \in \mathcal{J}_c$, the constraint $x_j \geq y_c$ forces $y_c = 0$.

- Conversely, if all the jobs of the class are executed, the class is complete:

$$(x_j \text{ AND } x_i \text{ AND } \dots) \implies y_c, \\ \text{contrapositive: } \text{NOT } y_c \implies (\text{NOT } x_j \text{ OR NOT } x_i \text{ OR } \dots).$$

Not imposed by the constraints, it follows from the objective in every optimum: since $v_c > 0$, if all the x_j of the class equal 1 and $y_c = 0$, setting $y_c = 1$ stays feasible (constraints (7.12b) ask $x_j \geq 1$, true) and increases the objective by v_c . Hence in every optimum the bonus is collected as soon as the class is complete.

- For each pair of jobs j, i of different classes, if both are executed then $z = 1$:

$$x_j \text{ AND } x_i \implies z, \\ \text{contrapositive: } \text{NOT } z \implies \text{NOT } (x_j \text{ AND } x_i).$$

If $z = 0$, two jobs of different classes are not both executed. Boolean expression

$$\text{NOT } (x_j \text{ AND } x_i) \text{ OR } z \text{ and, by De Morgan, } \text{NOT } x_j \text{ OR NOT } x_i \text{ OR } z,$$

i.e. $(1 - x_j) + (1 - x_i) + z \geq 1$: these are constraints (7.12c), $x_j + x_i - z \leq 1$. Check: if $x_j = x_i = 1$ the constraint forces $z = 1$; if $z = 0$ it becomes $x_j + x_i \leq 1$.

- Conversely, if the executed jobs all lie in one class, $z = 0$; by contraposition, $z = 1$ implies that for every class c at least one executed job lies outside $\mathcal{J}_c$. Not imposed by the constraints, it follows from the capacity constraint *without loss of optimality*: if the executed jobs lie in a single class and $z = 1$, setting $z = 0$ stays feasible (constraints (7.12c) become $x_j + x_i \leq 1$, true because no mixed pair is executed), frees $u > 0$ minutes in constraint (7.12d) and does not change the objective, where z does not appear. Hence there always exists an optimum with $z = 1$ only if classes are mixed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```
1 pairs = [(j, i) for c in range(q) for g in range(c + 1, q)
2           for j in J[c] for i in J[g]]
3 m = gp.Model("classes_bonus"); m.Params.OutputFlag = 0
4 x = m.addVars(n, vtype=GRB.BINARY, name="x")
5 y = m.addVars(q, vtype=GRB.BINARY, name="y")
```

```

6 z = m.addVar(vtype=GRB.BINARY, name="z")
7 m.setObjective(gp.quicksum(r[j] * x[j] for j in range(n))
8               + gp.quicksum(v[c] * y[c] for c in range(q)), GRB.
               MAXIMIZE)
9 m.addConstrs((x[j] - y[c] >= 0 for c in range(q) for j in J[c]),
10             name="all")
11 m.addConstrs((x[j] + x[i] - z <= 1 for (j, i) in pairs), name="mixed")
12 m.addConstr(gp.quicksum(t[j] * x[j] for j in range(n)) + u * z <= a,
13             name="availability")
14 m.optimize()

```

The instance

$n = 6$ jobs, $q = 3$ classes $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6\}$, $a = 50$, $u = 10$:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$		$c = 1$	$c = 2$	$c = 3$
r_j	10	5	20	12	10	22	v_c	5	4	10
t_j	5	15	25	15	10	38				

The model for the instance has objective $\max 10x_1 + 5x_2 + 20x_3 + 12x_4 + 10x_5 + 22x_6 + 5y_1 + 4y_2 + 10y_3$, six constraints (7.12b) ($x_1 \geq y_1$, $x_2 \geq y_1$, $x_3 \geq y_2$, $x_4 \geq y_2$, $x_5 \geq y_3$, $x_6 \geq y_3$), twelve constraints (7.12c) ($x_1 + x_3 - z \leq 1$, $x_1 + x_4 - z \leq 1$, $x_2 + x_3 - z \leq 1$, $x_2 + x_4 - z \leq 1$, $x_1 + x_5 - z \leq 1$, $x_1 + x_6 - z \leq 1$, $x_2 + x_5 - z \leq 1$, $x_2 + x_6 - z \leq 1$, $x_3 + x_5 - z \leq 1$, $x_3 + x_6 - z \leq 1$, $x_4 + x_5 - z \leq 1$, $x_4 + x_6 - z \leq 1$) and the capacity constraint $5x_1 + 15x_2 + 25x_3 + 15x_4 + 10x_5 + 38x_6 + 10z \leq 50$.

Constructive heuristic: the primal bound

Class by class; from the second class on, the first executed job also pays the reduction u ; at the end of a class, if all its jobs are executed, the bonus is collected.

- **Steps 1–2.** Class 1: $t_1 = 5 \leq 50$, $x[1] = 1$, $ra = 45$; $t_2 = 15 \leq 45$, $x[2] = 1$, $ra = 30$. Class complete: $y[1] = 1$.
- **Step 3.** Class 2, reduction not applied yet: $t_3 + u = 35 > 30$, job 3 is skipped.
- **Step 4.** $t_4 + u = 25 \leq 30$: $x[4] = 1$, $z = 1$, $ra = 5$. Class 2 incomplete.
- **Steps 5–6.** Class 3: $t_5 = 10 > 5$ and $t_6 = 38 > 5$, skipped.

Solution $\bar{x} = (1, 1, 0, 1, 0, 0)$, $\bar{y} = (1, 0, 0)$, $\bar{z} = 1$, revenue $10 + 5 + 12 + 5 = 32$: $z(MILP) \geq 32$. It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing all the binaries to ≥ 0 , with duals $\pi_j \leq 0$ for constraints (7.12b) ($\geq$ in a maximisation), $\lambda_{ji} \geq 0$ for constraints (7.12c) and $\mu \geq 0$ for constraint (7.12d):

Dual of the LP relaxation of model 7.6

$$\min \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + a \mu \quad (7.13a)$$

$$\text{subject to } \pi_j + \sum_{i \notin \mathcal{J}_c} \lambda_{ji} + t_j \mu \geq r_j, \quad \forall c \in \{1, 2, \dots, q\}, \forall j \in \mathcal{J}_c, \quad (7.13b)$$

$$- \sum_{j \in \mathcal{J}_c} \pi_j \geq v_c, \quad \forall c \in \{1, 2, \dots, q\}, \quad (7.13c)$$

$$- \sum_{c < g} \sum_{j \in \mathcal{J}_c} \sum_{i \in \mathcal{J}_g} \lambda_{ji} + u \mu \geq 0, \quad (7.13d)$$

$$\pi_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.13e)$$

$$\lambda_{ji} \geq 0, \quad \forall c, g \in \{1, 2, \dots, q\}, c < g, \forall j \in \mathcal{J}_c, \forall i \in \mathcal{J}_g, \quad (7.13f)$$

$$\mu \geq 0. \quad (7.13g)$$

Description. The variable π_j is the (non-positive) price of the link "if the class is rewarded, then job j must be done", λ_{ji} the price of the constraint on the mixed pair (j, i) , and μ the price of one unit of time. The objective (7.13a) adds up the prices of all the mixed pairs and the value of the available time a . Constraints (7.13b) are the columns of the x_j : doing job j releases the link with its class, touches all the mixed-pair constraints that contain it and consumes t_j of time; the total must cover the revenue r_j . Constraints (7.13c) are the columns of the y_c : the bonus v_c of class c must be covered by the links of the jobs that make it up. Constraint (7.13d) is the column of z : the variable that flags the presence of two different classes releases all the mixed-pair constraints and consumes u of time.

(In (7.13b) λ_{ji} denotes the variable of the pair $\{j, i\}$, whatever the order of the two indices.)

A hand-built dual solution. Constraints (7.13c) ask the π_j of every class to "cover" the bonus: it suffices to load it on a single job per class, $\bar{\pi}_1 = -5$, $\bar{\pi}_3 = -4$, $\bar{\pi}_5 = -10$, the others 0. With $\bar{\lambda} = 0$ constraint (7.13d) is satisfied for every $\mu \geq 0$ and constraints (7.13b) ask $\mu \geq (r_j - \bar{\pi}_j)/t_j$:

$$\bar{\mu} = \max \left\{ \frac{10+5}{5}, \frac{5}{15}, \frac{20+4}{25}, \frac{12}{15}, \frac{10+10}{10}, \frac{22}{38} \right\} = \max \left\{ 3, \frac{1}{3}, \frac{24}{25}, \frac{4}{5}, 2, \frac{11}{19} \right\} = 3,$$

value $a\bar{\mu} = 150$: $32 \leq z(MILP) \leq 150$. The bound is dominated by job 1, short and with the bonus loaded on it: spreading the bonus over the two jobs of the class ($\pi_1 = \pi_2 = -2.5$) would lower it — a good exercise.

What the solver says. $z(LP) = z(LP^+) = 5280/113 = 46.7$. The integer optimum is $z(MILP) = 42$: class 3 alone, complete, jobs 5 and 6 ($10 + 38 = 48 \leq 50$, $z = 0$), revenue $10 + 22 + 10$. The heuristic, started from class 1, stops at 32: a 24% gap.

LB (constructive heuristic)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
32	150	5280/113	42	23.8%

Additional considerations

- The optimality direction for y_c can be imposed explicitly with the clause

$$\text{NOT } x_j \text{ OR NOT } x_i \text{ OR } \dots \text{ OR } y_c,$$

i.e.

$$\sum_{j \in \mathcal{J}_c} x_j - y_c \leq |\mathcal{J}_c| - 1, \quad \forall c \in \{1, 2, \dots, q\}$$

(q optimality-preserving constraints, not valid: the model allows the complete class with $y_c = 0$).

- The optimality direction for z : for every class c , $z \leq \sum_{j \notin \mathcal{J}_c} x_j$ (q constraints), from $z \implies (x_j \text{ OR } x_i \text{ OR } \dots)$ over the jobs outside $\mathcal{J}_c$. These too preserve the optimum without being valid.
- Constraints (7.12c) are as many as the mixed pairs. An aggregated form with class variables, $w_c \geq x_j$ for $j \in \mathcal{J}_c$ and $\sum_{c=1}^q w_c - 1 \leq (q-1)z$, uses q more variables and $n+1$ constraints instead of the pairs: same set of integer solutions, different relaxation. Comparing them is question 7.6.2.

Additional modelling questions

Modelling question 7.11 — at least one job per class

A union agreement requires executing at least one job of every class. Model, say what happens to z and find the new optimum.

Modelling question 7.12 — penalty for a class started and not finished

Starting a class without completing it costs $w = 3$ euros of reorganisation. Model the penalty: which new variable is needed, and in which direction must its link be proved?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	32	heuristic solution (LB)
UB	150	certified bound (UB)
$z(MILP)$	42	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 23.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 281.0\%.$$

The sandwich on variant 6a

"At least one job per class" adds a family $\omega_c \leq 0$ to the dual, one per class, which enters the objective: being non-positive, it *lowers* it. Each ω_c is pushed down as far as the columns of that class allow. The heuristic starts from the shortest job of each class — so it is feasible by construction — and then fills the remaining time.

	value	what it is
UB	107	dual certificate built by hand
LB	32	heuristic solution
$z(LP)$	$\frac{309}{7}$	relaxation without the bounds
$z(LP^+)$	$\frac{309}{7}$	relaxation with the bounds
$z(MILP)$	40	optimum of the MILP

7.7 Total tardiness on one machine: sequencing with big-M

Script: `python/fam07_7_tardiness.py`

[Notebook in Colab](#)

Problem 7.7 — total tardiness

A company needs to process $n \in \mathbb{Z}_{\geq 1}$ jobs on a single machine. For each job $j \in \{1, 2, \dots, n\}$, $t_j \in \mathbb{Q}_{>0}$ is the processing time in minutes and $d_j \in \mathbb{Q}_{>0}$ the due date in minutes. The tardiness of job j is $\tau_j = \max\{0, \kappa_j - d_j\}$, where κ_j is the completion time. The machine processes one job at a time. The company wants a sequence of the jobs that minimises the total tardiness.

The problem in words. *We decide* the order of the jobs. *The objective:* the sum of the tardiness values. *The constraints:* for every pair, one of the two precedes the other; whoever comes later finishes at least t minutes after the completion of whoever comes first. The order is a *disjunction* ("either j before i or i before j "): it is linearised with a binary variable and a big-M.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of jobs, $j \in \{1, 2, \dots, n\}$
t_j	$\in \mathbb{Q}_{>0}$	processing time of job j
d_j	$\in \mathbb{Q}_{>0}$	due date of job j
M	$\in \mathbb{Q}_{>0}$	big-M constant, $M = \sum_{j=1}^n t_j$

Decision variables

We introduce $n(n-1)$ binary precedence variables and $2n$ non-negative variables:

$$\begin{cases} s_{ji} = 1 \text{ if job } j \text{ precedes job } i, 0 \text{ otherwise,} & \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \\ \kappa_j = \text{completion time of job } j, & \forall j \in \{1, 2, \dots, n\}, \\ \tau_j = \text{tardiness of job } j, & \forall j \in \{1, 2, \dots, n\}. \end{cases}$$

Model 7.7 — total tardiness

$$\min \sum_{j=1}^n \tau_j \quad (7.14a)$$

$$\text{subject to } s_{ji} + s_{ij} = 1, \quad \forall j, i \in \{1, 2, \dots, n\}, j < i, \quad (7.14b)$$

$$-M s_{ji} - \kappa_j + \kappa_i \geq t_i - M, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14c)$$

$$-\kappa_j + \tau_j \geq -d_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14d)$$

$$\kappa_j \geq t_j, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14e)$$

$$s_{ji} \in \{0, 1\}, \quad \forall j, i \in \{1, 2, \dots, n\}, j \neq i, \quad (7.14f)$$

$$\kappa_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.14g)$$

$$\tau_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.14h)$$

Description of the objective function and constraints

- the linear objective function (7.14a) minimises the total tardiness;
- the linear constraints (7.14b) link the precedence variables of every pair: either j precedes i or i precedes j ($n(n-1)/2$ linear constraints);
- the linear constraints (7.14c) link the precedence variables with the completion variables through the big-M: if j precedes i , job i cannot be completed before j is completed and i has been processed ($n(n-1)$ linear constraints). A valid value is $M = \sum_{j=1}^n t_j$: there always exists an optimal sequence without idle time, hence no completion needs to exceed the total time;
- the linear constraints (7.14d), together with constraints (7.14h), link the completions with the tardiness values and correctly define the tardiness of every job (n linear constraints);
- the linear constraints (7.14e) impose that every job completes no earlier than its processing time (n linear constraints);
- the constraints (7.14f)–(7.14h) define the variables of the model.

Link between the variables

- For each pair j, i : if j precedes i , the completion of i is at least the completion of j plus the time of i :

$$s_{ji} = 1 \implies \kappa_i \geq \kappa_j + t_i,$$

$$\text{and by contraposition: } \kappa_i < \kappa_j + t_i \implies s_{ji} = 0.$$

It is imposed with the big-M: $\kappa_i \geq \kappa_j + t_i - M(1 - s_{ji})$, i.e. constraints (7.14c). If $s_{ji} = 1$ the constraint becomes $\kappa_i \geq \kappa_j + t_i$; if $s_{ji} = 0$ it becomes $\kappa_i \geq \kappa_j + t_i - M$. The model does not impose $\kappa_j \leq M$, so the constraint is not true at *every* feasible

point; it is true at the ones among which the optimum is sought. With $M = \sum_{j=1}^n t_j$ there is always an optimal solution with no idle time — if the machine stops, the later jobs are pulled forward and no completion, hence no tardiness, gets worse — and in such a solution every $\kappa_j \leq \sum_{j=1}^n t_j = M$, whence $\kappa_j + t_i - M \leq t_i \leq \kappa_i$. This is the pattern of Section “Big-M: conditional constraints and disjunctions”: the big-M “switches off” the constraint.

- For each job j , the tardiness variable satisfies the quantitative bound

$$\tau_j \geq \max\{0, \kappa_j - d_j\},$$

imposed directly by constraints (7.14d) ($\tau_j \geq \kappa_j - d_j$) and (7.14h) ($\tau_j \geq 0$), whatever κ_j and d_j : there is no implication to prove, the link *is* the inequality. There is instead an optimality implication: if the job finishes on time, the tardiness is zero:

$$\kappa_j \leq d_j \implies \tau_j = 0,$$

$$\text{contrapositive: } \tau_j > 0 \implies \kappa_j > d_j.$$

The constraints do not impose it (with $\kappa_j \leq d_j$ they only ask $\tau_j \geq 0$), but it follows from the objective in every optimum: if $\kappa_j \leq d_j$ and $\tau_j > 0$, setting $\tau_j = 0$ stays feasible ($0 \geq \kappa_j - d_j$ because $\kappa_j - d_j \leq 0$) and decreases the objective by $\tau_j > 0$.

- *Synthesis*: in every optimum $\tau_j = \max\{0, \kappa_j - d_j\}$, exactly the tardiness. The lower bound holds in every feasible solution; if τ_j exceeded it, lowering it to the maximum would keep both constraints and decrease the objective by the difference, positive. No additional constraint is needed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 M = sum(t)
2 m = gp.Model("tardiness"); m.Params.OutputFlag = 0
3 s = m.addVars([(j, i) for j in range(n) for i in range(n) if j != i],
4               vtype=GRB.BINARY, name="s")
5 kappa = m.addVars(n, name="kappa"); tau = m.addVars(n, name="tau")
6 m.setObjective(tau.sum(), GRB.MINIMIZE)
7 m.addConstrs((s[j, i] + s[i, j] == 1 for j in range(n)
8               for i in range(j + 1, n)), name="order")
9 m.addConstrs((-M * s[j, i] - kappa[j] + kappa[i] >= t[i] - M
10               for j in range(n) for i in range(n) if j != i),
11               name="precedence")
12 m.addConstrs((-kappa[j] + tau[j] >= -d[j] for j in range(n)),
13               name="tardiness")
14 m.addConstrs((kappa[j] >= t[j] for j in range(n)), name="start")
15 m.optimize()

```

The instance

$n = 3$ jobs, $M = 5 + 4 + 6 = 15$:

	$j = 1$	$j = 2$	$j = 3$
t_j	5	4	6
d_j	3	4	10

Model (7.14) for the instance:

Model 7.7 on the instance

$$\begin{array}{llllllll}
 \min & & & & & & & \tau_1 + \tau_2 + \tau_3 \\
 \text{subject to} & s_{12} & & +s_{21} & & & & = 1 \\
 & & s_{13} & & +s_{31} & & & = 1 \\
 & & & s_{23} & & +s_{32} & & = 1 \\
 & -15s_{12} & & & & -\kappa_1 + \kappa_2 & & \geq -11 \\
 & & -15s_{13} & & & -\kappa_1 & +\kappa_3 & \geq -9 \\
 & & & -15s_{21} & & +\kappa_1 - \kappa_2 & & \geq -10 \\
 & & & & -15s_{23} & & -\kappa_2 + \kappa_3 & \geq -9 \\
 & & & & & -15s_{31} & +\kappa_1 & -\kappa_3 \geq -10 \\
 & & & & & & -15s_{32} & +\kappa_2 - \kappa_3 \geq -11 \\
 & & & & & & & -\kappa_1 & +\tau_1 \geq -3 \\
 & & & & & & & & -\kappa_2 & +\tau_2 \geq -4 \\
 & & & & & & & & & -\kappa_3 & +\tau_3 \geq -10 \\
 & & & & & & & & & \kappa_1 & \geq 5 \\
 & & & & & & & & & \kappa_2 & \geq 4 \\
 & & & & & & & & & \kappa_3 & \geq 6 \\
 & s_{12}, & s_{13}, & s_{21}, & s_{23}, & s_{31}, & s_{32} & & & & \in \{0, 1\} \\
 & & & & & & & \kappa_1, \kappa_2, \kappa_3 & & & \geq 0 \\
 & & & & & & & & \tau_1, \tau_2, \tau_3 & & \geq 0
 \end{array}$$

Constructive heuristic: the primal bound

The simplest: process the jobs in the given order $1 \rightarrow 2 \rightarrow 3$ (FIFO); a better rule orders by increasing due date (EDD, *earliest due date*), which here coincides.

- **Step 1.** Job 1: $\kappa_1 = 5$, $\tau_1 = \max\{0, 5 - 3\} = 2$.
- **Step 2.** Job 2: $\kappa_2 = 5 + 4 = 9$, $\tau_2 = \max\{0, 9 - 4\} = 5$.
- **Step 3.** Job 3: $\kappa_3 = 9 + 6 = 15$, $\tau_3 = \max\{0, 15 - 10\} = 5$.

Solution $\bar{s}_{12} = \bar{s}_{13} = \bar{s}_{23} = 1$ (the others 0), $\bar{\kappa} = (5, 9, 15)$, $\bar{\tau} = (2, 5, 5)$, value 12: $z(MILP) \leq 12$. It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

LP relaxation and dual: the dual bound

Relaxing (7.14f) to $s_{ji} \geq 0$, with duals α_{ji} free (7.14b), $\beta_{ji} \geq 0$ (7.14c), $\gamma_j \geq 0$ (7.14d) and $\delta_j \geq 0$ (7.14e):

Dual of the LP relaxation of model 7.7

$$\max \sum_{j < i} \alpha_{ji} + \sum_{j \neq i} (t_i - M) \beta_{ji} \quad (7.15a)$$

$$- \sum_{j=1}^n d_j \gamma_j + \sum_{j=1}^n t_j \delta_j$$

$$\text{subject to } \alpha_{ji} - M \beta_{ji} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15b)$$

$$\alpha_{ji} - M \beta_{ij} \leq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15c)$$

$$- \sum_{i \neq j} \beta_{ji} + \sum_{i \neq j} \beta_{ij} - \gamma_j + \delta_j \leq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15d)$$

$$\gamma_j \leq 1, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15e)$$

$$\alpha_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j < i, \quad (7.15f)$$

$$\beta_{ji} \geq 0, \quad \forall i, j \in \{1, 2, \dots, n\}, j \neq i, \quad (7.15g)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}, \quad (7.15h)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, n\}. \quad (7.15i)$$

Description. The variable α_{ji} is the price of the ordering choice between jobs j and i , β_{ji} the price of the precedence written with the big-M, γ_j the price of the constraint that defines the tardiness of job j , and δ_j that of its minimum completion time. The objective (7.15a) collects these prices with the right-hand sides of the corresponding constraints. Constraints (7.15b) and (7.15c) are the columns of the s_{ji} and of the s_{ij} : the ordering choice enters the equality with coefficient 1 and the precedence with $-M$, and the balance cannot be positive because those variables do not appear in the objective. Constraints (7.15d) are the columns of the κ_j : the completion time enters the precedences with opposite signs, the tardiness with -1 and the minimum time with $+1$. Constraints (7.15e) are the columns of the τ_j : tardiness costs 1 in the primal objective, so its price cannot exceed 1.

For the instance the objective is $\max \alpha_{12} + \alpha_{13} + \alpha_{23} - 11\beta_{12} - 9\beta_{13} - 10\beta_{21} - 9\beta_{23} - 10\beta_{31} - 11\beta_{32} - 3\gamma_1 - 4\gamma_2 - 10\gamma_3 + 5\delta_1 + 4\delta_2 + 6\delta_3$.

A hand-built dual solution. Every β_{ji} has coefficient $t_i - M < 0$: it pays to keep them at zero, and then (7.15b) and (7.15c) impose $\alpha = 0$. What is left is γ and δ , with $\delta_j \leq \gamma_j \leq 1$ from constraints (7.15d) and (7.15e): every job contributes at most $t_j \delta_j - d_j \gamma_j \leq t_j - d_j$, positive only if the job is late even when processed first. Only job 1 is ($t_1 = 5 > d_1 = 3$): $\bar{\gamma}_1 = \bar{\delta}_1 = 1$, all the rest 0, value $-3 + 5 = 2$. Hence $2 \leq z(MILP) \leq 12$. The reading: job 1 will be at least 2 minutes late wherever it is placed.

What the solver says. $z(LP) = 2$: the hand-built solution is optimal for the dual, and the LP relaxation of this model is *extremely weak*, as for all big-M models — the relaxation sets $s_{ji} = 1/2$ and releases the precedences. The integer optimum is $z(MILP) = 11$, sequence

$2 \rightarrow 1 \rightarrow 3$: $\tilde{\kappa} = (9, 4, 15)$, $\tilde{\tau} = (6, 0, 5)$. The given order cost 12; swapping the first two jobs saves 1.

LB (given order)	UB (hand dual)	$z(LP)$	$z(MILP)$	heuristic gap
12	2	2	11	9.1%

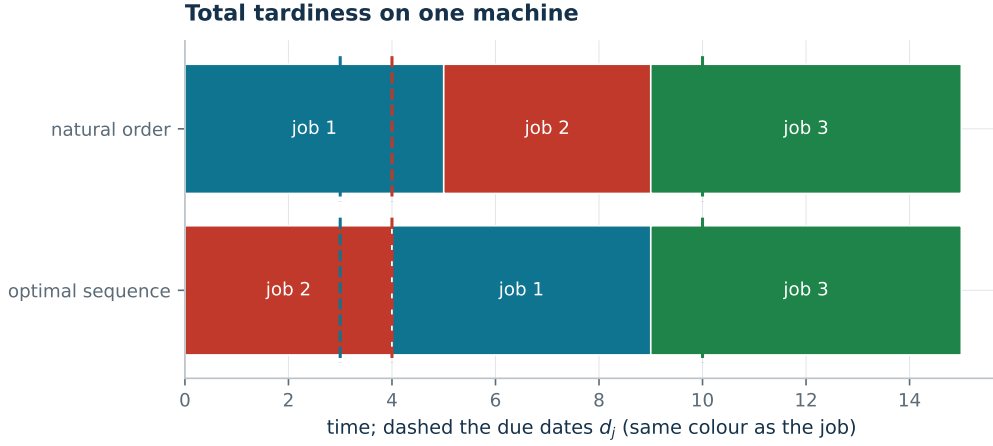


Figure 7.3: The two sequences on the instance: the given order (12) and the optimal sequence $2 \rightarrow 1 \rightarrow 3$ (11); dashed the due dates.

Additional considerations

- $s_{ji} \leq 1$ ($n(n-1)$) are valid and implied by (7.14b).
- *Transitivity*: for three distinct jobs j, i, k , if j precedes i and i precedes k then j precedes k :

$$s_{ji} \text{ AND } s_{ik} \implies s_{jk} \quad \text{and, by De Morgan,} \quad \text{NOT } s_{ji} \text{ OR NOT } s_{ik} \text{ OR } s_{jk},$$

i.e.

$$s_{ji} + s_{ik} - s_{jk} \leq 1$$

($n(n-1)(n-2)/6$ constraints per unordered triple, with its permutations). They are valid inequalities: in every real sequence transitivity holds, hence constraints (7.14c) imply them (a cycle $j \rightarrow i \rightarrow k \rightarrow j$ would give $\kappa_j \geq \kappa_j + t_i + t_k + t_j$, impossible). In the relaxation they cut off the 1/2 solutions: they are the first step to strengthen a big-M model.

- The choice of M : $\sum_{j=1}^n t_j$ is the smallest value that works in general; larger M leave the same integer set and a weaker relaxation (Section “Big-M: conditional constraints and disjunctions”).

Additional modelling questions

Modelling question 7.13 — release dates

Job 2 cannot start before time $\rho_2 = 2$ (the material arrives late); the others are available from the start. Model and find the new optimum.

Modelling question 7.14 — minimising the maximum tardiness

The company wants to minimise the tardiness of the latest job, not the sum. Model and solve; what changes in the link of τ_j ?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	12	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	11	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.1\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 90.9\%.$$

The sandwich on variant 7a

The release dates add a family $\varepsilon_j \geq 0$ to the dual, entering the objective with its right-hand side $\rho_j + t_j$. With $\gamma_j = 1$, the column of κ_j imposes $\delta_j + \varepsilon_j \leq 1$, and it pays to put all the weight on ε_j : the release replaces the processing time. The heuristic is the same EDD rule, which now waits for the release.

	value	what it is
UB	12	heuristic solution
LB	2	dual certificate built by hand
$z(LP)$	4	relaxation without the bounds
$z(LP^+)$	4	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

Location and coverage

Class: BIP / MILP

Script: one per problem, `python/fam08_1_capacitated.py ...`

[Online page](#)

The family

Four problems in which we decide *where* to open a facility — a location, a hub — and how this constrains the variables that depend on that decision: where to ship goods (problem 1), which clients to serve (problem 2), which clients to cover (problem 3), which hub each terminal connects to (problem 4). What changes from problem to problem is the nature of the link between the opening variable and the variables that depend on it, and how the budget on openings enters the model.

The links between variables revisited here:

- **aggregated activation** (problem 1): a single family of constraints acts as both the link and the capacity constraint, $\sum_c y_{lc} \leq u_l x_l$;
- **disaggregated activation** (problem 2): $x_l \geq y_{lc}$ for every location and client, derived from the CNF of a Boolean implication as in problem 7.5, here with a budget k on the number of open locations instead of a time budget;
- **if and only if** (problem 3): one direction imposed by two families of constraints (signal threshold and interference), the other following from the objective;
- **aggregated activation and a maximum variable** (problem 4): the same activation as problem 1 together with the maximum link $z_j \geq c_{ij} x_{ij}$ already seen in problem 7.4.

For each: model, proof of the links, instance, constructive heuristic (a bound from the feasible-solutions side), dual of the LP relaxation with a hand-built dual solution (a bound from the other side), solver solution, additional considerations, and two additional modelling questions with solution. Every number comes from the scripts `fam08_1_capacitated.py ... fam08_4_hub.py`.

Family notation

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations/hubs, indexed by $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients/terminals, indexed by $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	transport/connection cost from location/hub l to client/terminal c
i_l, f_l	$\in \mathbb{Q}_{\geq 0}$	opening/activation cost of location/hub l
u_l	$\in \mathbb{Q}_{>0}$	capacity of location l
d_c	$\in \mathbb{Q}_{>0}$	demand of client c
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of open locations / capacity of each hub
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	signal strength from location l to client c
p_c	$\in \mathbb{Q}_{>0}$	profit if client c is covered
t, b	$\in \mathbb{Q}_{>0}$	signal threshold and interference limit

8.1 Capacitated facility location

Script: `python/fam08_1_capacitated.py`[Notebook in Colab](#)

Problem 8.1 — capacitated facility location

A company must serve $n \in \mathbb{Z}_{\geq 1}$ clients with its product and has identified $m \in \mathbb{Z}_{\geq 1}$ candidate locations where a facility can be installed. For each client $c \in \{1, 2, \dots, n\}$, the value $d_c \in \mathbb{Q}_{>0}$ is the demand, in liters of product, of that client. For each location $l \in \{1, 2, \dots, m\}$ and each client c , the value $t_{lc} \in \mathbb{Q}_{>0}$ is the cost, in euros, of transporting one liter of product from location l to client c . For each location l , the value $u_l \in \mathbb{Q}_{>0}$ is the capacity, in liters, of the facility installed there, and $i_l \in \mathbb{Q}_{>0}$ is the installation cost. The company wants to decide where to install facilities and how to serve the clients, at minimum total cost.

The problem in words. We decide where to install facilities and how much product to transport from each location to each client. *The objective:* minimum total cost (installation plus transport). *The constraints:* a location that is not installed ships nothing, and an installed location does not exceed its capacity; every client's demand must be satisfied exactly. This is the *capacitated facility location* problem.

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
t_{lc}	$\in \mathbb{Q}_{>0}$	transport cost (per liter) from location l to client c
u_l	$\in \mathbb{Q}_{>0}$	capacity (liters) of location l
i_l	$\in \mathbb{Q}_{>0}$	installation cost of location l
d_c	$\in \mathbb{Q}_{>0}$	demand (liters) of client c

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if a facility is installed at location } l, \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following $m n$ non-negative continuous variables:

y_{lc} = liters of product transported from location l to client c

$$\forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a MILP model for the problem is the following:

Model 8.1 — capacitated facility location

$$\min \sum_{l=1}^m i_l x_l + \sum_{l=1}^m \sum_{c=1}^n t_{lc} y_{lc} \quad (8.1a)$$

$$\text{subject to } u_l x_l - \sum_{c=1}^n y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1b)$$

$$\sum_{l=1}^m y_{lc} = d_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.1c)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.1d)$$

$$y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.1e)$$

Description of the objective function and constraints

- the linear objective function (8.1a) minimizes the total cost, the sum of the installation costs of the open locations and the transport costs;
- the linear constraints (8.1b) link the transport variables to the installation variables and impose the capacity restriction: they ensure that if a positive quantity is shipped from a location, that location is installed, and that an uninstalled location ships nothing; moreover, if a location is installed, the total quantity shipped does not exceed its capacity u_l (m linear constraints);
- the linear constraints (8.1c) ensure that every client's demand is satisfied exactly (n linear constraints);
- constraints (8.1d) and (8.1e) define the variables of the model.

The link between the variables. For each location l , if a positive quantity of product ships from location l , then the location must be installed:

$$\sum_{c=1}^n y_{lc} > 0 \implies x_l = 1.$$

The contrapositive gives

$$x_l = 0 \implies \sum_{c=1}^n y_{lc} = 0 :$$

an uninstalled location ships nothing. This implication — together with its contrapositive — is imposed directly by constraints (8.1b): if $\sum_{c=1}^n y_{lc} > 0$, the constraint forces $u_l x_l \geq \sum_{c=1}^n y_{lc} > 0$ and, since $u_l > 0$, $x_l > 0$, i.e., $x_l = 1$ (being binary). Conversely, if $x_l = 0$, the constraint becomes $\sum_{c=1}^n y_{lc} \leq 0$ and, since every $y_{lc} \geq 0$, $y_{lc} = 0$ for every c . (When $x_l = 1$, the constraint reduces to $\sum_{c=1}^n y_{lc} \leq u_l$: the capacity.)

The opposite direction — if nothing ships from a location, it is not installed:

$$x_l = 1 \implies \sum_{c=1}^n y_{lc} > 0$$

— is not imposed explicitly by the constraints, but follows from the objective function in every optimal solution: since $i_l > 0$, if $\sum_{c=1}^n y_{lc} = 0$ and $x_l = 1$, setting $x_l = 0$ remains feasible and decreases the cost by i_l . Therefore, in every optimal solution, no installed location goes unused. A single family of constraints thus acts as both the activation link and the capacity constraint.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("capacitated_location")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, name="y")
4 mod.setObjective(gp.quicksum(i[l] * x[l] for l in range(m))
5                  + gp.quicksum(t[l][c] * y[l, c] for l in range(m) for
6                  c in range(n)), GRB.MINIMIZE)
7 mod.addConstrs((u[l] * x[l] - gp.quicksum(y[l, c] for c in range(n)) >=
8                0
9                for l in range(m)), name="capacity")
10 mod.addConstrs((gp.quicksum(y[l, c] for l in range(m)) == d[c] for c in
11                range(n)), name="demand")

```

The instance

$m = 2$ locations, $n = 3$ clients:

t_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	4	5	6
$l = 2$	6	4	3

	$l = 1$	$l = 2$
u_l	50	50
i_l	60	90

	$c = 1$	$c = 2$	$c = 3$
d_c	8	25	27

Constructive heuristic: the primal bound

Locations are scanned in natural order; for each, clients are scanned and each is shipped the minimum of the location's residual capacity and the client's residual demand, without exceeding either one; a location that shipped something is opened.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

Execution on the instance (script output): location 1 ships 8 to client 1 (demand exhausted, residual capacity 42), then 25 to client 2 (demand exhausted, residual capacity 17), then 17 to client 3 (location 1's capacity exhausted, 10 still owed); location 2 ships the remaining 10 to client 3. The feasible solution found is

$$\bar{x}_1 = \bar{x}_2 = 1, \quad \bar{y}_{11} = 8, \bar{y}_{12} = 25, \bar{y}_{13} = 17, \bar{y}_{23} = 10, \quad \text{all other } \bar{y}_{lc} = 0,$$

of value

$$\underbrace{60 + 90}_{\text{installation}} + \underbrace{4 \cdot 8 + 5 \cdot 25 + 6 \cdot 17 + 3 \cdot 10}_{\text{transport} = 32 + 125 + 102 + 30 = 289} = 150 + 289 = 439.$$

Hence

$$z(MILP) \leq UB = 439.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable μ_l with each constraint (8.1b) ($\geq$ direction) and a free dual variable π_c with each constraint (8.1c) (an equality). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.1

$$\max \sum_{c=1}^n d_c \pi_c \tag{8.2a}$$

$$\text{subject to } u_l \mu_l \leq i_l, \quad \forall l \in \{1, 2, \dots, m\}, \tag{8.2b}$$

$$-\mu_l + \pi_c \leq t_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \tag{8.2c}$$

$$\mu_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \tag{8.2d}$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \tag{8.2e}$$

Description. The variable μ_l is the price of one unit of capacity of site l and π_c the value of one unit of demand of customer c . The objective (8.2a) prices all the demand to be met at those values. Constraints (8.2b) are the columns of the x_l : the capacity u_l that opening site l makes available, priced at μ_l , cannot be worth more than the installation cost i_l . Constraints (8.2c) are the columns of the y_{lc} : carrying one unit from site l to customer c is worth π_c and consumes one unit of capacity at price μ_l ; the balance cannot exceed the transport cost t_{lc} .

A hand-built dual solution. Set $\bar{\mu}_l = i_l/u_l$: constraint (8.2b) holds with equality, spreading the installation cost over the capacity. On the instance,

$$\bar{\mu}_1 = 60/50 = 6/5, \quad \bar{\mu}_2 = 90/50 = 9/5.$$

Constraint (8.2c) becomes $\pi_c \leq t_{lc} + \bar{\mu}_l$ for every location l : the largest feasible value is $\bar{\pi}_c = \min_l(t_{lc} + \bar{\mu}_l)$:

$$\bar{\pi}_1 = \min(4 + 6/5, 6 + 9/5) = 26/5,$$

$$\bar{\pi}_2 = \min(5 + 6/5, 4 + 9/5) = 29/5,$$

$$\bar{\pi}_3 = \min(6 + 6/5, 3 + 9/5) = 24/5.$$

The value of this feasible dual solution is

$$8 \cdot \frac{26}{5} + 25 \cdot \frac{29}{5} + 27 \cdot \frac{24}{5} = \frac{208+725+648}{5} = \frac{1581}{5}.$$

By weak duality, $LB = 1581/5 \leq z(LP) \leq z(MILP) \leq UB = 439$. The recipe has a managerial meaning: spreading each location's fixed cost over its capacity and then serving every client at the minimum marginal cost is a lower bound anyone would write down; the dual formalizes it.

What the solver says. The optimum of the LP relaxation is exactly $z(LP) = 1581/5$: the hand-built dual solution above is already optimal (strong duality, no room for improvement). Strengthening the relaxation with $x_l \leq 1$ gives $z(LP^+) = 317$: a sizeable jump, a sign that the constraint $x_l \leq 1$ — implicit in the MILP but not in the relaxation without the bounds — matters here. The MILP has optimum $z(MILP) = 365$, with $\tilde{x}_1 = \tilde{x}_2 = 1$, $\tilde{y}_{11} = 8$, $\tilde{y}_{12} = 2$, $\tilde{y}_{22} = 23$, $\tilde{y}_{23} = 27$: both locations open, but location 1 serves only part of client 2's demand (the rest comes from location 2, which is cheaper for that client). The heuristic gap relative to the optimum is 20.3%.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
439	1581/5	1581/5	317	365	20.3%

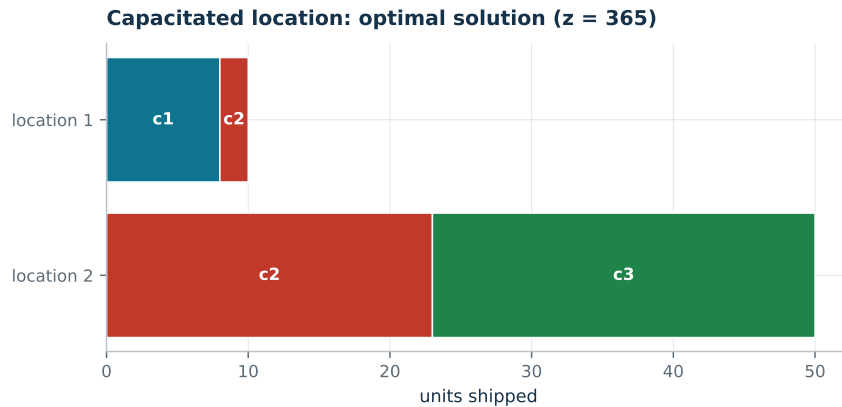


Figure 8.1: Optimal solution: for each open location, the liters shipped to each client.

Additional considerations

- For every location l and client c , if $u_l < d_c$ no single location can satisfy client c 's entire demand alone: not the case in this instance, but worth checking before assuming one location suffices.
- The constraints $y_{lc} \leq d_c x_l$ (for every l, c) are valid inequalities implied jointly by constraints (8.1b) and (8.1c): if $x_l = 0$ they force $y_{lc} = 0$, and if $x_l = 1$ they are never tighter than $y_{lc} \leq d_c$ (which follows anyway from demand satisfaction).

Additional modelling questions

Modelling question 8.1 — minimum lot for every open location

Every open location must ship at least 5 liters overall (it makes no sense to bear the installation cost for a negligible quantity). How does the model change? What is the new optimum for the instance?

Modelling question 8.2 — conditional opening

Location 2 can only be installed if location 1 is also installed (say, a supervision requirement). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	439	heuristic solution (UB)
LB	$\frac{1581}{5}$	certified bound (LB)
$z(MILP)$	365	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.3\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.6\%.$$

The sandwich on variant 1b

"Site 2 opens only if site 1 opens" adds $\rho \leq 0$ to the dual, which moves fixed cost between the two sites: $\mu_1 = (i_1 + \rho)/u_1$ and $\mu_2 = (i_2 - \rho)/u_2$. One tries ρ among the values that change the minimum defining π_c ; on this instance the best is $\rho = 0$, because site 1 already has the lowest cost per litre.

	value	what it is
UB	439	heuristic solution
LB	$\frac{1581}{5}$	dual certificate built by hand
$z(LP)$	325	relaxation without the bounds
$z(LP^+)$	325	relaxation with the bounds
$z(MILP)$	365	optimum of the MILP

8.2 p-median: at most k locations

Script: `python/fam08_2_pmedian.py`

[Notebook in Colab](#)

Problem 8.2 — p-median

A company must choose at most $k \in \mathbb{Z}_{\geq 1}$ locations, among $m \in \mathbb{Z}_{\geq 1}$ candidates, where to open a service point, and assign each of the $n \in \mathbb{Z}_{\geq 1}$ clients to the most convenient open location. For each location $l \in \{1, 2, \dots, m\}$ and each client $c \in \{1, 2, \dots, n\}$, the value $d_{lc} \in \mathbb{Q}_{>0}$ is the distance (or service cost) between location l and client c . Every client must be assigned to exactly one open location. The company wants to decide which locations to open and how to assign clients, minimizing the sum of the distances between each client and the location it is assigned to.

The problem in words. We decide which locations to open (at most k) and which open location to assign each client to. *The objective:* minimum sum of client-location distances. *The constraints:* every client assigned to exactly one location; a client can only be assigned to an open location; at most k open locations. This is the classic *p-median* problem.

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
d_{lc}	$\in \mathbb{Q}_{>0}$	distance between location l and client c
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of open locations

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if location } l \text{ is open,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following $m n$ binary variables:

$$y_{lc} = \begin{cases} 1 & \text{if client } c \text{ is served by location } l, \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a BIP model for the problem is the following:

Model 8.2 — p-median

$$\min \sum_{l=1}^m \sum_{c=1}^n d_{lc} y_{lc} \quad (8.3a)$$

$$\text{subject to } \sum_{l=1}^m y_{lc} = 1, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.3b)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.3c)$$

$$x_l - y_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.3d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.3e)$$

$$y_{lc} \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.3f)$$

Description of the objective function and constraints

- the linear objective function (8.3a) minimizes the sum of the distances between every client and the location it is assigned to;
- the linear constraints (8.3b) ensure that every client is assigned to exactly one location (n linear constraints);
- the linear constraint (8.3c) limits to k the number of open locations (one linear constraint);
- the linear constraints (8.3d) link the assignment variables to the opening variables, in disaggregated form ($m n$ linear constraints);
- constraints (8.3e) and (8.3f) define the variables of the model.

The link between the variables. For every location l and client c , if client c is served by location l , then location l must be open:

$$y_{lc} = 1 \implies x_l = 1.$$

In Boolean form the implication is equivalent, by its CNF, to

$$y_{lc} \implies x_l \quad \equiv \quad \text{NOT } y_{lc} \text{ OR } x_l,$$

i.e. $x_l \geq y_{lc}$: exactly constraint (8.3d), imposed directly (the objective is not needed). From the contrapositive,

$$x_l = 0 \implies y_{lc} = 0 :$$

no assignment can leave a closed location — again imposed directly by the same constraint. Unlike problem 8.1, here there is no opening cost that would discourage open-but-unused locations: constraint (8.3c) only caps the number of open locations, so the direction “ $x_l = 1 \Rightarrow$ at least one client served by l ” is neither imposed nor guaranteed by optimality. The link is disaggregated — one row per pair (l, c) , as in problem 7.5 — because it is binary against binary for each pair, not a total against a single activation.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("p_median")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(dist[l][c] * y[l, c] for l in range(m) for
   c in range(n)), GRB.MINIMIZE)
5 mod.addConstrs((y.sum("*", c) == 1 for c in range(n)), name="assign")
6 mod.addConstr(x.sum() <= k, name="number_of_locations")
7 mod.addConstrs((x[l] - y[l, c] >= 0 for l in range(m) for c in range(n)
   ), name="link")

```

The instance

$m = 3$ locations, $n = 3$ clients, $k = 2$:

d_{lc}	$c = 1$	$c = 2$	$c = 3$
$l = 1$	5	6	10
$l = 2$	3	12	9
$l = 3$	10	9	4

Constructive heuristic: the primal bound

The first k locations are opened in natural order; then every client is served by the nearest open location.

Execution on the instance (script output): locations 1 and 2 are opened ($x_1 = x_2 = 1$). Client 1: the nearest open location is 2 (distance 3). Client 2: the nearest open location is 1 (distance 6). Client 3: the nearest open location is 2 (distance 9). The feasible solution found is

$$\bar{y}_{21} = 1, \quad \bar{y}_{12} = 1, \quad \bar{y}_{23} = 1, \quad \text{all other } \bar{y}_{lc} = 0,$$

of value $3 + 6 + 9 = 18$. Hence

$$z(MILP) \leq UB = 18.$$

LP relaxation and dual: the dual bound

We associate a free dual variable μ_c with each constraint (8.3b) (an equality), a non-positive dual variable ϱ with constraint (8.3c) (a $\leq$ direction in a minimization problem) and a non-negative dual variable π_{lc} with each constraint (8.3d) (a $\geq$ direction). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.2

$$\max \sum_{c=1}^n \mu_c + k \varrho \quad (8.4a)$$

$$\text{subject to } \varrho + \sum_{c=1}^n \pi_{lc} \leq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.4b)$$

$$\mu_c - \pi_{lc} \leq d_{lc}, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}, \quad (8.4c)$$

$$\mu_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.4d)$$

$$\varrho \leq 0, \quad (8.4e)$$

$$\pi_{lc} \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \forall c \in \{1, 2, \dots, n\}. \quad (8.4f)$$

Description. The variable μ_c is the value of the assignment of customer c , ϱ the (non-positive) price of the budget of k sites, and π_{lc} the price of the link "to serve c from l one must open l ". The objective (8.4a) prices the assignments and the budget. Constraints (8.4b) are the columns of the x_l : opening site l consumes one unit of budget and releases the links of all customers; the balance cannot be positive, because opening a site costs nothing in the objective of this model. Constraints (8.4c) are the columns of the y_{lc} : serving customer c from site l is worth μ_c and consumes the link π_{lc} ; the balance cannot exceed the distance d_{lc} .

A hand-built dual solution. With $\bar{\varrho} = 0$ and $\bar{\pi}_{lc} = 0$ for every pair, constraint (8.4b) holds ($0 \leq 0$) and constraint (8.4c) becomes $\mu_c \leq d_{lc}$ for every location l : the largest feasible value is $\bar{\mu}_c = \min_l d_{lc}$, the distance to the nearest location in absolute terms (open or not):

$$\bar{\mu}_1 = \min(5, 3, 10) = 3, \quad \bar{\mu}_2 = \min(6, 12, 9) = 6, \quad \bar{\mu}_3 = \min(10, 9, 4) = 4,$$

of value $3 + 6 + 4 + 2 \cdot 0 = 13$. By weak duality, $LB = 13 \leq z(LP) \leq z(MILP) \leq UB = 18$. The recipe has a meaning: "no client can cost less than its nearest location overall" is a trivial lower bound that ignores the budget constraint k ; the dual represents it exactly with $\varrho = 0$.

What the solver says. The optimum of the LP relaxation is $z(LP) = 15$, equal to the relaxation with the bounds $z(LP^+) = 15$: the constraint $y_{lc} \leq 1$ is never relevant here. The MILP has optimum $z(MILP) = 15$, with $\tilde{x}_1 = \tilde{x}_3 = 1$ (locations 1 and 3 open, not 1 and 2 as in the heuristic) and $\tilde{y}_{11} = \tilde{y}_{12} = \tilde{y}_{33} = 1$: the LP relaxation is already integral on this instance, and the heuristic — which always opens the first k locations regardless of cost — loses $18/15 - 1 = 20.0\%$.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
18	13	15	15	15	20.0%

Additional considerations

- Constraint (8.3c) is "at most k ", not "exactly k ": opening fewer than k locations is never advantageous when distances are positive (opening one more location can only bring some client closer), but the model does not impose this explicitly. Question 2a below checks that, on this instance, the optimum does not change when equality is imposed.

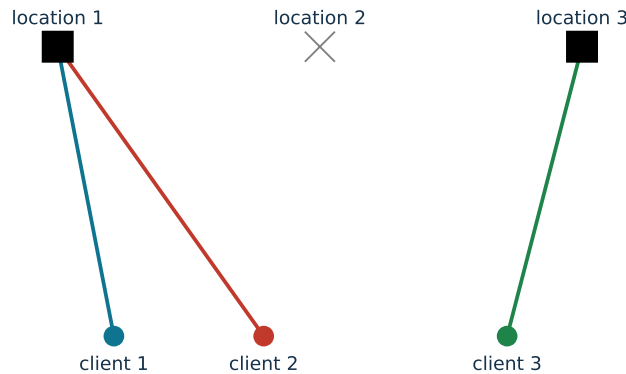
p-median: optimal solution ($z = 15$); square = open location

Figure 8.2: Optimal solution: square = open location, cross = closed location, lines = client–location assignment.

- Constraints (8.3d), summed over c for a fixed location l , give $n x_l \geq \sum_{c=1}^n y_{lc}$: an aggregated valid inequality, weaker than the disaggregated one used in the model.

Additional modelling questions

Modelling question 8.3 — exactly k open locations

For organisational reasons, exactly k locations must be open (not at most). How does the model change? What is the new optimum for the instance?

Modelling question 8.4 — proximity coverage for one client

Client 1 must be served within distance 4 (a service-level requirement). How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	18	heuristic solution (UB)
LB	13	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 20.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 33.3\%.$$

The sandwich on variant 2a

Imposing *exactly* k sites instead of *at most* k adds $\sigma \geq 0$ with right-hand side k to the dual. The algebra settles it in one line: the column of the x_l imposes $\varrho + \sigma \leq 0$, and the objective contains $k(\varrho + \sigma)$, which is therefore never positive. The maximum is at $\varrho + \sigma = 0$ and the value is $\sum_c \mu_c$ again: the relaxation does not move — and here the integer optimum does not move either: it stays 15. That is not an accident of the instance: with no opening cost one more site cannot worsen the assignment, so from a solution with fewer than k sites one always gets an equally good one with exactly k . The equality constraint is redundant; it bites only when opening costs something.

	value	what it is
UB	18	heuristic solution
LB	13	dual certificate built by hand
$z(LP)$	15	relaxation without the bounds
$z(LP^+)$	15	relaxation with the bounds
$z(MILP)$	15	optimum of the MILP

8.3 Signal coverage with interference

Script: `python/fam08_3_coverage.py`

[Nobook in Colab](#)

Problem 8.3 — coverage with interference

An operator must choose at most $k \in \mathbb{Z}_{\geq 1}$ locations, among $m \in \mathbb{Z}_{\geq 1}$ candidates, where to install an antenna, to serve $n \in \mathbb{Z}_{\geq 1}$ clients. For each location $l \in \{1, 2, \dots, m\}$ and each client $c \in \{1, 2, \dots, n\}$, the value $s_{lc} \in \mathbb{Q}_{\geq 0}$ is the signal strength received by client c if location l is installed. A client c is *covered* if and only if the total signal strength received from all installed locations is at least the threshold $t \in \mathbb{Q}_{>0}$ and at most one installed location generates for that client a signal greater than or equal to the interference limit $b \in \mathbb{Q}_{>0}$. For each client c , the value $p_c \in \mathbb{Q}_{>0}$ is the profit if the client is covered. The operator wants to decide which locations to install to maximize the total profit.

The problem in words. We decide which locations to install (at most k). The objective: maximum total profit, the sum of the profits of the covered clients. The constraints: a client is covered if and only if it receives enough signal and not too much interference; the number of installed locations does not exceed k .

Data (model input)

Symbol	Type	Meaning
m	$\in \mathbb{Z}_{\geq 1}$	number of candidate locations, $l \in \{1, 2, \dots, m\}$
n	$\in \mathbb{Z}_{\geq 1}$	number of clients, $c \in \{1, 2, \dots, n\}$
s_{lc}	$\in \mathbb{Q}_{\geq 0}$	signal received by client c if location l is installed
p_c	$\in \mathbb{Q}_{>0}$	profit if client c is covered
t	$\in \mathbb{Q}_{>0}$	total signal threshold
b	$\in \mathbb{Q}_{>0}$	interference limit
k	$\in \mathbb{Z}_{\geq 1}$	maximum number of installed locations

For every client c define $\mathcal{L}_c = \{l : s_{lc} \geq b\}$, the “strong” locations for that client.

Decision variables

We introduce the following m binary variables:

$$x_l = \begin{cases} 1 & \text{if location } l \text{ is installed,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall l \in \{1, 2, \dots, m\},$$

and the following n binary variables:

$$y_c = \begin{cases} 1 & \text{if client } c \text{ is covered,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall c \in \{1, 2, \dots, n\}.$$

Using these variables, a BIP model for the problem is the following:

Model 8.3 — coverage with interference

$$\max \sum_{c=1}^n p_c y_c \quad (8.5a)$$

$$\text{subject to} \quad - \sum_{l=1}^m s_{lc} x_l + t y_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5b)$$

$$\sum_{l \in \mathcal{L}_c} x_l + (m-1) y_c \leq m, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.5c)$$

$$\sum_{l=1}^m x_l \leq k, \quad (8.5d)$$

$$x_l \in \{0, 1\}, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.5e)$$

$$y_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (8.5f)$$

Description of the objective function and constraints

- the linear objective function (8.5a) maximizes the total profit, the sum of the profits of the covered clients;
- the linear constraints (8.5b) link the coverage variables to the received signal: they ensure that if a client is covered, the total signal received is at least t (n linear constraints);
- the linear constraints (8.5c) link the coverage variables to interference: they ensure that if a client is covered, at most one location in $\mathcal{L}_c$ is installed (n linear constraints);
- the linear constraint (8.5d) limits to k the number of installed locations (one linear constraint);
- constraints (8.5e) and (8.5f) define the variables of the model.

The link between the variables: an “if and only if”. For every client c , the client is covered if and only if the installed locations jointly satisfy the threshold condition and the

interference condition:

$$y_c = 1 \iff \left(\sum_{l=1}^m s_{lc} x_l \geq t \right) \text{ AND } \left(\sum_{l \in \mathcal{L}_c} x_l \leq 1 \right).$$

One direction, imposed directly by the constraints. If $y_c = 1$, constraint (8.5b) forces $\sum_{l=1}^m s_{lc} x_l \geq t y_c = t$: the threshold is met. Constraint (8.5c) forces $\sum_{l \in \mathcal{L}_c} x_l \leq m - (m-1) y_c = 1$: at most one strong location is installed. So

$$y_c = 1 \implies \text{both conditions.}$$

The other direction, which follows from optimality. If both conditions are satisfied by a given set of installed locations, constraints (8.5b) and (8.5c) still allow both $y_c = 0$ and $y_c = 1$: nothing in the constraints forces $y_c = 1$. But since $p_c > 0$ and y_c appears only in these two constraints (with a positive coefficient on the side that bounds it from above), raising y_c from 0 to 1 remains feasible — by assumption both conditions hold — and raises the objective by p_c . Hence, in every optimal solution, $y_c = 1$ whenever both conditions are satisfied. The same pattern — one direction from the constraints, the other from optimality in a maximisation problem — as in problem 7.6.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("coverage_interference")
2 x = mod.addVars(m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(n, vtype=GRB.BINARY, name="y")
4 mod.setObjective(gp.quicksum(p[c] * y[c] for c in range(n)), GRB.
    MAXIMIZE)
5 mod.addConstrs((-gp.quicksum(s[l][c] * x[l] for l in range(m)) + t * y[
    c] <= 0
6                 for c in range(n)), name="threshold")
7 mod.addConstrs((gp.quicksum(x[l] for l in L[c]) + (m - 1) * y[c] <= m
8                 for c in range(n)), name="interference")
9 mod.addConstr(x.sum() <= k, name="budget")

```

The instance

$m = 3$ locations, $n = 5$ clients, $t = 5$, $b = 4$, $k = 2$:

s_{lc}	$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$
$l = 1$	6	0	5	3	1
$l = 2$	4	5	2	0	0
$l = 3$	0	7	5	4	2

	$c = 1$	$c = 2$	$c = 3$	$c = 4$	$c = 5$
p_c	10	20	5	15	25

With $b = 4$: $\mathcal{L}_1 = \{1, 2\}$, $\mathcal{L}_2 = \{2, 3\}$, $\mathcal{L}_3 = \{1, 3\}$, $\mathcal{L}_4 = \{3\}$, $\mathcal{L}_5 = \emptyset$.

Constructive heuristic: the primal bound

The first k locations are opened in natural order; a client is covered if the total signal received from the open locations reaches the threshold t and at most one open location in $\mathcal{L}_c$ is open.

Execution on the instance (script output): locations 1 and 2 are opened. Client 1: total signal $6 + 4 = 10 \geq 5$, but both strong locations in $\mathcal{L}_1 = \{1, 2\}$ are open ($2 > 1$): **not covered**. Client 2: total signal $0 + 5 = 5 \geq 5$, strong locations in $\mathcal{L}_2 = \{2, 3\}$ open: location 2 only ($1 \leq 1$): **covered**. Client 3: total signal $5 + 2 = 7 \geq 5$, strong locations in $\mathcal{L}_3 = \{1, 3\}$ open: only location 1 ($1 \leq 1$): **covered**. Client 4: total signal $3 + 0 = 3 < 5$: **not covered**. Client 5: total signal $1 + 0 = 1 < 5$: **not covered**. The feasible solution found covers clients 2 and 3, of value $20 + 5 = 25$. Hence

$$z(MILP) \geq LB = 25.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable π_c with each constraint (8.5b) (a $\leq$ direction in a maximisation problem), a non-negative dual variable λ_c with each constraint (8.5c), and a non-negative dual variable μ with constraint (8.5d). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.3

$$\min \quad m \sum_{c=1}^n \lambda_c + k \mu \quad (8.6a)$$

$$\text{subject to} \quad - \sum_{c=1}^n s_{lc} \pi_c + \sum_{c: l \in \mathcal{L}_c} \lambda_c + \mu \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (8.6b)$$

$$t \pi_c + (m - 1) \lambda_c \geq p_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6c)$$

$$\pi_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6d)$$

$$\lambda_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (8.6e)$$

$$\mu \geq 0. \quad (8.6f)$$

Description. The variable π_c is the price of the threshold constraint of customer c , λ_c that of its interference constraint, and μ the price of one unit of budget. The objective (8.6a) adds up the right-hand sides of those constraints priced at those values. Constraints (8.6b) are the columns of the x_l : installing antenna l brings signal to the customers it covers, enters the interference constraints of the customers whose list contains it, and consumes one unit of budget; the balance must stay non-negative, because the x_l do not appear in the objective. Constraints (8.6c) are the columns of the y_c : declaring customer c served requires t units of signal at price π_c and $m - 1$ units of interference at price λ_c , and together they must cover the reward p_c .

A hand-built dual solution. With $\bar{\pi}_c = 0$ and $\bar{\mu} = 0$, constraint (8.6b) holds ($\sum \lambda_c \geq 0$), and constraint (8.6c) becomes $(m - 1)\bar{\lambda}_c \geq p_c$, i.e. $\bar{\lambda}_c = p_c / (m - 1) = p_c / 2$:

$$\bar{\lambda}_1 = 5, \quad \bar{\lambda}_2 = 10, \quad \bar{\lambda}_3 = 5/2, \quad \bar{\lambda}_4 = 15/2, \quad \bar{\lambda}_5 = 25/2.$$

The value of this feasible dual solution is $m \sum_{c=1}^n \bar{\lambda}_c + k \cdot 0 = 3 \cdot (5 + 10 + 5/2 + 15/2 + 25/2) =$

$3 \cdot 75/2 = 225/2$. By weak duality, $LB = 25 \leq z(MILP) \leq z(LP) \leq UB = 225/2$ (a maximisation problem: the heuristic gives the lower bound, the dual the upper bound).

What the solver says. The optimum of the LP relaxation is $z(LP) = 41925/646 \approx 64.9$, while strengthening with $x_l \leq 1$ gives $z(LP^+) = 125/2 = 62.5$: a clear improvement. The MILP has optimum $z(MILP) = 45$, with $\tilde{x}_1 = \tilde{x}_3 = 1$ and $\tilde{y}_1 = \tilde{y}_2 = \tilde{y}_4 = 1$ (clients 1, 2 and 4 covered, not 3 or 5): a different configuration from the one found by the heuristic, which covered clients 2 and 3. The heuristic gap is 44.4%: on this maximisation problem, simply opening the first k locations is a much less effective choice than on the family's minimum-cost problems.

LB (heuristic)	UB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
25	225/2	41925/646	125/2	45	44.4%

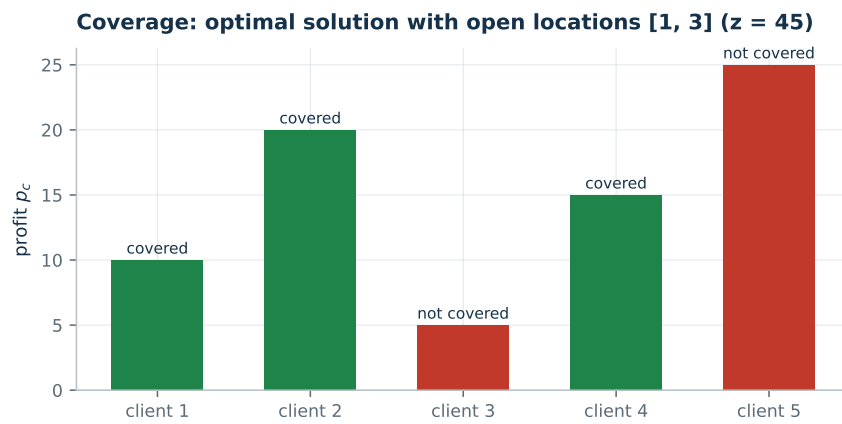


Figure 8.3: Optimal solution: profit of each client, green if covered, red if not covered.

Additional considerations

- Client 5 can never be covered in this instance: $\mathcal{L}_5 = \emptyset$, but the maximum receivable signal is $s_{15} + s_{25} + s_{35} = 1 + 0 + 2 = 3 < t = 5$ even opening all locations. Variable y_5 could be fixed to 0 upfront.
- Constraints (8.5c), for a client c with $|\mathcal{L}_c| \leq 1$, are redundant (never binding): the case for clients 2, 4 and 5 in this instance.

Additional modelling questions

Modelling question 8.5 — guaranteed minimum coverage

By contract, at least 3 clients must be covered. How does the model change? What is the new optimum?

Modelling question 8.6 — conditional installation

For maintenance reasons, location 1 can only be installed if location 3 is also installed. How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	25	heuristic solution (LB)
UB	$\frac{225}{2}$	certified bound (UB)
$z(MILP)$	45	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 44.4\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 194.4\%.$$

The sandwich on variant 3a

"At least three clients covered" adds $\omega \leq 0$ to the dual. It is best left at zero, and the arithmetic says why: lowering it forces every λ_c up by $-\omega/(m-1)$, and in the objective those λ weigh more than the right-hand side saves. The heuristic, with few sites, tries every choice of k and keeps the best feasible one.

	value	what it is
UB	$\frac{225}{2}$	dual certificate built by hand
LB	45	heuristic solution
$z(LP)$	$\frac{41925}{646}$	relaxation without the bounds
$z(LP^+)$	$\frac{125}{2}$	relaxation with the bounds
$z(MILP)$	45	optimum of the MILP

8.4 Hub location with maximum cost

Script: `python/fam08_4_hub.py`

[Notebook in Colab](#)

Problem 8.4 — hub with maximum connection cost

In a computer network there are $n \in \mathbb{Z}_{\geq 1}$ terminals, each of which must be connected to exactly one hub. There are $m \in \mathbb{Z}_{\geq 1}$ hubs available, and each hub can serve at most $k \in \mathbb{Z}_{\geq 1}$ terminals. A hub must be activated before it can serve any terminals: for each hub $j \in \{1, 2, \dots, m\}$, the value $f_j \in \mathbb{Q}_{\geq 0}$ is the activation cost. For each terminal $i \in \{1, 2, \dots, n\}$ and each hub j , the value $c_{ij} \in \mathbb{Q}_{\geq 0}$ is the cost of connecting terminal i to hub j . The objective is to decide how to assign terminals to hubs to minimize the sum of the activation costs of the hubs used and the maximum connection cost of each hub, subject to the constraints described.

The problem in words. We decide which hubs to activate and which hub to connect each terminal to. *The objective:* activation costs plus, for each hub, the highest connection cost among the terminals assigned to it (not the sum: the bottleneck). *The constraints:* every terminal connected to exactly one hub; an inactive hub can serve no one, and an active one serves at most k terminals.

Data (model input)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of terminals, $i \in \{1, 2, \dots, n\}$
m	$\in \mathbb{Z}_{\geq 1}$	number of hubs, $j \in \{1, 2, \dots, m\}$
k	$\in \mathbb{Z}_{\geq 1}$	capacity (terminals) of each hub
c_{ij}	$\in \mathbb{Q}_{\geq 0}$	cost of connecting terminal i to hub j
f_j	$\in \mathbb{Q}_{\geq 0}$	activation cost of hub j

Decision variables

We introduce the following $n m$ binary variables:

$$x_{ij} = \begin{cases} 1 & \text{if terminal } i \text{ is connected to hub } j, \\ 0 & \text{otherwise,} \end{cases} \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\},$$

the following m binary variables:

$$y_j = \begin{cases} 1 & \text{if hub } j \text{ is activated,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall j \in \{1, 2, \dots, m\},$$

and the following m non-negative continuous variables:

$$z_j = \text{maximum connection cost among the terminals assigned to hub } j$$

$$\forall j \in \{1, 2, \dots, m\}.$$

Using these variables, a MILP model for the problem is the following:

Model 8.4 — hub with maximum cost

$$\min \sum_{j=1}^m f_j y_j + \sum_{j=1}^m z_j \quad (8.7a)$$

$$\text{subject to } \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.7b)$$

$$-\sum_{i=1}^n x_{ij} + k y_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7c)$$

$$-c_{ij} x_{ij} + z_j \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7d)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.7e)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.7f)$$

$$z_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (8.7g)$$

Description of the objective function and constraints

- the linear objective function (8.7a) minimizes the total cost, the sum of the activation costs of the hubs used and the maximum connection cost of each hub;
- the linear constraints (8.7b) ensure that every terminal is connected to exactly one hub (n linear constraints);
- the linear constraints (8.7c) link the assignment variables to the activation variables and impose the capacity: if a hub is not activated it can serve no one, and if it is activated it serves at most k terminals (m linear constraints);
- the linear constraints (8.7d) link the assignment variables to the maximum-cost variable: if terminal i is connected to hub j , then z_j cannot be smaller than c_{ij} (nm linear constraints);
- constraints (8.7e), (8.7f) and (8.7g) define the variables of the model.

First link: activation (aggregated). For every hub j , if at least one terminal is connected to j , then j must be activated: $\sum_{i=1}^n x_{ij} \geq 1 \Rightarrow y_j = 1$; from the contrapositive, $y_j = 0 \Rightarrow \sum_{i=1}^n x_{ij} = 0$. This implication — together with its contrapositive — is imposed directly, in aggregated form, by constraint (8.7c): if $y_j = 0$ the constraint becomes $\sum_{i=1}^n x_{ij} \leq 0$, i.e. $x_{ij} = 0$ for every i ; if $y_j = 1$ it becomes $\sum_{i=1}^n x_{ij} \leq k$, the capacity. The opposite direction — an activated hub serves at least one terminal, $y_j = 1 \Rightarrow \sum_{i=1}^n x_{ij} \geq 1$ — is not imposed by the constraints: $y_j = 1$ with all $x_{ij} = 0$ satisfies (8.7c). It follows from optimality, with a strength that depends on the sign of f_j . If $f_j > 0$: given an optimal solution with $\sum_{i=1}^n x_{ij} = 0$ and $y_j = 1$, setting $y_j = 0$ stays feasible and reduces the cost by $f_j > 0$, contradicting optimality; hence **in every optimum** no activated hub is left empty. If instead $f_j = 0$ — which the statement allows, having declared $f_j \in \mathbb{Q}_{\geq 0}$ — the exchange does not worsen but does not improve either, and the correct conclusion is the weaker one: **there exists an optimum** in which the empty hubs are switched off. On the instance $f = (5, 6, 7)$ is strictly positive and the strong version holds; but a proof must be written with the assumption it uses, not with the one that is convenient. Exactly the activation pattern already seen in problem 7.2, here in aggregated form.

Second link: maximum variable. For every terminal i and hub j , if terminal i is connected to hub j , then $z_j \geq c_{ij}$: $x_{ij} = 1 \Rightarrow z_j \geq c_{ij}$, imposed directly by constraint (8.7d) (equivalent to $z_j \geq c_{ij}x_{ij}$). Combining over all terminals connected to j , $z_j \geq \max_{i: x_{ij}=1} c_{ij}$ (with the convention that the maximum over the empty set is 0, guaranteed by $z_j \geq 0$). Since the objective minimizes $\sum_{j=1}^m z_j$ and z_j appears in no other constraint, in every optimal solution z_j takes the smallest feasible value: if $z_j > \max_{i: x_{ij}=1} c_{ij}$, decreasing it to that maximum remains feasible and decreases the objective. Hence, in every optimum, $z_j = \max_{i: x_{ij}=1} c_{ij}$ exactly: the same pattern as the maximum variable of problem 7.4.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mod = gp.Model("hub_max")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, vtype=GRB.BINARY, name="y")
4 z = mod.addVars(m, name="z")
5 mod.setObjective(gp.quicksum(f[j] * y[j] for j in range(m)) + z.sum(),
6                     GRB.MINIMIZE)
7 mod.addConstrs((gp.quicksum(x[i, j] for j in range(m)) == 1 for i in
8                     range(n)), name="assignment")
9 mod.addConstrs((-gp.quicksum(x[i, j] for i in range(n)) + k * y[j] >= 0
10                    for j in range(m)), name="activation")
11 mod.addConstrs((-c[i][j] * x[i, j] + z[j] >= 0 for i in range(n) for j
12                    in range(m)), name="maximum")

```

The instance

$n = 3$ terminals, $m = 3$ hubs, $k = 2$:

c_{ij}	$j = 1$	$j = 2$	$j = 3$
$i = 1$	5	10	2
$i = 2$	5	4	6
$i = 3$	5	4	6

	$j = 1$	$j = 2$	$j = 3$
f_j	5	6	7

Constructive heuristic: the primal bound

A *next-fit* heuristic, inspired by bin packing: one hub at a time is filled up to k terminals, then the algorithm moves to the next one; if all hubs have been used before every terminal is assigned, the algorithm fails. This is the same generic heuristic used for the scheduling problems of the previous family (hubs are the “machines” with capacity k , terminals the “jobs” with unit time independent of the machine): the script calls it from `euristiche.py` instead of rewriting it.

Execution on the instance (script output): terminal 1 goes to hub 1 (residual capacity $2 \rightarrow 1$); terminal 2 goes to hub 1 (residual capacity $1 \rightarrow 0$); terminal 3 no longer fits on hub 1 (full), it moves to hub 2. Hubs 1 and 2 are activated: the maximum connection cost of hub 1 is $\max(c_{11}, c_{21}) = \max(5, 5) = 5$, that of hub 2 is $c_{32} = 4$. The feasible solution found is

$$\bar{x}_{11} = \bar{x}_{21} = \bar{x}_{32} = 1, \quad \bar{y}_1 = \bar{y}_2 = 1, \quad \bar{z}_1 = 5, \bar{z}_2 = 4,$$

of value $5 + 6 + 5 + 4 = 20$. Hence

$$z(MILP) \leq UB = 20.$$

LP relaxation and dual: the dual bound

We associate a free dual variable α_i with each constraint (8.7b) (an equality), a non-negative dual variable β_j with each constraint (8.7c) (a $\geq$ direction), and a non-negative dual variable γ_{ij} with each constraint (8.7d) (a $\geq$ direction). The dual of the LP relaxation is:

Dual of the LP relaxation of model 8.4

$$\max \sum_{i=1}^n \alpha_i \quad (8.8a)$$

$$\text{subject to } \alpha_i - \beta_j - c_{ij} \gamma_{ij} \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (8.8b)$$

$$k \beta_j \leq f_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8c)$$

$$\sum_{i=1}^n \gamma_{ij} \leq 1, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (8.8e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (8.8f)$$

$$\gamma_{ij} \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}. \quad (8.8g)$$

Description. The variable α_i is the value of terminal i , β_j the price of one of the k slots of hub j , and γ_{ij} the price of the link that pushes the maximum cost of hub j above c_{ij} . The objective (8.8a) prices the terminals, all of which must be assigned. Constraints (8.8b) are the columns of the x_{ij} : assigning terminal i to hub j satisfies its assignment constraint, takes one slot and forces the maximum cost of the hub up to c_{ij} ; the balance cannot be positive. Constraints (8.8c) are the columns of the y_j : the k slots that opening hub j makes available cannot be worth more than the fixed cost f_j . Constraints (8.8d) are the columns of the z_j : the maximum cost of hub j weighs 1 in the primal objective, so the prices of the links that push it up sum to at most one.

A hand-built dual solution. Set $\bar{\gamma}_{ij} = 0$ for every pair (constraint (8.8d) is automatically satisfied) and $\bar{\beta}_j = f_j/k$, the largest value allowed by constraint (8.8c): on the instance $\bar{\beta} = (5/2, 3, 7/2)$. Constraint (8.8b) then becomes $\alpha_i \leq \bar{\beta}_j$ for *every* hub j (not only the most convenient one): the largest feasible value is $\bar{\alpha}_i = \min_j \bar{\beta}_j = 5/2$ for every terminal i . The value of this feasible dual solution is $3 \cdot 5/2 = 15/2$. By weak duality, $LB = 15/2 \leq z(LP) \leq z(MILP) \leq UB = 20$.

Setting $\bar{\gamma}_{ij} = 0$ only for the “inconvenient” hubs and hoping that constraint (8.8b) bounds α_i only against the cheapest one is a mistake: the constraint holds for *every* j , so α_i is bounded by the *minimum* over all hubs, not by a single one. A dual solution that ignores this — say, setting $\bar{\gamma}_{i1} = 1/n$ only for the cheapest hub and leaving $\bar{\beta}_j = 0$ elsewhere — is not feasible: it would violate constraint (8.8b) for the other hubs.

What the solver says. The optimum of the LP relaxation is $z(LP) = 25/2$, which strengthening with $x_{ij} \leq 1$ improves to $z(LP^+) = 1015/78 \approx 13.0$. The MILP has optimum $z(MILP) = 19$, with $\tilde{x}_{13} = \tilde{x}_{21} = \tilde{x}_{31} = 1$, $\tilde{y}_1 = \tilde{y}_3 = 1$, $\tilde{z}_1 = 5$, $\tilde{z}_3 = 2$: hubs 1 and 3 are activated (not 1 and 2 as in the heuristic), with terminal 1 alone on hub 3 (the cheapest for it, $c_{13} = 2$) and terminals 2 and 3 together on hub 1. The heuristic gap is 5.3%: here next-fit, which ignores costs, still comes reasonably close to the optimum.

UB (heuristic)	LB (hand dual)	$z(LP)$	$z(LP^+)$	$z(MILP)$	heuristic gap
20	15/2	25/2	1015/78	19	5.3%

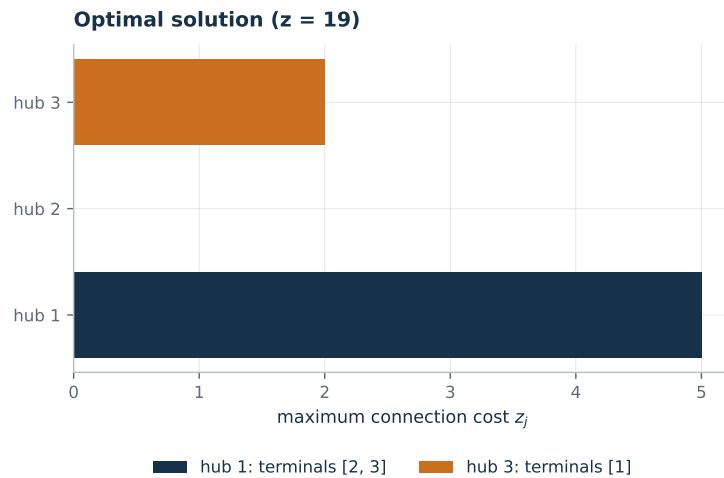


Figure 8.4: Optimal solution: maximum connection cost of each activated hub, with the terminals assigned.

Additional considerations

- The disaggregated constraints $x_{ij} \leq y_j$ (for every i, j) are valid inequalities implied by the aggregated constraint (8.7c): they give the same activation logic in a different form, and question 4a below checks that they do not change the optimum on this instance.
- Setting $M_j = \max_i c_{ij}$, the constraints $z_j \leq M_j y_j$ are not valid inequalities for the model (which allows $z_j > 0$ even with $y_j = 0$), but they are *optimality-preserving constraints*: in every optimum $z_j = 0$ when $y_j = 0$, because the objective minimizes z_j .

Additional modelling questions

Modelling question 8.7 — disaggregated activation link

Add the disaggregated links $x_{ij} \leq y_j$ to model (8.7). Does the optimum for the instance change? Does the relaxation change? And what happens if, instead of adding them, one replaces the aggregated constraint (8.7c) by them?

Modelling question 8.8 — forbidden connection

Terminal 1 cannot be connected to hub 2 (a security requirement). How is this modelled? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	20	heuristic solution (UB)
LB	$\frac{15}{2}$	certified bound (LB)
$z(MILP)$	19	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 5.3\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 65.8\%.$$

The sandwich on variant 4b

Forbidding a connection means removing a column from the primal, hence *removing* a constraint from the dual: α_1 no longer has to stand the comparison with the forbidden hub and can rise. It is given one γ on each of the hubs left to it — each hub has a budget of 1 and no other terminal uses it — and the bound grows. This is the case where one more constraint in the primal *improves* the certificate, instead of leaving it still.

	value	what it is
UB	20	heuristic solution
LB	$\frac{21}{2}$	dual certificate built by hand
$z(LP)$	$\frac{25}{2}$	relaxation without the bounds
$z(LP^+)$	$\frac{79}{6}$	relaxation with the bounds
$z(MILP)$	19	optimum of the MILP

Production planning

Class: MILP

Script: one per problem, `python/fam09_1_lotsizing.py` ...

[Online page](#)

The family

Three problems in which one decides *how much* to produce, not only *whether* to do something. The quantity variables are continuous or integer, and binaries sit on top of them to switch on, to limit or to reward: the production run with its fixed cost (problem 1), the workforce with the hirings (2), the minimum lot per type with a bonus for variety (3).

The common feature is the *balance*: a quantity that comes in, one that goes out and one that stays. In problems 1 and 2 it is the inventory balance, $x_t + s_{t-1} - s_t = d_t$; in problem 2 there is also the workforce balance, $y_t - y_{t-1} - z_t = 0$. These are equality constraints, and their duals are free variables: it is the first family of the course in which this happens systematically.

The links between variables that come back here:

- **fixed cost** (problem 1): $x_t \leq M_t y_t$, with M_t read off the data (the residual demand) and not picked at random;
- **integer counts** (problem 2): the workforce y_t and the hirings z_t are numbers of people, and the hours available are $r y_t$;
- **minimum lot** (problem 3): the semi-integer variable $q_j y_j \leq x_j \leq M_j y_j$, which is zero or at least q_j ;
- **counting the types and if and only if** (problem 3): the variety bonus is collected only if the count of active types reaches two, and the missing direction is imposed by optimality because the bonus is positive.

For each of them: model, proof of the links, instance, constructive heuristic, dual of the LP relaxation with a hand-built dual solution, solver solution, additional considerations and two modelling questions with their answers. All the numbers come from the scripts `fam09_1_lotsizing.py`, `fam09_2_workforce.py` and `fam09_3_vehicles.py` in the `python/` folder, one per problem, each with its own notebook.

Notation of the family

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of periods, indexed by $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	demand of period t
p_t	$\in \mathbb{Q}_{> 0}$	unit production cost in period t
q_t	$\in \mathbb{Q}_{\geq 0}$	fixed set-up cost of a run in period t
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost of one unit at the end of period t
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	initial inventory and required final inventory
M_t	$\in \mathbb{Q}_{> 0}$	largest useful production in period t : $\sum_{\tau \geq t} d_\tau + r_n$
m_0	$\in \mathbb{Z}_{\geq 0}$	workers on duty at the start of the horizon
w	$\in \mathbb{Q}_{> 0}$	wage of one worker per period
u	$\in \mathbb{Q}_{\geq 0}$	cost of hiring one worker
r	$\in \mathbb{Q}_{> 0}$	hours worked by one worker in one period
g	$\in \mathbb{Q}_{> 0}$	labour hours needed for one unit of product
s	$\in \mathbb{Z}_{\geq 1}$	number of product types, indexed by j
k	$\in \mathbb{Z}_{\geq 1}$	number of resources, indexed by i
a_{ij}	$\in \mathbb{Q}_{\geq 0}$	amount of resource i per unit of type j
b_i	$\in \mathbb{Q}_{> 0}$	availability of resource i
$\bar{p}_j$	$\in \mathbb{Q}_{> 0}$	unit profit of type j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	minimum lot of type j , if it is produced
$\bar{r}$	$\in \mathbb{Q}_{> 0}$	bonus if at least two different types are produced

9.1 Lot sizing with a fixed set-up cost

Script: `python/fam09_1_lotsizing.py`[Notebook in Colab](#)

Problem 9.1 — lot sizing with a fixed cost

A company must plan the production of a single product over $n \in \mathbb{Z}_{\geq 1}$ periods. For every period $t \in \{1, 2, \dots, n\}$, the value $d_t \in \mathbb{Q}_{\geq 0}$ is the demand to be met, $p_t \in \mathbb{Q}_{> 0}$ the cost of producing one unit and $q_t \in \mathbb{Q}_{\geq 0}$ the fixed cost to be paid if production is started in that period. For every period $t \in \{1, 2, \dots, n-1\}$, the value $h_t \in \mathbb{Q}_{\geq 0}$ is the cost of keeping one unit in stock from the end of period t to the start of the next one. The initial inventory is $r_0 \in \mathbb{Q}_{\geq 0}$ and the one required at the end of the horizon is $r_n \in \mathbb{Q}_{\geq 0}$. The company wants to decide how much to produce in every period so as to meet all the demand at minimum total cost.

The problem in words. We *decide* how much to produce in every period, and therefore how much stays in stock. *The objective*: minimum total cost (production, set-ups and storage). *The constraints*: the demand of every period must be met exactly, with what is produced and with what has been kept aside; and one cannot produce without paying the fixed set-up cost. It is the *lot sizing* problem with set-up.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of periods, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Q}_{\geq 0}$	demand of period t
p_t	$\in \mathbb{Q}_{> 0}$	unit production cost in period t
q_t	$\in \mathbb{Q}_{\geq 0}$	fixed set-up cost in period t
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost at the end of period t , $t \leq n - 1$
r_0, r_n	$\in \mathbb{Q}_{\geq 0}$	initial and required final inventory

Decision variables

We introduce the following n non-negative continuous variables:

$$x_t = \text{units produced in period } t, \quad \forall t \in \{1, 2, \dots, n\},$$

the following $n - 1$ non-negative continuous variables:

$$s_t = \text{units in stock at the end of period } t, \quad \forall t \in \{1, 2, \dots, n - 1\},$$

and the following n binary variables:

$$y_t = \begin{cases} 1 & \text{if production is started in period } t, \\ 0 & \text{otherwise,} \end{cases} \quad \forall t \in \{1, 2, \dots, n\}.$$

Using these variables, a MILP model for the problem reads as follows:

Model 9.1 — lot sizing with a fixed cost

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^n q_t y_t + \sum_{t=1}^{n-1} h_t s_t \quad (9.1a)$$

$$\text{subject to } x_1 - s_1 = d_1 - r_0, \quad (9.1b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, 3, \dots, n - 1\}, \quad (9.1c)$$

$$x_n + s_{n-1} = d_n + r_n, \quad (9.1d)$$

$$-x_t + M_t y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1e)$$

$$x_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.1f)$$

$$s_t \geq 0, \quad \forall t \in \{1, 2, \dots, n - 1\}, \quad (9.1g)$$

$$y_t \in \{0, 1\}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.1h)$$

Description. The objective function (9.1a) adds up the production cost, the cost of the set-ups and the storage cost. Constraints (9.1b)–(9.1d) are the inventory balance, one per period (n equality constraints): what one had plus what one produces minus what one sells is what is left. Constraints (9.1e) link quantity and set-up (n linear constraints).

The link between the variables. Constraint (9.1e) reads $x_t \leq M_t y_t$. If $y_t = 0$ then $x_t \leq 0$ and, together with $x_t \geq 0$, one gets $x_t = 0$: without a set-up nothing is produced. If $y_t = 1$ the constraint becomes $x_t \leq M_t$ and is not restrictive, provided M_t is chosen large enough. The other direction — if $x_t = 0$ then $y_t = 0$ — is not imposed by any constraint, but follows from optimality: if $x_t = 0$ and $y_t = 1$, setting $y_t = 0$ stays feasible and saves $q_t \geq 0$.

A valid M_t must be at least the largest quantity that it is worth producing in period t . In an optimal solution one never produces more than the demand still to be covered, final inventory included:

$$M_t = \sum_{\tau=t}^n d_{\tau} + r_n.$$

In the first period the initial inventory, which is a datum, can be subtracted: $M_1 = \sum_{\tau=1}^n d_{\tau} + r_n - r_0$. From the second period on the incoming inventory is not known in advance, and the bound stays the one above. On the instance $r_0 = 0$, so the two values coincide. Any larger value is still valid but weakens the LP relaxation, and any smaller value may cut off optimal solutions. On the instance, $M = (110, 90, 80, 50, 10)$: the big-M of the last period is 10, not 110.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("lot_sizing")
2 x = m.addVars(n, name="x")
3 s = m.addVars(n - 1, name="s")
4 y = m.addVars(n, vtype=GRB.BINARY, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6                 + gp.quicksum(q[t] * y[t] for t in range(n))
7                 + gp.quicksum(h[t] * s[t] for t in range(n - 1)), GRB.
MINIMIZE)
8 m.addConstr(x[0] - s[0] == d[0] - r0, name="balance[0]")
9 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
name="balance")
10 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1] + rn, name=f"balance[{n -
1}]")
11 m.addConstrs((-x[t] + M[t] * y[t] >= 0 for t in range(n)), name="run")

```

The instance

$n = 5$ periods (days), $r_0 = r_n = 0$:

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
d_t	20	10	30	40	10
p_t	2	3	2	3	2
q_t	50	50	50	50	50
M_t	110	90	80	50	10

	$t = 1$	$t = 2$	$t = 3$	$t = 4$
h_t	1	1	1	1

The total demand is 110 units.

Model 9.1 on the instance

$$\begin{aligned}
 & \min 2x_1 + 3x_2 + 2x_3 + 3x_4 + 2x_5 + s_1 + s_2 + s_3 + s_4 + 50y_1 + 50y_2 + 50y_3 + 50y_4 + 50y_5 \\
 \text{subject to } & x_1 - s_1 = 20 \\
 & x_2 + s_1 - s_2 = 10 \\
 & x_3 + s_2 - s_3 = 30 \\
 & x_4 + s_3 - s_4 = 40 \\
 & x_5 + s_4 = 10 \\
 & -x_1 + 110y_1 \geq 0 \\
 & -x_2 + 90y_2 \geq 0 \\
 & -x_3 + 80y_3 \geq 0 \\
 & -x_4 + 50y_4 \geq 0 \\
 & -x_5 + 10y_5 \geq 0 \\
 & x_1, x_2, x_3, x_4, x_5 \geq 0 \\
 & s_1, s_2, s_3, s_4 \geq 0 \\
 & y_1, y_2, y_3, y_4, y_5 \in \{0, 1\}
 \end{aligned}$$

Dual on the instance

$$\begin{aligned}
 & \max 20\mu_1 + 10\mu_2 + 30\mu_3 + 40\mu_4 + 10\mu_5 \\
 \text{subject to } & \mu_1 - \pi_1 \leq 2 \\
 & \mu_2 - \pi_2 \leq 3 \\
 & \mu_3 - \pi_3 \leq 2 \\
 & \mu_4 - \pi_4 \leq 3 \\
 & \mu_5 - \pi_5 \leq 2 \\
 & 110\pi_1 \leq 50 \\
 & 90\pi_2 \leq 50 \\
 & 80\pi_3 \leq 50 \\
 & 50\pi_4 \leq 50 \\
 & 10\pi_5 \leq 50 \\
 & -\mu_1 + \mu_2 \leq 1 \\
 & -\mu_2 + \mu_3 \leq 1 \\
 & -\mu_3 + \mu_4 \leq 1 \\
 & -\mu_4 + \mu_5 \leq 1 \\
 & \mu_1, \mu_2, \mu_3, \mu_4, \mu_5 \geq 0 \\
 & \pi_1, \pi_2, \pi_3, \pi_4, \pi_5 \geq 0
 \end{aligned}$$

Constructive heuristics: the primal bound

Two classic rules, both feasible by construction.

(a) **Lot-for-lot.** Every day one produces exactly the demand of the day: no inventory, but a set-up every day. The cost is

$$\sum_{t=1}^5 p_t d_t + \sum_{t=1}^5 q_t = 270 + 250 = 520.$$

(b) **Least unit cost.** One starts from the first uncovered period and chooses to cover with a single run the k periods that minimise the average cost per unit, that is

$$\frac{q + \sum_{i=1}^{k-1} h \cdot (\text{units kept } i \text{ periods})}{\sum_{i=0}^{k-1} d_{t+i}},$$

then one restarts from the first period that is still uncovered.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

The running time is $O(n^2)$. On the instance:

- period 1: covers up to period 2, quantity 30, average cost 2 per unit;
- period 3: covers up to period 4, quantity 70, average cost ≈ 1.286 ;
- period 5: covers period 5 alone, quantity 10, average cost 5.

Runs are started on days 1, 3, 5; the total cost is 420. Keeping the better of the two heuristics,

$$z(MILP) \leq UB = 420.$$

LP relaxation and dual: the dual bound

We associate a free dual variable μ_t with each balance constraint (9.1b)–(9.1d) (equalities) and a non-negative dual variable π_t with each constraint (9.1e) ($\geq$ direction). With $b_1 = d_1 - r_0$, $b_t = d_t$ for $2 \leq t \leq n-1$ and $b_n = d_n + r_n$, the dual of the LP relaxation is:

Dual of the LP relaxation of model 9.1

$$\max \sum_{t=1}^n b_t \mu_t \quad (9.2a)$$

$$\text{subject to } \mu_t - \pi_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2b)$$

$$M_t \pi_t \leq q_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2c)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.2d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.2e)$$

$$\pi_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.2f)$$

Description. The variable μ_t is the value attached to one unit available in period t , and π_t is the price of one unit of setup capacity. The objective (9.2a) prices the demand of each period at those values. Constraints (9.2b), one per period, are the columns of the x_t : producing one unit in period t is worth μ_t and consumes setup capacity at price π_t , and the balance cannot exceed the unit cost p_t . Constraints (9.2c) are the columns of the y_t : all the capacity M_t that a setup makes available, priced at π_t , cannot be worth more than the fixed cost q_t . Constraints (9.2d) are the columns of the s_t : carrying one unit from period t to $t + 1$ earns $\mu_{t+1} - \mu_t$, which cannot exceed the holding cost h_t . The μ_t are free because the balances are equalities; the π_t are non-negative because the setup constraints are of type $\geq$ in a minimisation.

A hand-built dual solution. Set $\bar{\pi}_t = 0$ for every t : constraints (9.2c) are satisfied because $q_t \geq 0$, and the set-ups become free. What is left is $\mu_t \leq p_t$ and $\mu_{t+1} \leq \mu_t + h_t$: the largest feasible value is built forward,

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t) \quad \text{for } t \geq 2.$$

The reading is direct: $\bar{\mu}_t$ is the lowest unit cost of having one unit available in period t , either by producing it then, or by producing it earlier and keeping it in stock. On the instance $h = 1$ is low and the production costs are 2, 3, 2, 3, 2, so

$$\bar{\mu} = (2, 3, 2, 3, 2),$$

and the dual value is

$$LB = \sum_{t=1}^5 d_t \bar{\mu}_t = 2 \cdot 20 + 3 \cdot 10 + 2 \cdot 30 + 3 \cdot 40 + 2 \cdot 10 = 270.$$

It is the production cost if the set-ups were free: a valid bound, and deliberately optimistic. The LP relaxation, which pays for the set-ups in fractions ($\pi_t = q_t/M_t$ is feasible), goes higher:

$$z(LP) = z(LP^+) = \frac{3890}{11} \approx 353.64.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
run y_t	1	0	1	0	0
production x_t	30	0	80	0	0
inventory s_t	10	0	50	10	—

The cost is $z(MILP) = 390$, and the chain of bounds is

$$270 = LB \leq z(LP) = \frac{3890}{11} \leq z(MILP) = 390 \leq UB = 420.$$

The heuristic gap is $(420 - 390)/390 = 7.7\%$; the certified gap, before solving the MILP, is $(420 - 270)/390 = 38.5\%$.

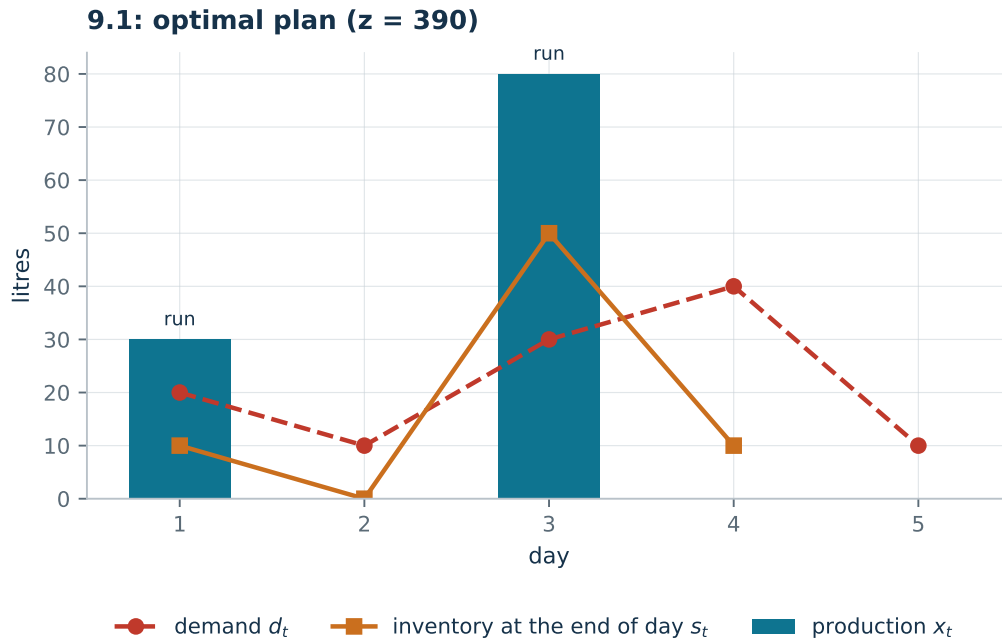


Figure 9.1: Optimal plan: production, demand and inventory at the end of the day. The two runs cover, the first one, days 1–2 and, the second one, days 3–5.

Additional considerations

- The model is a special case of the problem solved in $O(n^2)$ time by the Wagner–Whitin dynamic program. Least unit cost is *not* that algorithm: it is a myopic rule that looks at one run at a time, and in general it does not find the optimum. On the instance it indeed stops at 420 against 390.
- The valid inequality $x_t \leq M_t$, obtained from constraint (9.1e) with $y_t \leq 1$, adds nothing: M_t is already the largest useful production. Indeed $z(LP) = z(LP^+)$.
- An alternative formulation, much tighter, uses the variables $x_{t\tau}$ = "units produced in t and sold in τ " (the *facility location* formulation of lot sizing). Its LP relaxation is integral, but it has $O(n^2)$ variables: it is the typical trade-off between the size of the model and the quality of the relaxation.

Additional modelling questions

Modelling question 9.1 — daily capacity

The plant cannot produce more than 35 units a day. How does the model change? What is the new optimum for the instance?

Modelling question 9.2 — minimum lot

If one produces on a given day, at least 25 units must be produced (below that threshold the set-up is not justified). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	420	heuristic solution (UB)
LB	270	certified bound (LB)
$z(MILP)$	390	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 7.7\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 38.5\%.$$

The sandwich on variant 1a

The cap $x_t \leq 35$ adds to the dual a family $\nu_t \geq 0$ with right-hand side -35 : it enters the objective with a negative sign and loosens the column of x_t . Raising ν_t by one unit lifts the cap on μ_t and collects d_t , but costs 35, so it only pays where the demand exceeds the capacity; and on this instance not even the busiest day justifies it, because the chain of the stocks already keeps μ_t below the cap. The certificate is therefore the one of the base problem, $LB = 270$. What changes is the heuristic: on day four lot-for-lot would exceed the cap, and the 5 excess units are anticipated to day three, which holds them in stock.

	value	what it is
UB	520	heuristic solution
LB	270	dual certificate built by hand
$z(LP)$	$\frac{305365}{792}$	relaxation without the bounds
$z(LP^+)$	$\frac{305365}{792}$	relaxation with the bounds
$z(MILP)$	470	optimum of the MILP

9.2 Production and workforce: two equivalent formulations

Script: `python/fam09_2_workforce.py`

[Notebook in Colab](#)

Problem 9.2 — production and workforce

A shoe factory must plan its production over $n \in \mathbb{Z}_{\geq 1}$ months. For every month $t \in \{1, 2, \dots, n\}$, the value $d_t \in \mathbb{Z}_{\geq 0}$ is the demand in pairs and $p_t \in \mathbb{Q}_{>0}$ the cost of the raw materials for one pair. For every month $t \in \{1, 2, \dots, n-1\}$, the value $h_t \in \mathbb{Q}_{\geq 0}$ is the cost of keeping one pair in stock at the end of the month. At the start of the horizon the company has $m_0 \in \mathbb{Z}_{\geq 0}$ workers; every worker works $r \in \mathbb{Q}_{>0}$ hours a month and costs $w \in \mathbb{Q}_{>0}$ euros a month, and producing one pair requires $g \in \mathbb{Q}_{>0}$ labour hours. At the start of every month workers may be hired, at a cost of $u \in \mathbb{Q}_{\geq 0}$ euros each; nobody is laid off. The company wants to decide how much to produce and how many workers to employ

in every month, at minimum total cost.

The problem in words. We *decide* how much to produce, how much to keep in stock and how many workers to hire. *The objective:* minimum total cost (raw materials, storage, wages and hirings). *The constraints:* the demand must be met exactly; the production of a month cannot require more hours than the workers on duty can do.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of months, $t \in \{1, 2, \dots, n\}$
d_t	$\in \mathbb{Z}_{\geq 0}$	demand (pairs) of month t
p_t	$\in \mathbb{Q}_{>0}$	cost of the raw materials for one pair
h_t	$\in \mathbb{Q}_{\geq 0}$	storage cost at the end of month t
m_0	$\in \mathbb{Z}_{\geq 0}$	workers on duty at the start
w	$\in \mathbb{Q}_{>0}$	monthly wage of one worker
u	$\in \mathbb{Q}_{\geq 0}$	cost of hiring one worker
r	$\in \mathbb{Q}_{>0}$	hours worked by one worker in one month
g	$\in \mathbb{Q}_{>0}$	labour hours for one pair

Two formulations

The same decision can be written in two ways, and it is worth putting them side by side because it is the first case in the course in which two apparently different models describe the same set of plans.

Formulation A: the hirings. The staff variables are

$$z_t = \text{workers hired at the start of month } t, \quad \forall t \in \{1, 2, \dots, n\},$$

integer and non-negative. A worker hired in month t stays on duty until the end of the horizon, so they cost u once plus w for each of the remaining $n - t + 1$ months. The wage of the m_0 initial workers, $m_0 w n$, is a constant term that does not depend on the decisions: it is left out of the model and added to the final value.

Formulation B: the workforce. The staff variables are

$$y_t = \text{workers on duty in month } t, \quad \forall t \in \{1, 2, \dots, n\},$$

integer and non-negative, with $y_t \geq y_{t-1}$ because nobody is laid off. The cost is $w y_t$ every month, plus u for every hiring, that is $u(y_n - m_0)$ in total, because the hirings are the increments of the workforce and the telescoping sum leaves only the endpoints.

Model 9.2A — with the hirings

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n (u + w(n-t+1)) z_t \quad (9.3a)$$

$$\text{subject to } x_1 - s_1 = d_1, \quad (9.3b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, 3, \dots, n-1\}, \quad (9.3c)$$

$$x_n + s_{n-1} = d_n, \quad (9.3d)$$

$$-g x_t + r \sum_{j=1}^t z_j \geq -r m_0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3e)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.3f)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.3g)$$

$$z_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.3h)$$

Model 9.2B — with the workforce

$$\min \sum_{t=1}^n p_t x_t + \sum_{t=1}^{n-1} h_t s_t + \sum_{t=1}^n w y_t + u(y_n - m_0) \quad (9.4a)$$

$$\text{subject to } x_1 - s_1 = d_1, \quad (9.4b)$$

$$x_t + s_{t-1} - s_t = d_t, \quad \forall t \in \{2, 3, \dots, n-1\}, \quad (9.4c)$$

$$x_n + s_{n-1} = d_n, \quad (9.4d)$$

$$-g x_t + r y_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4e)$$

$$y_1 \geq m_0, \quad (9.4f)$$

$$-y_{t-1} + y_t \geq 0, \quad \forall t \in \{2, 3, \dots, n\}, \quad (9.4g)$$

$$x_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.4h)$$

$$s_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.4i)$$

$$y_t \in \mathbb{Z}_{\geq 0}, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.4j)$$

Description. The two formulations share the stock balance constraints, (9.3b)–(9.3d) and (9.4b)–(9.4d): one per month, they say that what is produced plus what is in stock covers the demand exactly. Constraints (9.3e) and (9.4e), one per month, are the available hours: a month's production cannot require more hours than the workers on the payroll can do. Formulation *B* has in addition constraint (9.4f), which starts from the m_0 initial workers, and

constraints (9.4g), one per month from the second on, which forbid layoffs. In A those same facts are hidden in the domain of the z_t , which are non-negative.

The two formulations describe the same problem

The correspondence is

$$y_t = m_0 + \sum_{j=1}^t z_j, \quad \text{that is} \quad z_t = y_t - y_{t-1} \quad (\text{with } y_0 = m_0).$$

Feasibility. With this substitution constraint (9.3e) becomes exactly (9.4e). The conditions $z_t \geq 0$ become $y_t \geq y_{t-1}$, that is (9.4g), and $y_1 \geq m_0$, that is (9.4f). The balance constraints contain no staff variables and stay identical. The correspondence is therefore a bijection between the feasible plans of the two models.

Cost. It is enough to compute the staff cost in the two ways. In B it is

$$\sum_{t=1}^n w y_t + u (y_n - m_0) = \sum_{t=1}^n w \left(m_0 + \sum_{j \leq t} z_j \right) + u \sum_{t=1}^n z_t.$$

The first term equals $m_0 w n + w \sum_{t=1}^n (n - t + 1) z_t$, because z_j appears in every month from j onwards, that is $n - j + 1$ times. Adding the second one gives

$$m_0 w n + \sum_{t=1}^n (u + w (n - t + 1)) z_t,$$

that is the constant $m_0 w n$ plus exactly the third term of (9.3a). The two objective functions therefore differ by that constant alone, and the two models have the same optima.

Why keep both

Formulation A has fewer constraints (no monotonicity) but cost coefficients that depend on the period; formulation B has uniform coefficients and extends better if layoffs are added (one only needs a second family of variables $\ell_t \geq 0$ and the balance $y_t = y_{t-1} + z_t - \ell_t$). On the instance even the *relaxations* coincide: $z(LP) = 15\,960$ for both. Two formulations that are equivalent on the integers are not always equivalent on the relaxation; here they are, and the check must be made, not assumed.

The model in gurobipy

```

1 m = gp.Model("workforce_B")
2 x = m.addVars(n, vtype=GRB.INTEGER, name="x")
3 s = m.addVars(n - 1, vtype=GRB.INTEGER, name="s")
4 y = m.addVars(n, vtype=GRB.INTEGER, name="y")
5 m.setObjective(gp.quicksum(p[t] * x[t] for t in range(n))
6                 + gp.quicksum(h[t] * s[t] for t in range(n - 1))
7                 + gp.quicksum(w * y[t] for t in range(n)) + u * (y[n -
8                   1] - m0), GRB.MINIMIZE)
9 m.addConstr(x[0] - s[0] == d[0], name="balance[0]")
10 m.addConstrs((x[t] + s[t - 1] - s[t] == d[t] for t in range(1, n - 1)),
11              name="balance")

```

```

10 m.addConstr(x[n - 1] + s[n - 2] == d[n - 1], name=f"balance[{n - 1}]")
11 m.addConstrs((-g * x[t] + r * y[t] >= 0 for t in range(n)), name="hours")
12 m.addConstr(y[0] >= m0, name="initial_workforce")
13 m.addConstrs((-y[t - 1] + y[t] >= 0 for t in range(1, n)), name="no_layoffs")

```

The instance

$n = 3$ months, $m_0 = 2$ workers, $w = 1500$, $u = 100$, $r = 160$ hours, $g = 4$ hours per pair, $h_t = 3$:

	$t = 1$	$t = 2$	$t = 3$
d_t	60	100	140
p_t	15	15	15

With two workers the initial capacity is $2 \cdot 160/4 = 80$ pairs a month: enough for the first month, not for the other two.

Model 9.2 on the instance

$$\begin{aligned}
 & \min 15x_1 + 15x_2 + 15x_3 + 3s_1 + 3s_2 + 1500y_1 + 1500y_2 + 1600y_3 \\
 & \text{subject to} \quad x_1 - s_1 = 60 \\
 & \quad x_2 + s_1 - s_2 = 100 \\
 & \quad x_3 + s_2 = 140 \\
 & \quad -4x_1 + 160y_1 \geq 0 \\
 & \quad -4x_2 + 160y_2 \geq 0 \\
 & \quad -4x_3 + 160y_3 \geq 0 \\
 & \quad y_1 \geq 2 \\
 & \quad -y_1 + y_2 \geq 0 \\
 & \quad -y_2 + y_3 \geq 0 \\
 & \quad x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0} \\
 & \quad s_1, s_2 \in \mathbb{Z}_{\geq 0} \\
 & \quad y_1, y_2, y_3 \in \mathbb{Z}_{\geq 0}
 \end{aligned}$$

Dual on the instance

$$\begin{aligned}
& \max 60\mu_1 + 100\mu_2 + 140\mu_3 - 320\nu_1 - 320\nu_2 - 320\nu_3 \\
& \text{subject to} \quad \mu_1 \qquad \qquad \qquad -4\nu_1 \qquad \qquad \leq 15 \\
& \qquad \qquad \mu_2 \qquad \qquad \qquad -4\nu_2 \qquad \qquad \leq 15 \\
& \qquad \qquad \qquad \mu_3 \qquad \qquad \qquad -4\nu_3 \leq 15 \\
& \qquad -\mu_1 \quad +\mu_2 \qquad \qquad \qquad \leq 3 \\
& \qquad \qquad -\mu_2 \quad +\mu_3 \qquad \qquad \leq 3 \\
& \qquad \qquad \qquad 160\nu_1 + 160\nu_2 + 160\nu_3 \leq 4600 \\
& \qquad \qquad \qquad \qquad 160\nu_2 + 160\nu_3 \leq 3100 \\
& \qquad \qquad \qquad \qquad \qquad 160\nu_3 \leq 1600 \\
& \qquad \mu_1, \quad \mu_2, \quad \mu_3 \qquad \qquad \qquad \geq 0 \\
& \qquad \qquad \nu_1, \quad \nu_2, \quad \nu_3 \geq 0
\end{aligned}$$

Constructive heuristic: the primal bound

"Just in time" production: every month one produces exactly the demand and hires the minimum number of workers needed.

It is a constructive heuristic of the family seen in chapter "Constructive heuristics": a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance: month 1, $\lceil 4 \cdot 60/160 \rceil = 2$ workers, no hiring; month 2, $\lceil 4 \cdot 100/160 \rceil = 3$, one hiring; month 3, $\lceil 4 \cdot 140/160 \rceil = 4$, one more hiring. The cost, including the constant term $m_0 w n = 9000$, is

$$z(MILP) \leq UB = 18\,200.$$

LP relaxation and dual: the dual bound

On formulation A , we associate a free dual variable μ_t with each balance and a non-negative one ν_t with each constraint (9.3e). The dual of the LP relaxation is:

Dual of the LP relaxation of model 9.2A

$$\max \sum_{t=1}^n d_t \mu_t - r m_0 \sum_{t=1}^n \nu_t \quad (9.5a)$$

$$\text{subject to} \quad \mu_t - g \nu_t \leq p_t, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5b)$$

$$-\mu_t + \mu_{t+1} \leq h_t, \quad \forall t \in \{1, 2, \dots, n-1\}, \quad (9.5c)$$

$$r \sum_{t=j}^n \nu_t \leq u + w(n-j+1), \quad \forall j \in \{1, 2, \dots, n\}, \quad (9.5d)$$

$$\mu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}, \quad (9.5e)$$

$$\nu_t \geq 0, \quad \forall t \in \{1, 2, \dots, n\}. \quad (9.5f)$$

Description. The variable μ_t is the value of one pair available in month t and ν_t the price of one working hour in month t . The objective (9.5a) prices the demand at those values and

subtracts $r m_0 \sum_t \nu_t$, that is, the hours the m_0 initial workers supply for free. Constraints (9.5b) are the columns of the x_t : producing one pair in month t is worth μ_t and consumes g hours at price ν_t , and the balance cannot exceed the raw-material cost p_t . Constraints (9.5c) are the columns of the s_t : holding one pair in stock earns $\mu_{t+1} - \mu_t$, which cannot exceed h_t . Constraints (9.5d) are the columns of the z_j : a worker hired at the beginning of month j supplies r hours in each month from j onwards, and their total value cannot exceed what that hire costs, namely $u + w(n - j + 1)$.

A hand-built dual solution. Set $\bar{\nu}_t = 0$: the working hours are not priced, constraints (9.5d) are satisfied because the right-hand side is positive, and what is left is $\mu_t \leq p_t$ and $\mu_{t+1} \leq \mu_t + h_t$. As in problem 9.1, the largest feasible value is

$$\bar{\mu}_1 = p_1, \quad \bar{\mu}_t = \min(\bar{\mu}_{t-1} + h_{t-1}, p_t).$$

On the instance $\bar{\mu} = (15, 15, 15)$ and

$$\sum_{t=1}^n d_t \bar{\mu}_t = 15 \cdot 300 = 4500,$$

to which the constant term 9000 must be added:

$$LB = 4500 + 9000 = 13\,500.$$

It is the cost of the raw materials alone plus the wage of the workers already on duty: the extra labour is given away. The LP relaxation, which pays for it in fractions of a person, reaches $z(LP) = z(LP^+) = 15\,960$.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$t = 1$	$t = 2$	$t = 3$
production x_t	60	120	120
workforce y_t	2	3	3
hirings z_t	0	1	0
inventory s_t	0	20	—

The cost is $z(MILP) = 16\,660$, with the chain

$$13\,500 = LB \leq z(LP) = 15\,960 \leq z(MILP) = 16\,660 \leq UB = 18\,200.$$

The heuristic gap is 9.2%. The optimal plan hires *one* worker instead of two and moves twenty pairs from the third month to the second: paying 3 euros of storage for twenty pairs costs 60 euros against the 1600 of a hiring in the third month.

Additional considerations

- The monotonicity constraint (9.4g) is what ties the months together *on the staffing side*: if one could lay off at no cost, every month would pick on its own the smallest staff able to produce x_t , and that family would disappear. The problem would not split into n independent problems, though: the stocks s_t still tie each month to the next.

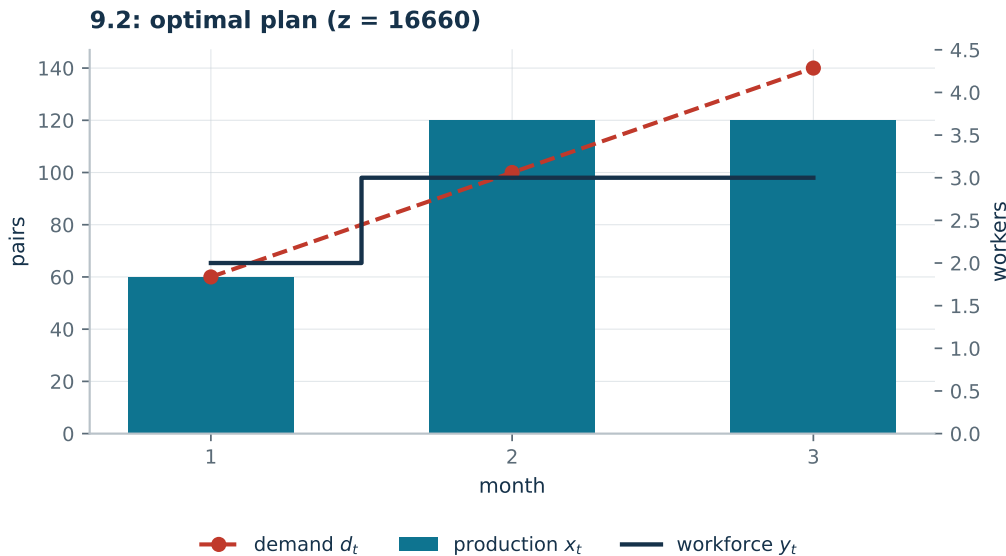


Figure 9.2: Optimal plan: production, demand and workforce. The workforce goes up once only, and the early production covers the final peak.

- The variables x_t and s_t are declared integer because pairs of shoes cannot be split. On this model they could be left continuous without changing the optimum — but not because the matrix is totally unimodular: it is not, as the relaxation shows, worth 15 960 against an integer optimum of 16 660. The reason is more local: once the staff y is fixed at integer values, what is left in (x, s) is a balance problem with an interval matrix, totally unimodular, and integer right-hand sides (the demands, and the capacities $r y_t/g = 40 y_t$); an optimal vertex of it is therefore integer. It is an argument about the subproblem, not about the whole model.
- The constant term $m_0 w n$ must be remembered in every comparison: forgetting it makes formulation A look much cheaper than B .

Additional modelling questions

Modelling question 9.3 — very expensive hirings

The hiring cost goes up from 100 to 3000 euros (selection and training). How does the optimal plan change?

Modelling question 9.4 — overtime

Every worker may do up to 40 hours of overtime a month, paid 25 euros an hour. How does the model change? Is it worth using them?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	18200	heuristic solution (UB)
LB	13500	certified bound (LB)
$z(MILP)$	16660	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 9.2\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 28.2\%.$$

The sandwich on variant 2a

The variant adds no constraint: it changes a datum, $u = 3000$ instead of 100. Model, dual and recipe are those of the base problem, and the certificate does not notice the change, because μ_t is the lowest cost of having a pair available in month t and the hiring cost does not appear there: the bound stays 13500, while the optimum rises from 16660 to 19560. It is the limiting case of this chapter: a correct dual bound can be completely insensitive to the datum that moves the problem, and here it is the heuristic that follows it.

	value	what it is
UB	24000	heuristic solution
LB	13500	dual certificate built by hand
$z(LP)$	17490	relaxation without the bounds
$z(LP^+)$	17490	relaxation with the bounds
$z(MILP)$	19560	optimum of the MILP

9.3 Vehicles: minimum lot and a bonus for variety

Script: `python/fam09_3_vehicles.py`

[Notebook in Colab](#)

Problem 9.3 — vehicles with minimum lot and bonus

A company produces $s \in \mathbb{Z}_{\geq 1}$ types of vehicle using $k \in \mathbb{Z}_{\geq 1}$ resources. For every resource $i \in \{1, 2, \dots, k\}$ and every type $j \in \{1, 2, \dots, s\}$, the value $a_{ij} \in \mathbb{Q}_{\geq 0}$ is the amount of resource i needed for one unit of type j , and $b_i \in \mathbb{Q}_{> 0}$ is the availability of resource i . For every type j , the value $\bar{p}_j \in \mathbb{Q}_{> 0}$ is the profit of one unit and $\bar{q}_j \in \mathbb{Z}_{\geq 1}$ the minimum quantity to be produced if that type is chosen. If the production includes at least two different types, the company collects a bonus $\bar{r} \in \mathbb{Q}_{> 0}$ (a grant for diversification). The company wants to maximise the total profit.

The problem in words. We *decide* how many units of each type to produce. *The objective*: maximum profit, bonus included. *The constraints*: the resources must not be exceeded; and of a given type one produces either zero units or at least $\bar{q}_j$.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of types, $j \in \{1, 2, \dots, s\}$
k	$\in \mathbb{Z}_{\geq 1}$	number of resources, $i \in \{1, 2, \dots, k\}$
a_{ij}	$\in \mathbb{Q}_{\geq 0}$	resource i for one unit of type j
b_i	$\in \mathbb{Q}_{> 0}$	availability of resource i
$\bar{p}_j$	$\in \mathbb{Q}_{> 0}$	unit profit of type j
$\bar{q}_j$	$\in \mathbb{Z}_{\geq 1}$	minimum lot of type j
$\bar{r}$	$\in \mathbb{Q}_{> 0}$	variety bonus
M_j	$\in \mathbb{Z}_{\geq 1}$	$\min\{\lfloor b_i/a_{ij} \rfloor : a_{ij} > 0\}$: largest producible amount of type j alone

Decision variables

We introduce s non-negative integer variables, s binaries and one binary:

$$x_j = \text{units produced of type } j, \quad \forall j \in \{1, 2, \dots, s\},$$

$$y_j = \begin{cases} 1 & \text{if type } j \text{ is produced,} \\ 0 & \text{otherwise,} \end{cases} \quad z = \begin{cases} 1 & \text{if the bonus is collected,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 9.3 — vehicles with minimum lot and bonus

$$\max \sum_{j=1}^s \bar{p}_j x_j + \bar{r} z \quad (9.6a)$$

$$\text{subject to } \sum_{j=1}^s a_{ij} x_j \leq b_i, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.6b)$$

$$x_j - \bar{q}_j y_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6c)$$

$$x_j - M_j y_j \leq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6d)$$

$$-\sum_{j=1}^s y_j + 2z \leq 0, \quad (9.6e)$$

$$x_j \in \mathbb{Z}_{\geq 0}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6f)$$

$$y_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.6g)$$

$$z \in \{0, 1\}. \quad (9.6h)$$

Description. The objective (9.6a) adds up the profits of the vehicles produced and the bonus for variety. Constraints (9.6b), one per resource, are the availabilities. Constraints (9.6c) and (9.6d), one per type each, make x_j semi-integer: either zero, or at least the minimum lot $\bar{q}_j$ and at most M_j . Constraint (9.6e), a single one, says that the bonus is collected only if at least two types are active.

The link between the variables: semicontinuity. The two constraints (9.6c) and (9.6d) together say

$$\bar{q}_j y_j \leq x_j \leq M_j y_j.$$

If $y_j = 0$ both give $x_j = 0$. If $y_j = 1$ one gets $\bar{q}_j \leq x_j \leq M_j$. The variable x_j is therefore zero or between the minimum lot and the maximum: it is a *semi-integer* variable. Neither constraint alone is enough: without (9.6d) a type could be produced with $y_j = 0$; without (9.6c) the minimum lot would be empty.

The bonus is collected only with at least two types

Constraint (9.6e) reads $2z \leq \sum_{j=1}^s y_j$. If $z = 1$ then $\sum_{j=1}^s y_j \geq 2$: at least two types are activated, and by semicontinuity at least two types are actually produced (in quantity $\geq \bar{q}_j > 0$). The other direction — if there are two active types then $z = 1$ — is not imposed by the constraint, but follows from optimality: if $\sum_{j=1}^s y_j \geq 2$ and $z = 0$, setting $z = 1$ stays feasible and increases the objective by $\bar{r} > 0$. Hence in every optimal solution z is a faithful indicator. Question 3b below shows that with $\bar{r} = 0$ this argument breaks down, and z stops being faithful.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 mm = gp.Model("vehicles")
2 x = mm.addVars(n, vtype=GRB.INTEGER, name="x")
3 y = mm.addVars(n, vtype=GRB.BINARY, name="y")
4 z = mm.addVar(vtype=GRB.BINARY, name="z")
5 mm.setObjective(gp.quicksum(p[j] * x[j] for j in range(n)) + r * z, GRB
    .MAXIMIZE)
6 mm.addConstrs((gp.quicksum(a[i][j] * x[j] for j in range(n)) <= b[i]
7     for i in range(m)), name="resource")
8 mm.addConstrs((x[j] - q[j] * y[j] >= 0 for j in range(n)), name="
    minimum_lot")
9 mm.addConstrs((x[j] - M[j] * y[j] <= 0 for j in range(n)), name="
    activate")
10 mm.addConstr(-gp.quicksum(y[j] for j in range(n)) + 2 * z <= 0, name="
    bonus")

```

The instance

$s = 3$ types, $k = 2$ resources (steel in tons and labour hours), $\bar{q}_j = 10$ for every type, $\bar{r} = 500$:

a_{ij}	$j = 1$	$j = 2$	$j = 3$		$i = 1$	$i = 2$
$i = 1$ (steel)	2	3	5			
$i = 2$ (hours)	30	25	40			
					b_i	100 1200

	$j = 1$	$j = 2$	$j = 3$
$\bar{p}_j$	200	250	300
M_j	40	33	20

The big-Ms are computed from the data: $M_1 = \min(\lfloor 100/2 \rfloor, \lfloor 1200/30 \rfloor) = 40$, and likewise $M_2 = 33$, $M_3 = 20$.

Constructive heuristic: the primal bound

The problem is a maximisation, so the heuristic gives the *primal* bound, which sits below the optimum. The types are sorted by profit relative to the consumption of the tightest resource, the two best ones are activated at their minimum lot (so the bonus is guaranteed) and then one fills up with the most profitable among those activated.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance types 2 and 1 are activated at their minimum lot, then one fills up: the production is $(11, 26, 0)$, the steel runs out and 220 hours are left. The profit is 8700 plus the bonus of 500:

$$z(MILP) \geq LB = 9200.$$

LP relaxation and dual: the dual bound

The primal is a maximisation with $\leq$ and $\geq$ constraints: following the conversion table of chapter “Relaxations, duality and bounds”, we associate $\pi_i \geq 0$ with constraints (9.6b), $\lambda_j \leq 0$ with (9.6c), $\beta_j \geq 0$ with (9.6d) and $\gamma \geq 0$ with (9.6e). Writing $\lambda_j = -\ell_j$ with $\ell_j \geq 0$, the dual is:

Dual of the LP relaxation of model 9.3

$$\min \sum_{i=1}^k b_i \pi_i \quad (9.7a)$$

$$\text{subject to } \sum_{i=1}^k a_{ij} \pi_i - \ell_j + \beta_j \geq \bar{p}_j, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7b)$$

$$\bar{q}_j \ell_j - M_j \beta_j - \gamma \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7c)$$

$$2\gamma \geq \bar{r}, \quad (9.7d)$$

$$\pi_i \geq 0, \quad \forall i \in \{1, 2, \dots, k\}, \quad (9.7e)$$

$$\ell_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7f)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, s\}, \quad (9.7g)$$

$$\gamma \geq 0. \quad (9.7h)$$

Description. The variable π_i is the price of one unit of resource i ; ℓ_j and β_j are the prices of the two semicontinuity constraints of type j , and γ is the price of the bonus. The objective (9.7a) prices all the available resources at those values. Constraints (9.7b) are the columns of the x_j : the resources one unit of type j consumes, priced at π_i and corrected by the two semicontinuity constraints, must cover the profit $\bar{p}_j$ of that unit. Constraints (9.7c) are the columns of the y_j : switching on type j forces at least $\bar{q}_j$ units and allows at most M_j , and the balance of the two prices must cover the bonus γ . Constraint (9.7d) is the column of z : the bonus $\bar{r}$ is collected only with two active types, and indeed it has to be covered by 2γ .

A hand-built dual solution. For example one can proceed in three steps.

1. Constraint (9.7d) forces $\gamma \geq \bar{r}/2$: take the minimum, $\bar{\gamma} = \bar{r}/2 = 250$.
2. Set $\bar{\beta}_j = 0$ and read from constraint (9.7c) the smallest feasible $\bar{\ell}_j$,

$$\bar{\ell}_j = \frac{\bar{\gamma}}{\bar{q}_j} = \frac{250}{10} = 25 \quad \text{for every } j.$$

The reading is that every activated type "carries" its share of the bonus.

3. What is left is (9.7b), which becomes $\sum_{i=1}^k a_{ij} \pi_i \geq \bar{p}_j + \bar{\ell}_j$. One prices *a single* resource, so that it covers all the types:

$$\bar{\pi}_i = \max_j \frac{\bar{p}_j + \bar{\ell}_j}{a_{ij}}, \quad \text{bound} = b_i \bar{\pi}_i,$$

and keeps the resource giving the smaller bound.

On the instance:

$$\begin{aligned} \bar{\pi}_1 &= \max\left(\frac{225}{2}, \frac{275}{3}, \frac{325}{5}\right) = \frac{225}{2}, & b_1 \bar{\pi}_1 &= 100 \cdot \frac{225}{2} = 11\,250, \\ \bar{\pi}_2 &= \max\left(\frac{225}{30}, \frac{275}{25}, \frac{325}{40}\right) = 11, & b_2 \bar{\pi}_2 &= 1200 \cdot 11 = 13\,200. \end{aligned}$$

The better bound is the one from the steel:

$$z(MILP) \leq UB = 11\,250.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal production is (26, 16, 0): types 1 and 2 are activated, the bonus is collected, all 100 tons of steel are consumed and 1180 hours out of the 1200 available. The value is

$$z(MILP) = 200 \cdot 26 + 250 \cdot 16 + 500 = 9700,$$

and the chain of bounds is

$$9200 = LB \leq z(MILP) = 9700 \leq z(LP^+) = 9750 \leq z(LP) = \frac{20625}{2} \leq UB = 11\,250.$$

The heuristic gap is 5.2%.

Here the relaxation with the bounds beats the hand-built dual

It is the only problem in the chapter in which $z(LP^+) = 9750$ is *better* than the hand-built dual bound (11 250), and only slightly worse than the optimum (9700). The reason is that the relaxation without the bounds lets y_j and z grow above 1, and with them the bonus: $z(LP) = 20625/2 \approx 10\,312$. Adding $y_j \leq 1$ and $z \leq 1$ removes exactly that freedom. When a model contains rewarded indicators, the relaxation with the bounds is not a detail.

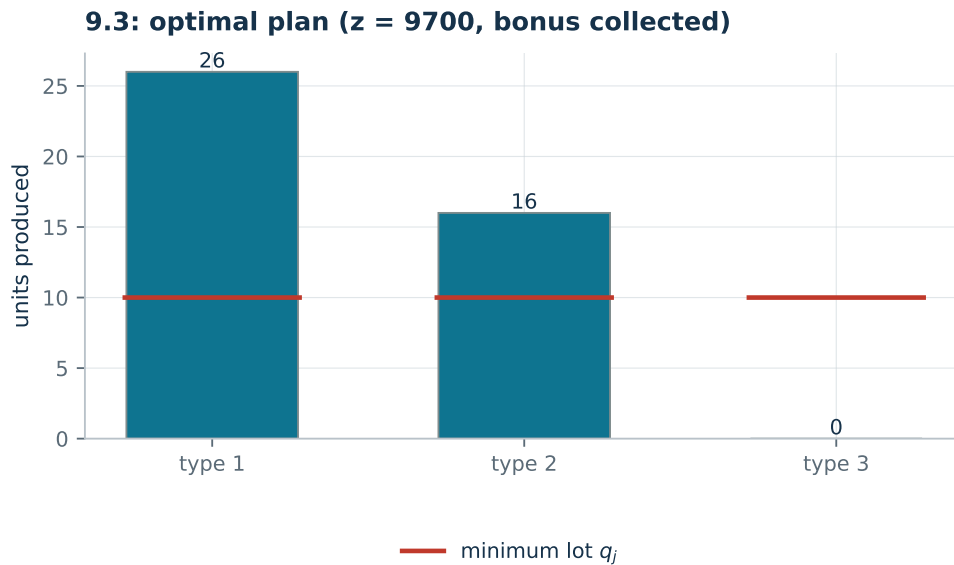


Figure 9.3: Optimal plan: units produced per type, with the minimum lot $\bar{q}_j$ marked in red. Type 3 stays at zero.

Additional considerations

- The big-M M_j is the largest producible amount of type j *alone*, not of the whole plan: it is already a much tighter value than an arbitrary constant, and indeed $M_3 = 20$ while the steel alone would allow 20 units and the hours 30.
- If for some type we had $\bar{q}_j > M_j$, that type would be impossible to produce and could be removed from the model by a check on the data. It is a pre-processing step always worth doing.
- The bonus is modelled with *one* binary and *one* constraint. With a different threshold (at least f types) one only replaces the coefficient 2 with f ; with several bonuses in brackets one would need one variable per bracket and a chain of constraints, as in the piecewise linear functions of section 3.14.

Additional modelling questions

Modelling question 9.5 — a more demanding bonus

The bonus is collected only if at least *three* different types are produced. How does the model change? What is the new optimum?

Modelling question 9.6 — zero bonus

The diversification grant is abolished, that is $\bar{r} = 0$. What happens to the variable z ?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	9200	heuristic solution (LB)
UB	11250	certified bound (UB)
$z(MILP)$	9700	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 5.2\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 21.1\%.$$

The sandwich on variant 3a

The bonus constraint touches a single column of the dual, the one of z , which now sees $\bar{r}/3$ instead of $\bar{r}/2$; the rest of the dual is identical. The recipe is the one of the base problem with $\gamma = \bar{r}/3 = 500/3$, and the bound drops from 11250 to $32500/3$: asking for three types instead of two spreads the bonus over more activations, so each type carries less of it and the dual prices fall. The heuristic starts from the base plan, tries to switch on a third type at its minimum lot and with these resources does not manage: it stays at 8700, and the optimum drops from 9700 to 9200.

	value	what it is
UB	$\frac{32500}{3}$	dual certificate built by hand
LB	8700	heuristic solution
$z(LP)$	$\frac{29875}{3}$	relaxation without the bounds
$z(LP^+)$	$\frac{28750}{3}$	relaxation with the bounds
$z(MILP)$	9200	optimum of the MILP

Mixed problems

Class: BIP / MILP

Script: one per problem, `python/fam10_1_prizes.py` ...

[Online page](#)

Why a chapter of mixed problems

The three families of the previous chapters have a recognisable structure: one assigns, one locates, one plans. The nine problems in this chapter do not have one, and none should be forced on them: each puts together different pieces, and the modelling work consists precisely in recognising which.

- **Selection with alternative modes** (problems 10.1 and 10.2): a subset is chosen, but each object has more than one way of being chosen, and the ways exclude one another.
- **Counts with a minimum lot** (problem 10.3): the variables are not binary but integer quantities, and a quantity may be zero or else above a threshold.
- **Covering with containers** (problems 10.4 and 10.5): a requirement must be covered by buying packs of fixed composition, and the excess is either paid for or wasted.
- **Splitting and balancing** (problems 10.6–10.9): a set must be divided among several containers and the quality is measured by how much the containers resemble one another.

Four of these problems share a trait that did not appear in the three families: the **LP relaxation is weak**, and in two cases it is exactly zero. The reason is always the same: a fractional solution can split every object in half and put one half in each container, levelling everything. The useful dual bound must then be looked for elsewhere:

- in **parity**, when a count cannot but be even;
- in the **number of containers** needed, read off the capacities;
- in the **dominance of one class** of objects over the others.

These are *combinatorial* arguments: they come from integrality, and the dual of the relaxation cannot see them.

For each problem, as in the families: description, variables, symbolic model, model on the instance, a feasible solution with its primal bound, the linear relaxation with its dual, a dual solution with its dual bound, the optimal solution and the final comparison.

10.1 Prizes obtainable in two ways

Script: `python/fam10_1_prizes.py`

[Notebook in Colab](#)

Problem 10.1 — prizes with two modes

A loyalty programme offers $s \in \mathbb{Z}_{\geq 1}$ prizes and a customer has $p \in \mathbb{Q}_{>0}$ points. Every prize $i \in \{1, 2, \dots, s\}$ can be obtained in two alternative ways: with points only, spending $a_i \in \mathbb{Q}_{>0}$ of them; or spending only $b_i \in \mathbb{Q}_{>0}$ points (with $b_i < a_i$) and adding a cash contribution of $c_i \in \mathbb{Q}_{\geq 0}$ euros. Every prize can be obtained at most once, and in one mode only. Every prize carries a preference value $d_i \in \mathbb{Q}_{>0}$, and the customer wants an overall preference of at least $\ell \in \mathbb{Q}_{>0}$. The customer wants to minimise the money spent.

The problem in words. We *decide* which prizes to take and, for each of them, in which of the two modes. *The objective:* minimum cash contribution. *The constraints:* the points are not enough for everything; the overall preference must reach the threshold; and the same prize is not taken twice.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of prizes, $i \in \{1, 2, \dots, s\}$
a_i	$\in \mathbb{Q}_{>0}$	points required by the points-only mode
b_i	$\in \mathbb{Q}_{>0}$	points required by the mode with contribution, $b_i < a_i$
c_i	$\in \mathbb{Q}_{\geq 0}$	contribution in euros of the second mode
d_i	$\in \mathbb{Q}_{>0}$	preference value of prize i
p	$\in \mathbb{Q}_{>0}$	points available
ℓ	$\in \mathbb{Q}_{>0}$	minimum preference required

Decision variables

Two families of binaries are needed, one per mode:

$$x_i = \begin{cases} 1 & \text{if prize } i \text{ is taken with points only,} \\ 0 & \text{otherwise,} \end{cases}$$

$$y_i = \begin{cases} 1 & \text{if prize } i \text{ is taken with points and contribution,} \\ 0 & \text{otherwise,} \end{cases}$$

for every $i \in \{1, 2, \dots, s\}$: $2s$ binary variables in total.

Model 10.1 — prizes with two modes

$$\min \sum_{i=1}^s c_i y_i \quad (10.1a)$$

$$\text{subject to } x_i + y_i \leq 1, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.1b)$$

$$\sum_{i=1}^s (a_i x_i + b_i y_i) \leq p, \quad (10.1c)$$

$$\sum_{i=1}^s d_i (x_i + y_i) \geq \ell, \quad (10.1d)$$

$$x_i, y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.1e)$$

Description. The objective (10.1a) counts money only: the points-only mode does not appear, because it costs no euros. Constraints (10.1b) are the mutual exclusion (s linear constraints); constraint (10.1c) is the points budget (a single one); constraint (10.1d) is the preference threshold (a single one).

What mutual exclusion does and does not say

The constraint $x_i + y_i \leq 1$ allows *three* configurations: $(0, 0)$, $(1, 0)$ and $(0, 1)$. It only forbids $(1, 1)$. In particular:

- it does not say "prize i must be taken": the configuration $(0, 0)$ is legitimate and means that the prize is given up;
- it does not say "if I do not take it with points then I take it with a contribution": the converse

$$x_i = 0 \implies y_i = 1$$

is false, and the counterexample is precisely $(0, 0)$;

- if one wanted to force every prize to be taken, the constraint would have to be written as an equality, $x_i + y_i = 1$: that is a set *partitioning* instead of a set *packing*, and the problem changes.

The quantity $x_i + y_i$ is therefore the indicator "prize i has been taken, one way or the other", and it is exactly that sum which appears in the preference constraint.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```
1 m = gp.Model("prizes")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(c[i] * y[i] for i in range(s)), GRB.MINIMIZE)
5 m.addConstrs((x[i] + y[i] <= 1 for i in range(s)), name="one_mode")
```

```

6 m.addConstr(gp.quicksum(a[i] * x[i] + b[i] * y[i] for i in range(s)) <=
    p, name="points")
7 m.addConstr(gp.quicksum(d[i] * (x[i] + y[i]) for i in range(s)) >= ell,
    name="preference")

```

The instance

$s = 5$ prizes, $p = 20$ points, $\ell = 16$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
a_i	8	6	10	5	7
b_i	4	3	6	2	4
c_i	10	8	15	5	9
d_i	5	4	7	3	6

Constructive heuristic: the primal bound

The prizes are scanned by decreasing preference. Each one is taken with points only if they suffice, otherwise with the contribution if the reduced points suffice, otherwise it is skipped; one stops as soon as the required preference is reached.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the order by preference is 3, 5, 1, 2, 4.

- Prize 3 (preference 7): the points alone are enough ($10 \leq 20$), it is taken; 10 points are left, preference 7.
- Prize 5 (preference 6): the points alone are enough ($7 \leq 10$), it is taken; 3 points are left, preference 13.
- Prize 1 (preference 5): the points are not enough for mode a ($8 > 3$) nor for mode b ($4 > 3$): it is skipped.
- Prize 2 (preference 4): the points are not enough for mode a ($6 > 3$) but they are for mode b ($3 \leq 3$): it is taken with a contribution of 8 euros; the preference reaches $17 \geq 16$ and we stop.

The total contribution is

$$z(MILP) \leq UB = 8.$$

LP relaxation and dual: the dual bound

We associate $\sigma_i \geq 0$ with constraints (10.1b), $\pi \geq 0$ with the points constraint (10.1c) and $\rho \geq 0$ with the preference threshold (10.1d). The primal is a minimisation, so the $\leq$ constraints give duals of negative sign: we write $-\sigma_i$ and $-\pi$ with $\sigma_i, \pi \geq 0$. The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.1

$$\max \quad -\sum_{i=1}^s \sigma_i - p\pi + \ell\rho \quad (10.2a)$$

$$\text{subject to} \quad -\sigma_i - a_i\pi + d_i\rho \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2b)$$

$$-\sigma_i - b_i\pi + d_i\rho \leq c_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2c)$$

$$\sigma_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.2d)$$

$$\pi \geq 0, \quad (10.2e)$$

$$\rho \geq 0. \quad (10.2f)$$

Description. The variable π is the price of one point, ρ the value of one unit of preference and σ_i the price of the mutual exclusion of prize i . The objective (10.2a) collects the threshold ℓ priced at ρ and pays the point budget p priced at π together with the multipliers σ_i . Constraints (10.2b) are the columns of the x_i : taking prize i with points only yields $d_i\rho$ of preference, consumes $a_i\pi$ of points and σ_i of mutual exclusion, and the balance cannot exceed the money cost of that option, which is zero. Constraints (10.2c) say the same for the second option, which consumes only b_i points: there the balance may reach c_i , the contribution that option charges.

A hand-built dual solution. Here the recipe has *two* free parameters, not one: the price π of a point and the price ρ of one unit of preference. Once these are fixed, the multipliers of the mutual exclusion follow from constraint (10.2b) by taking the smallest feasible value,

$$\bar{\sigma}_i = \max(0, d_i\rho - a_i\pi),$$

and only constraints (10.2c) on the second mode remain to be checked. The function

$$V(\pi, \rho) = -\sum_{i=1}^s \bar{\sigma}_i(\pi, \rho) - p\pi + \ell\rho$$

is concave and piecewise linear: it can be explored on a grid. On the instance the maximum is at

$$\bar{\pi} = 2, \quad \bar{\rho} = 3, \quad \bar{\sigma} = (0, 0, 1, 0, 4),$$

with value

$$LB = -(0 + 0 + 1 + 0 + 4) - 20 \cdot 2 + 16 \cdot 3 = -5 - 40 + 48 = 3.$$

This solution is *optimal* for the relaxation without the bounds: indeed $z(LP) = z(LP^+) = 3$.

Here $LB = 3$ and $z(MILP) = 5$: the gap between the dual bound and the integer optimum is 40%, and the certified gap between heuristic and dual is 100%. Nothing is wrong: the LP relaxation may take "half a prize" at half preference, and that freedom is worth a lot. It is the most extreme case in the course, and it is a reminder that a valid bound is not automatically a useful bound.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$
points only x_i	0	0	1	0	1
with contribution y_i	0	0	0	1	0

Prizes 3 and 5 are taken with points only ($10 + 7 = 17$) and prize 4 with the contribution (2 points and 5 euros): 19 points are used out of 20, the preference is $7 + 6 + 3 = 16$, exactly the threshold, and the contribution is

$$z(MILP) = 5.$$

The chain of bounds is

$$3 = LB = z(LP) \leq z(MILP) = 5 \leq UB = 8,$$

with a heuristic gap of 60%. The heuristic goes wrong because it only looks at the preference: it takes prize 5 with points (correctly), but then it is left with 3 points and has to buy prize 2 for 8 euros, whereas the optimum saves the points for prize 4, which only costs 5 euros of contribution.

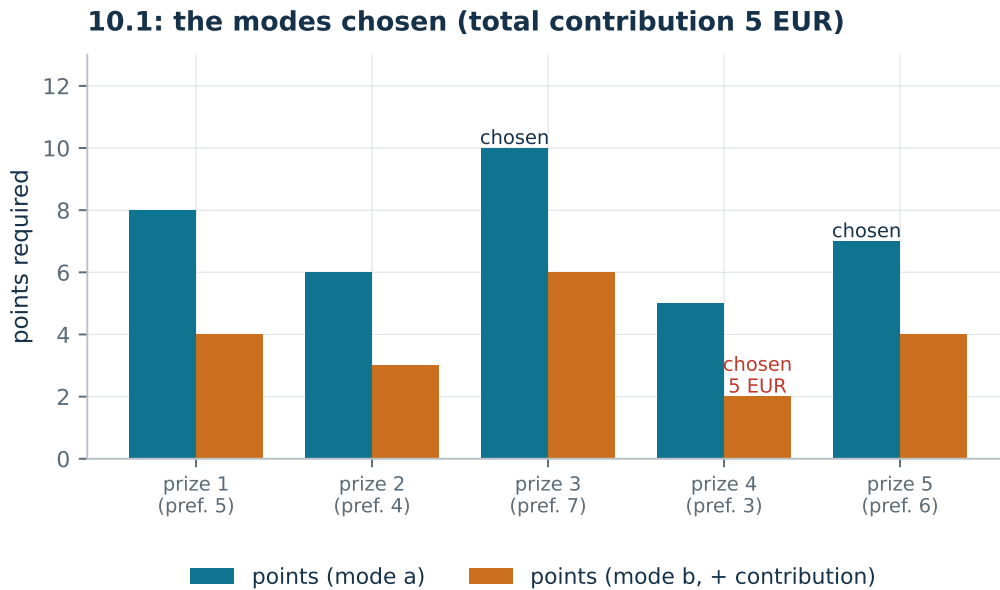


Figure 10.1: For every prize, the points required by the two modes; the labels mark which mode is chosen at the optimum and the contribution paid.

Additional considerations

- The valid inequalities $x_i \leq 1$ and $y_i \leq 1$ are implied by constraints (10.1b): indeed $z(LP) = z(LP^+)$.
- If for some prize we had $c_i = 0$, the second mode would dominate the first one (fewer points, same cost) and the variable x_i could be removed. It is a check on the data that is worth doing.

- The model extends effortlessly to k modes: k families of binaries and the constraint $\sum_{m=1}^k x_i^{(m)} \leq 1$. The structure stays a set packing by rows.

Additional modelling questions

Modelling question 10.1 — alternative prizes

Prizes 3 and 5 come from the same supplier and are alternatives: at most one of them may be taken, in either mode. How does the model change? What is the new optimum?

Modelling question 10.2 — at least four prizes

Besides the preference threshold, the customer wants at least four different prizes. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	8	heuristic solution (UB)
LB	3	certified bound (LB)
$z(MILP)$	5	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 60.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

The sandwich on variant 1b

The constraint $\sum_i (x_i + y_i) \geq 4$ adds to the dual a $\kappa \geq 0$ with right-hand side 4: it enters the objective and appears in *both* columns of every prize, because a prize counts however it was taken. The recipe of the base problem extends by searching κ too on the grid of quarters, and gives $\pi = \varrho = 3/2$, $\kappa = 9/2$, that is $LB = 6$. The heuristic enumerates the $3^5 = 243$ possible states — each prize not taken, on points or with a contribution — and finds 13, which is the optimum: with five objects enumeration is the most honest rule.

	value	what it is
UB	13	heuristic solution
LB	6	dual certificate built by hand
$z(LP)$	$\frac{25}{2}$	relaxation without the bounds
$z(LP^+)$	$\frac{25}{2}$	relaxation with the bounds
$z(MILP)$	13	optimum of the MILP

10.2 Combinatorial auction

Script: `python/fam10_2_auction.py`

[Notebook in Colab](#)

Problem 10.2 — combinatorial auction

An auctioneer has a set $S = \{1, 2, \dots, n\}$ of $n \in \mathbb{Z}_{\geq 1}$ items to sell and has received $r \in \mathbb{Z}_{\geq 1}$ bids. For every bid $j \in \{1, 2, \dots, r\}$, the set $B_j \subseteq S$ is the subset of items requested and $p_j \in \mathbb{Q}_{>0}$ the profit in euros if the bid is accepted. Each item can be sold at most once, and a bid can be accepted only if all its items are available. The auctioneer wants to select a set of bids of maximum total profit.

The problem in words. We *decide* which bids to accept. *The objective:* maximum profit. *The constraint:* two accepted bids cannot ask for the same item. A bid is all-or-nothing: one does not accept part of it.

Data (input of the model)

Symbol	Type	Meaning
n	$\in \mathbb{Z}_{\geq 1}$	number of items, $i \in S = \{1, 2, \dots, n\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of bids, $j \in \{1, 2, \dots, r\}$
B_j	$\subseteq S$	items requested by bid j
p_j	$\in \mathbb{Q}_{>0}$	profit of bid j

Decision variables

A single family of r binary variables:

$$x_j = \begin{cases} 1 & \text{if bid } j \text{ is accepted,} \\ 0 & \text{otherwise,} \end{cases} \quad \forall j \in \{1, 2, \dots, r\}.$$

Model 10.2 — combinatorial auction

$$\max \sum_{j=1}^r p_j x_j \quad (10.3a)$$

$$\text{subject to } \sum_{j: i \in B_j} x_j \leq 1, \quad \forall i \in S, \quad (10.3b)$$

$$x_j \in \{0, 1\}, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.3c)$$

Description. It is a pure *set packing*: one constraint per item (n linear constraints) and one variable per bid. Constraint (10.3b) says that item i is sold at most once.

By rows, not by pairs

An alternative, and worse, way of writing the same constraint is to list all the *pairs* of conflicting bids: $x_j + x_{j'} \leq 1$ for every $j \neq j'$ with $B_j \cap B_{j'} \neq \emptyset$. On the instance that would be nine constraints instead of four, and on real instances the number of pairs grows like r^2 while the constraints per item stay n . The row form is also tighter: if three bids all ask for item i , the row constraint says $x_1 + x_2 + x_3 \leq 1$, whereas the three pairwise inequalities allow $x_1 = x_2 = x_3 = 1/2$. It is the same comparison between aggregated and disaggregated form already seen in section 3.1, but here the direction of the advantage is

reversed.

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("auction")
2 x = m.addVars(r, vtype=GRB.BINARY, name="x")
3 m.setObjective(gp.quicksum(p[j] * x[j] for j in range(r)), GRB.MAXIMIZE
4 )
5 m.addConstrs((gp.quicksum(x[j] for j in range(r) if i in B[j]) <= 1
6                 for i in range(n)), name="item")

```

The instance

$n = 4$ items, $r = 6$ bids:

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$	$j = 6$
B_j	{1}	{2}	{3, 4}	{1, 3}	{2, 4}	{1, 3, 4}
p_j	6	3	12	12	10	16

The model of the instance is

$$\max 6x_1 + 3x_2 + 12x_3 + 12x_4 + 10x_5 + 16x_6$$

subject to

$$\begin{aligned}
 x_1 + x_4 + x_6 &\leq 1 && \text{(item 1)} \\
 x_2 + x_5 &\leq 1 && \text{(item 2)} \\
 x_3 + x_4 + x_6 &\leq 1 && \text{(item 3)} \\
 x_3 + x_5 + x_6 &\leq 1 && \text{(item 4)}
 \end{aligned}$$

with $x_j \in \{0, 1\}$.

Constructive heuristic: the primal bound

The problem is a maximisation, so the heuristic gives the *primal* bound, which sits below the optimum. The bids are sorted by decreasing profit *per item* and those whose items are still free are accepted.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

The running time is $O(r \log r + r n)$. On the instance the ratios are 6, 3, 6, 6, 5 and $16/3 \approx 5.33$, so the order is 1, 3, 4, 6, 5, 2:

- bid 1 {1}: accepted (profit 6);
- bid 3 {3, 4}: accepted (profit 12);
- bid 4 {1, 3}: rejected, items 1 and 3 are already sold;
- bid 6 {1, 3, 4}: rejected;

- bid 5 {2, 4}: rejected, item 4 is already sold;
- bid 2 {2}: accepted (profit 3).

Bids 1, 2, 3 are accepted:

$$z(MILP) \geq LB = 21.$$

LP relaxation and dual: the dual bound

We associate a non-negative dual variable λ_i with each constraint (10.3b). The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.2

$$\min \sum_{i \in S} \lambda_i \quad (10.4a)$$

$$\text{subject to } \sum_{i \in B_j} \lambda_i \geq p_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.4b)$$

$$\lambda_i \geq 0, \quad \forall i \in S. \quad (10.4c)$$

Description. The variable λ_i is the price the auctioneer puts on item i . The objective (10.4a) is the total value of the lots at those prices. Constraints (10.4b) are the columns of the x_j , one per bid: the items bid j asks for, priced at λ_i , must be worth at least what the bid pays, namely p_j . In short, no bid may turn out to be a bargain.

A hand-built dual solution. For example one can proceed as follows: spread the profit of every bid over its items and take, for every item, the maximum over the bids that ask for it,

$$\bar{\lambda}_i = \max_{j: i \in B_j} \frac{p_j}{|B_j|}.$$

Feasibility is immediate: for every bid j ,

$$\sum_{i \in B_j} \bar{\lambda}_i \geq \sum_{i \in B_j} \frac{p_j}{|B_j|} = |B_j| \cdot \frac{p_j}{|B_j|} = p_j.$$

On the instance:

$$\bar{\lambda}_1 = \max\left(\frac{6}{1}, \frac{12}{2}, \frac{16}{3}\right) = 6, \quad \bar{\lambda}_2 = \max\left(\frac{3}{1}, \frac{10}{2}\right) = 5,$$

$$\bar{\lambda}_3 = \max\left(\frac{12}{2}, \frac{12}{2}, \frac{16}{3}\right) = 6, \quad \bar{\lambda}_4 = \max\left(\frac{12}{2}, \frac{10}{2}, \frac{16}{3}\right) = 6,$$

and

$$z(MILP) \leq UB = 6 + 5 + 6 + 6 = 23.$$

The "obvious" recipe $\bar{\lambda}_i = \max_{j: i \in B_j} p_j$ (without dividing by $|B_j|$) is feasible as well, but gives $16 + 10 + 16 + 16 = 58$: more than twice as much. Dividing by the cardinality of the requested set is what makes the bound useful. It is worth trying several recipes and keeping the best: they are all valid, not all informative.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. At the optimum bids 4 ($\{1, 3\}$, profit 12) and 5 ($\{2, 4\}$, profit 10) are accepted: all four items are sold and

$$z(MILP) = 22.$$

The chain of bounds is

$$21 = LB \leq z(MILP) = 22 = z(LP^+) = z(LP) \leq UB = 23,$$

with a heuristic gap of 4.5%. The auctioneer sells everything, but not because a constraint forces it: the constraints are $\leq$, not $=$. With other bids the optimal solution could leave items on the shelf.

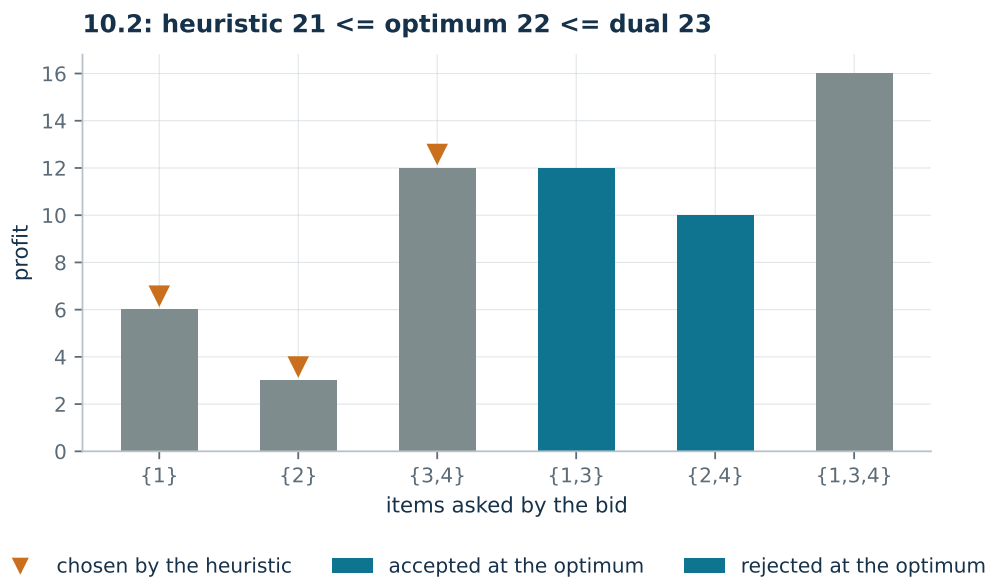


Figure 10.2: The six bids with their profit: in teal those accepted at the optimum, in grey those rejected; the triangle marks those chosen by the heuristic.

Additional considerations

- The valid inequalities $x_j \leq 1$ are implied by constraints (10.3b) whenever $B_j \neq \emptyset$: indeed $z(LP) = z(LP^+) = 22$.
- On this instance one also has $z(LP) = z(MILP)$: the relaxation lands on an integer vertex. It is a lucky case, not a property of set packing. The minimal counterexample is the *triangle*: three items and three bids asking for two of them each, all with profit 1. There $z(LP) = 3/2$ (with $x = 1/2$ on all three) against $z(MILP) = 1$.
- Set packing is NP-hard in general, but it becomes easy when the incidence matrix is *perfect* or *balanced*. The triangle is precisely the smallest graph that is not perfect in this sense, and it is the reason why *clique inequalities* are the classic cuts for this family.

Additional modelling questions

Modelling question 10.3 — bids from the same bidder

Bids 4 and 5 come from the same bidder, who may win at most one of them. How does the model change? What is the new optimum?

Modelling question 10.4 — limited deliveries

In this round the auctioneer can deliver at most two items in total. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	21	heuristic solution (LB)
UB	23	certified bound (UB)
$z(MILP)$	22	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 4.5\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 9.1\%.$$

The sandwich on variant 2b

The cap on the items delivered, $\sum_j |B_j| x_j \leq 2$, adds to the dual a $\mu \geq 0$ with right-hand side 2, and in the column of every bid it appears multiplied by how many items that bid asks for. With μ fixed, every bid spreads over its items net of μ and λ_i is the largest among the bids asking for item i : trying μ among the per-item profits one reaches $\mu = 6$, $\lambda = 0$ and $UB = 12$. The heuristic accepts bid 3 alone and is worth 12 as well: the sandwich closes, and the optimum is known without solving the MILP.

	value	what it is
UB	12	dual certificate built by hand
LB	12	heuristic solution
$z(LP)$	12	relaxation without the bounds
$z(LP^+)$	12	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

10.3 Diet with a count of the foods and a minimum lot

Script: `python/fam10_3_diet.py`

[Notebook in Colab](#)

Problem 10.3 — diet with variety

A nutritionist must compose a monthly diet choosing among $s \in \mathbb{Z}_{\geq 1}$ foods and controlling $r \in \mathbb{Z}_{\geq 1}$ nutrients. For every food $i \in \{1, 2, \dots, s\}$, the value $w_i \in \mathbb{Q}_{>0}$ is the cost of one kilo; for every food i and every nutrient $j \in \{1, 2, \dots, r\}$, the value $g_{ij} \in \mathbb{Q}_{\geq 0}$ is the amount of nutrient j contained in one kilo of food i . For every nutrient j , the monthly intake must lie between $a_j \in \mathbb{Q}_{>0}$ and $b_j \in \mathbb{Q}_{>0}$. If a food enters the diet, at least $c_i \in \mathbb{Q}_{>0}$ kilos of it must be consumed and at most $d_i \in \mathbb{Q}_{>0}$. The diet must include at least $t \in \mathbb{Z}_{\geq 1}$ different foods. The goal is the diet of minimum cost.

The problem in words. We *decide* which foods to use and in what quantity. *The objective:* minimum cost. *The constraints:* every nutrient within its window; every chosen food in a quantity between the minimum lot and the cap; at least t different foods.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of foods, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of nutrients, $j \in \{1, 2, \dots, r\}$
w_i	$\in \mathbb{Q}_{>0}$	cost of one kilo of food i
g_{ij}	$\in \mathbb{Q}_{\geq 0}$	grams of nutrient j per kilo of food i
a_j, b_j	$\in \mathbb{Q}_{>0}$	monthly minimum and maximum of nutrient j
c_i, d_i	$\in \mathbb{Q}_{>0}$	minimum and maximum quantity of food i , if chosen
t	$\in \mathbb{Z}_{\geq 1}$	minimum number of different foods

Decision variables

s non-negative continuous variables and s binaries:

x_i = kilos of food i consumed in the month,

$$y_i = \begin{cases} 1 & \text{if food } i \text{ enters the diet,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 10.3 — diet with variety

$$\min \sum_{i=1}^s w_i x_i \quad (10.5a)$$

$$\text{subject to } \sum_{i=1}^s g_{ij} x_i \geq a_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5b)$$

$$\sum_{i=1}^s g_{ij} x_i \leq b_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.5c)$$

$$x_i - c_i y_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5d)$$

$$x_i - d_i y_i \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5e)$$

$$\sum_{i=1}^s y_i \geq t, \quad (10.5f)$$

$$x_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.5g)$$

$$y_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}. \quad (10.5h)$$

Description. The objective (10.5a) is the total cost. Constraints (10.5b), one per nutrient, impose the minimum requirement; constraints (10.5c), again one per nutrient, impose the cap. Constraints (10.5d) and (10.5e), one per food each, make x_i a semicontinuous variable: either it is zero, or it lies between the minimum lot c_i and the maximum d_i . Constraint (10.5f), a single one, requires at least t different foods to be bought.

Why the count needs the minimum lot

The variety constraint (10.5f) counts the indicators y_i , not the quantities. If only the activation constraint (10.5e) were present, the indicator could be 1 with $x_i = 0$: foods would be "switched on" without a single gram being consumed, and the variety constraint would be satisfied by empty indicators. It is the one-way link of section 3.10, and here it produces a model that does not describe the problem.

The minimum lot constraint (10.5d) closes the loop: with $y_i = 1$ one gets $x_i \geq c_i > 0$, so every switched-on indicator corresponds to a food that is really consumed. On the instance, setting $c_i = 0$ the optimum drops from 48/5 to 46/5 and the solution "switches on" four foods, two of which with zero quantity: the count no longer says anything.

The model in gurobipy

```

1 m = gp.Model("diet")
2 x = m.addVars(s, name="x")
3 y = m.addVars(s, vtype=GRB.BINARY, name="y")
4 m.setObjective(gp.quicksum(w[i] * x[i] for i in range(s)), GRB.MINIMIZE
5 )
6 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) >= a[j] for
7 j in range(r)),
8 name="minimum")

```

```

7 m.addConstrs((gp.quicksum(g[i][j] * x[i] for i in range(s)) <= b[j] for
  j in range(r)),
8              name="maximum")
9 m.addConstrs((x[i] - c[i] * y[i] >= 0 for i in range(s)), name="
  minimum_lot")
10 m.addConstrs((x[i] - d[i] * y[i] <= 0 for i in range(s)), name="
  activate")
11 m.addConstr(gp.quicksum(y[i] for i in range(s)) >= t, name="variety")

```

The instance

$s = 4$ foods, $r = 2$ nutrients, $c_i = 1$, $d_i = 8$, $t = 3$:

	milk	rice	bread	potatoes		iron	calcium
w_i (euros/kg)	2	3	1	4	a_j	60	40
iron (g/kg)	10	20	5	25	b_j	200	150
calcium (g/kg)	5	10	15	5			

Constructive heuristic: the primal bound

The t cheapest foods are switched on at their minimum lot, then the residual requirement of each nutrient is covered by adding the already active food with the lowest cost per gram.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the three cheapest foods are bread (1), milk (2) and rice (3): they are switched on at their minimum lot, that is 1 kg each. With these quantities one has $5 + 10 + 20 = 35$ grams of iron (60 are needed) and $15 + 5 + 10 = 30$ of calcium (40 are needed).

- **Iron:** 25 grams are missing. Among the active foods the lowest cost per gram of iron is that of rice, $3/20 = 0.15$; $25/20 = 1.25$ extra kilos are needed, and the rice reaches 2.25 kg.
- **Calcium:** after that addition one has $30 + 12.5 = 42.5$ grams, already above the minimum of 40: nothing else is needed.

The final diet is milk 1 kg, rice 2.25 kg, bread 1 kg, at a cost of

$$z(MILP) \leq UB = \frac{39}{4} = 9.75.$$

LP relaxation and dual: the dual bound

We associate $\alpha_j \geq 0$ with the minima (10.5b), $\beta_j \geq 0$ with the maxima (10.5c), $\lambda_i \geq 0$ with the minimum lots (10.5d), $\mu_i \geq 0$ with the caps (10.5e) and $\tau \geq 0$ with the variety (10.5f). The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.3

$$\max \sum_{j=1}^r a_j \alpha_j - \sum_{j=1}^r b_j \beta_j + t \tau \quad (10.6a)$$

$$\text{subject to } \sum_{j=1}^r g_{ij} (\alpha_j - \beta_j) + \lambda_i - \mu_i \leq w_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6b)$$

$$-c_i \lambda_i + d_i \mu_i + \tau \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6c)$$

$$\alpha_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6d)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.6e)$$

$$\lambda_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6f)$$

$$\mu_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.6g)$$

$$\tau \geq 0. \quad (10.6h)$$

Description. The variable α_j is the price of one unit of nutrient j when it serves to reach the minimum requirement, β_j what is paid not to exceed the cap, λ_i and μ_i the prices of the two semicontinuity constraints of food i , and τ the price of variety. The objective (10.6a) collects the requirements priced at α , pays the caps priced at β and collects the threshold t priced at τ . Constraints (10.6b) are the columns of the x_i : the nutritional content of one unit of food i , priced at $\alpha_j - \beta_j$ and corrected by the two semicontinuity constraints, cannot exceed the price w_i of that food. Constraints (10.6c) are the columns of the y_i : switching on food i grants c_i units of minimum lot at price λ_i , forces at most d_i at price μ_i , and must cover the price τ of variety.

A hand-built dual solution. Set $\beta = \mu = \tau = 0$: the maxima, the caps and the variety are not priced. Constraints (10.6c) become $-c_i \lambda_i \leq 0$, satisfied with $\lambda = 0$. What is left is $\sum_{j=1}^r g_{ij} \alpha_j \leq w_i$ for every food. The simplest recipe prices a *single* nutrient, at the lowest cost per gram among the foods:

$$\bar{\alpha}_j = \min_{i: g_{ij} > 0} \frac{w_i}{g_{ij}}, \quad \text{bound} = a_j \bar{\alpha}_j,$$

and one keeps the nutrient giving the larger bound. On the instance:

$$\text{iron: } \bar{\alpha}_1 = \min\left(\frac{2}{10}, \frac{3}{20}, \frac{1}{5}, \frac{4}{25}\right) = \frac{3}{20}, \quad 60 \cdot \frac{3}{20} = 9,$$

$$\text{calcium: } \bar{\alpha}_2 = \min\left(\frac{2}{5}, \frac{3}{10}, \frac{1}{15}, \frac{4}{5}\right) = \frac{1}{15}, \quad 40 \cdot \frac{1}{15} = \frac{8}{3}.$$

The better one is the iron:

$$z(MILP) \geq LB = 9.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal diet is rice 1.4 kg, bread 1.4 kg,

potatoes 1 kg: three different foods, as required, with 60 grams of iron (exactly the minimum) and 40 of calcium (again exactly the minimum). The cost is

$$z(MILP) = \frac{48}{5} = 9.6,$$

and the chain of bounds is

$$9 = LB \leq z(LP) = \frac{46}{5} \leq z(LP^+) = \frac{48}{5} = z(MILP) \leq UB = \frac{39}{4}.$$

The heuristic gap is a mere 1.6%: it is the tightest sandwich of the chapter. Note that here the relaxation with the bounds coincides with the integer optimum while the pure one is below: adding $y_i \leq 1$ does matter, because the caps $d_i = 8$ are much wider than the quantities in play, and without $y_i \leq 1$ the relaxation would use fractional indicators larger than one.

10.3: heuristic diet (39/4 EUR) and optimal one (48/5 EUR)

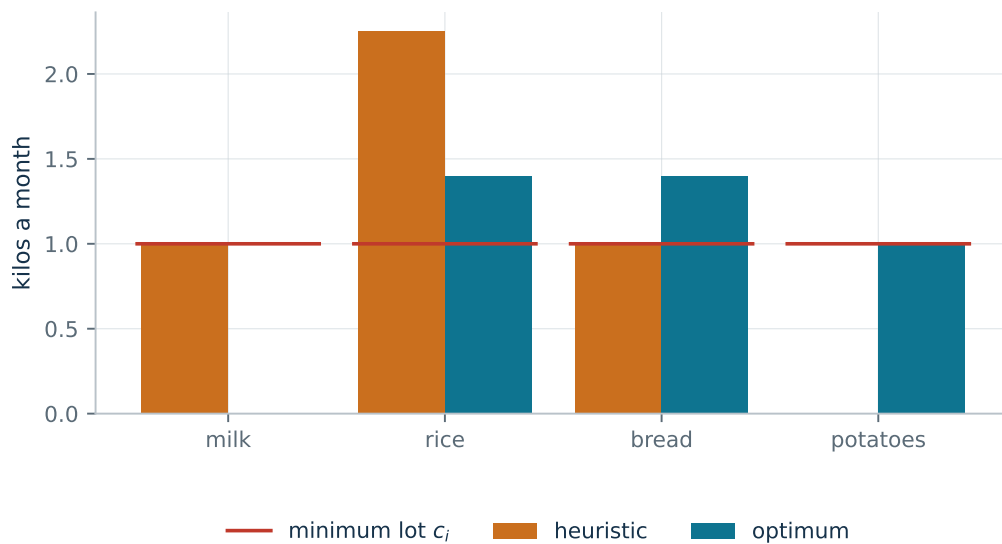


Figure 10.3: Quantity per food in the heuristic diet and in the optimal one; the red bar marks the minimum lot c_i .

Additional considerations

- The maximum constraints (10.5c) are not active at the optimum ($60 \leq 200$ and $40 \leq 150$): on this instance they could be removed without changing anything. They stay in the model because the problem requires them, and because on other instances they would bite.
- The classic form of the diet problem (Stigler, 1945) has neither minimum lots nor counts: it is a pure LP. It is exactly the two integer additions that turn it into a MILP, and that is why the problem belongs to this chapter.
- The variety constraint could also be written as a *covering* constraint: at least one food from each food group, that is $\sum_{i \in G_k} y_i \geq 1$ for every group k . It is a more informative formulation from the nutritional point of view, and no harder.

Additional modelling questions

Modelling question 10.5 — a higher minimum lot

The minimum lot goes up from 1 to 2 kilos for every chosen food. How does the model change? What is the new optimum?

Modelling question 10.6 — more variety

At least four different foods are wanted instead of three. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	$\frac{39}{4}$	heuristic solution (UB)
LB	9	certified bound (LB)
$z(MILP)$	$\frac{48}{5}$	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 1.6\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 7.8\%.$$

The sandwich on variant 3b

The variant changes a datum, $t = 4$, not the structure. The count of the foods, however, is not priced: τ enters the objective with its right-hand side, but the column of the y_i imposes $\tau \leq c_i \lambda_i - d_i \mu_i$, and with the recipe of the base problem — $\lambda = \mu = 0$ and a single positive α , on iron — it stays $\tau = 0$. The dual bound does not move, the optimum does: the heuristic switches on the fourth food at its minimum lot, completes with bread and reaches $31/3$, which is the optimum.

	value	what it is
UB	$\frac{31}{3}$	heuristic solution
LB	9	dual certificate built by hand
$z(LP)$	$\frac{28}{3}$	relaxation without the bounds
$z(LP^+)$	$\frac{31}{3}$	relaxation with the bounds
$z(MILP)$	$\frac{31}{3}$	optimum of the MILP

10.4 Christmas trees and boxes of lights

Script: `python/fam10_4_lights.py`

[Notebook in Colab](#)

Problem 10.4 — trees and lights

The city administration wants to decorate $q \in \mathbb{Z}_{\geq 1}$ trees for the holidays. Every tree can be decorated according to one of the $n \in \mathbb{Z}_{\geq 1}$ possible configurations, each characterised by lights of $m \in \mathbb{Z}_{\geq 1}$ different colours. For every configuration $c \in \{1, 2, \dots, n\}$ and every colour $l \in \{1, 2, \dots, m\}$, the value $u_{cl} \in \mathbb{Z}_{\geq 0}$ is the number of lights of colour l required by configuration c , and $i_c \in \mathbb{Q}_{>0}$ is the installation cost of a tree decorated according to it. All the lights must be bought on the market, where they are sold in boxes of $k \in \mathbb{Z}_{\geq 1}$ types: for every type $b \in \{1, 2, \dots, k\}$ and every colour l , the value $v_{bl} \in \mathbb{Z}_{\geq 0}$ is the number of lights of colour l contained in a box of type b , and $p_b \in \mathbb{Q}_{\geq 0}$ is its cost. To ensure a pleasant visual variety at least $f \in \mathbb{Z}_{\geq 1}$ different configurations must be used. The goal is to decorate the trees at minimum total cost.

The problem in words. We *decide* how many trees to decorate with each configuration and how many boxes to buy of each type. *The objective:* minimum total cost (installation plus boxes). *The constraints:* all the trees must be decorated; the lights bought must be enough for those required, colour by colour; at least f different configurations.

Decision variables

n integer variables, k integer variables and n binaries:

x_c = trees decorated with configuration c , y_b = boxes bought of type b ,

$$z_c = \begin{cases} 1 & \text{if configuration } c \text{ is used,} \\ 0 & \text{otherwise.} \end{cases}$$

Model 10.4 — trees and lights

$$\min \sum_{c=1}^n i_c x_c + \sum_{b=1}^k p_b y_b \quad (10.7a)$$

$$\text{subject to } \sum_{c=1}^n x_c = q, \quad (10.7b)$$

$$\sum_{b=1}^k v_{bl} y_b - \sum_{c=1}^n u_{cl} x_c \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.7c)$$

$$\sum_{c=1}^n z_c \geq f, \quad (10.7d)$$

$$x_c - z_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7e)$$

$$x_c \in \mathbb{Z}_{\geq 0}, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.7f)$$

$$y_b \in \mathbb{Z}_{\geq 0}, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.7g)$$

$$z_c \in \{0, 1\}, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.7h)$$

Description. The objective (10.7a) adds up the cost of the installations and that of the boxes bought. Constraint (10.7b), a single one, says that the decorated trees are exactly q . Constraints (10.7c), one per colour, say that the lights bought are enough to cover those used. Constraint (10.7d), a single one, imposes at least f different configurations. Constraints (10.7e), one per configuration, tie the indicator z_c to the actual use of the configuration.

The link between the two storeys. Constraint (10.7c) is the availability: for every colour, the lights bought must be at least those required. There is no big-M, because both sides are sums of real quantities. The count of the boxes comes from here: if the lights of colour l required are u_l and a box contains v_{bl} of them, buying only boxes of type b one would need $\lceil u_l/v_{bl} \rceil$ boxes. The model does not write the ceiling: it obtains it by making y_b integer.

The "configuration used" link

Constraint (10.7e) reads $z_c \leq x_c$. If $z_c = 1$ then $x_c \geq 1$: the configuration is really used on at least one tree. The other direction is not imposed, but it is not needed: the minimisation objective has no interest in raising z_c , and constraint (10.7d) forces it to do so exactly f times. Note that no big-M is needed here because x_c and z_c are directly comparable: x_c is a count, not a continuous quantity.

A frequent mistake is to write $x_c \leq M z_c$ thinking of activation: that would be the link in the *other* direction, and would impose that an unused configuration has $x_c = 0$ — true but useless, because it is already guaranteed by $\sum_{c=1}^n x_c = q$ and by $x_c \geq 0$.

The model in gurobipy

```

1 m = gp.Model("lights")
2 x = m.addVars(nc, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nb, vtype=GRB.INTEGER, name="y")
4 z = m.addVars(nc, vtype=GRB.BINARY, name="z")
5 m.setObjective(gp.quicksum(i[c] * x[c] for c in range(nc))
6                 + gp.quicksum(p[b] * y[b] for b in range(nb)), GRB.
7                 MINIMIZE)
8 m.addConstr(x.sum() == q, name="trees")
9 m.addConstrs((gp.quicksum(v[b][l] * y[b] for b in range(nb))
10               - gp.quicksum(u[c][l] * x[c] for c in range(nc)) >= 0
11               for l in range(nl)), name="lights")
12 m.addConstr(z.sum() >= f, name="variety")
13 m.addConstrs((x[c] - z[c] >= 0 for c in range(nc)), name="used")

```

The instance

$q = 20$ trees, $n = 3$ configurations, $m = 2$ colours, $k = 2$ box types, $f = 2$:

u_{cl}	colour 1	colour 2	i_c
$c = 1$	4	2	7
$c = 2$	2	3	6
$c = 3$	2	2	8

v_{bl}	colour 1	colour 2	p_b
$b = 1$	10	2	100
$b = 2$	15	4	200

The price of one light depends on the colour and on the box type: colour 1 at 10 in box 1 and at 40/3 in box 2; colour 2 at 50 in both. Colour 2 is much more expensive, and it is what drives the solution.

Constructive heuristic: the primal bound

Two phases. First the configurations: $q - f + 1$ trees with the one that is cheapest to install and one tree for each of the other $f - 1$, so that variety is met at the lowest installation cost. Then the boxes: while some light is missing, buy the box with the lowest price per missing light.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the cheapest configuration to install is number 2 (cost 6), the second one is number 1 (cost 7): 19 trees are decorated with the 2 and one with the 1. Then 42 lights of colour 1 and 59 of colour 2 are needed, and the constructive heuristic buys 30 boxes of type 1. The total cost is

$$z(MILP) \leq UB = 3121.$$

The heuristic picks the configuration that is cheapest to install, number 2, which is however the greediest for the expensive colour: the boxes pay the bill. It is the typical mistake of a constructive heuristic that looks at a single cost item.

LP relaxation and dual: the dual bound

We associate α free with constraint (10.7b) (an equality), $\beta_l \geq 0$ with constraints (10.7c), $\gamma \geq 0$ with (10.7d) and $\delta_c \geq 0$ with (10.7e). The dual is:

Dual of the LP relaxation of model 10.4

$$\max \quad q\alpha + f\gamma \quad (10.8a)$$

$$\text{subject to} \quad \alpha - \sum_{l=1}^m u_{cl} \beta_l + \delta_c \leq i_c, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8b)$$

$$\sum_{l=1}^m v_{bl} \beta_l \leq p_b, \quad \forall b \in \{1, 2, \dots, k\}, \quad (10.8c)$$

$$\gamma - \delta_c \leq 0, \quad \forall c \in \{1, 2, \dots, n\}, \quad (10.8d)$$

$$\alpha \geq 0, \quad (10.8e)$$

$$\beta_l \geq 0, \quad \forall l \in \{1, 2, \dots, m\}, \quad (10.8f)$$

$$\gamma \geq 0, \quad (10.8g)$$

$$\delta_c \geq 0, \quad \forall c \in \{1, 2, \dots, n\}. \quad (10.8h)$$

Description. The variable α is the value of one decorated tree, β_l the price of one light of colour l , γ the price of variety and δ_c that of the link between configuration c and its indicator. The objective (10.8a) prices the q trees at α and the variety threshold f at γ . Constraints (10.8b) are the columns of the x_c : using configuration c on a tree is worth α , consumes u_{cl} lights of each colour and releases δ_c , and the balance cannot exceed the installation cost i_c . Constraints (10.8c)

are the columns of the y_b : a box of type b supplies v_{bl} lights of each colour, and their total value cannot exceed the price p_b of the box. Constraints (10.8d) are the columns of the z_c : the price of variety cannot exceed that of the link which makes it claimable.

A hand-built dual solution. Set $\gamma = 0$ and $\delta_c = 0$: the variety is not priced and constraint (10.8d) is satisfied. What is left are constraints (10.8c) on the boxes, which cap β , and constraints (10.8b) on the configurations, which cap α .

One prices *a single* colour, at the highest price per light that no box can beat:

$$\bar{\beta}_l = \min_{b: v_{bl} > 0} \frac{p_b}{v_{bl}},$$

and then takes the largest feasible α ,

$$\bar{\alpha} = \min_c (i_c + u_{cl} \bar{\beta}_l),$$

that is: every tree costs at least the installation of the most convenient configuration, lights included. On the instance the better colour is number 2, where both box types give the same price per light, $\bar{\beta}_2 = 50$:

$$i_1 + 2 \cdot 50 = 107, \quad i_2 + 3 \cdot 50 = 156, \quad i_3 + 2 \cdot 50 = 108,$$

hence $\bar{\alpha} = 107$ and

$$z(MILP) \geq LB = 20 \cdot 107 = 2140.$$

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. 19 trees are decorated with configuration 1 and one with configuration 3, and 20 boxes of type 1 are bought:

$$z(MILP) = 19 \cdot 7 + 8 + 20 \cdot 100 = 2141.$$

The lights needed are 78 of colour 1 (200 are bought: many are left over) and 40 of colour 2 (exactly 40 are bought). The chain of bounds is

$$2140 = LB = z(LP) \leq z(LP^+) = 2141 = z(MILP) \leq UB = 3121.$$

The dual bound is off by *one* unit out of 2141, and the strengthened relaxation is exact. The heuristic gap is 45.8%.

Additional considerations

- Colour 1 is left over in abundance: the boxes are bought for colour 2, and colour 1 comes "for free". That is why the dual bound pricing colour 2 alone is almost exact.
- Constraint (10.7b) is an equality: all the trees must be decorated. If it were $\leq q$ the problem would become trivial ($x = 0$ and zero cost); if it were $\geq q$ nothing would change, because decorating more trees costs more.
- The structure "two storeys tied by an availability" appears identically in *cutting stock* problems: below the pieces to be cut, above the bars to be bought.

Additional modelling questions

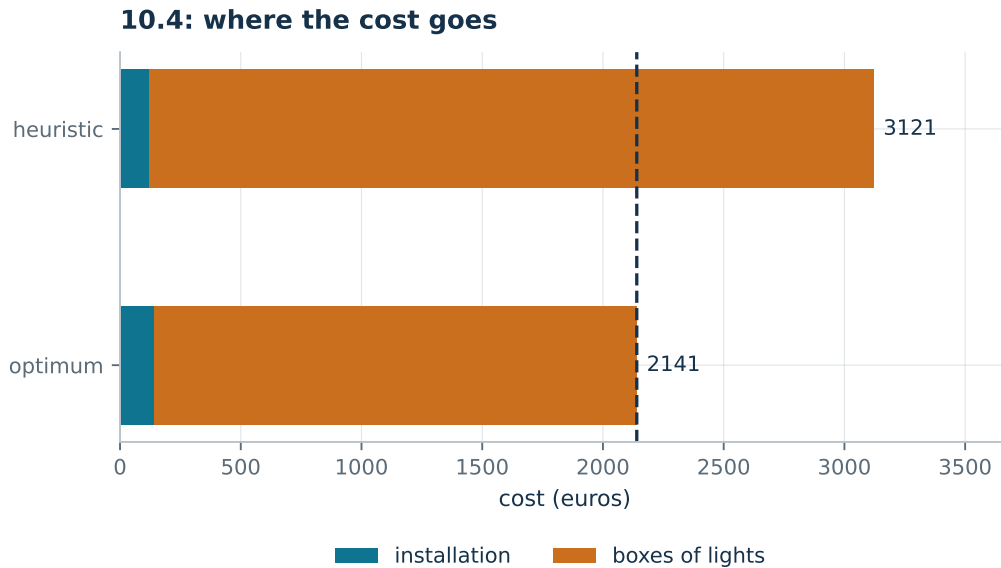


Figure 10.4: Where the cost goes in the two solutions: installation and boxes, with the dual bound dashed.

Modelling question 10.7 — all the configurations

All three configurations are required to appear. How does the model change? What is the new optimum?

Modelling question 10.8 — minimum lot per configuration

Every configuration used must decorate at least three trees (below that threshold it is not worth equipping the crew). How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	3121	heuristic solution (UB)
LB	2140	certified bound (LB)
$z(MILP)$	2141	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 45.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 45.8\%.$$

The sandwich on variant 4a

Asking for all three configurations changes the right-hand side of the variety constraint, not the structure. But that constraint is not priced: the recipe of the base problem keeps $\gamma = \delta = 0$ and puts the whole price on a single colour, so the bound stays 2140 while the optimum rises from 2141 to 2239. The heuristic is the same, and to respect the variety it installs the two missing configurations on one tree each: it costs 3023, and the certified gap stays wide. Here the MILP really is needed.

	value	what it is
UB	3023	heuristic solution
LB	2140	dual certificate built by hand
$z(LP)$	2140	relaxation without the bounds
$z(LP^+)$	2190	relaxation with the bounds
$z(MILP)$	2239	optimum of the MILP

10.5 Shipments in boxes

Script: `python/fam10_5_shipments.py`

[Notebook in Colab](#)

Problem 10.5 — shipments in boxes

A company needs to organise the shipments from its $n \in \mathbb{Z}_{\geq 1}$ production plants to its $m \in \mathbb{Z}_{\geq 1}$ customers. The company produces $k \in \mathbb{Z}_{\geq 1}$ product types. Products are shipped in boxes: every box travels directly from one plant to one customer and has maximum capacity $w \in \mathbb{Z}_{\geq 1}$, expressed as the maximum number of product units it can carry. For every product $p \in \{1, 2, \dots, k\}$ and every customer $c \in \{1, 2, \dots, m\}$, the value $d_{pc} \in \mathbb{Z}_{\geq 0}$ is the number of units ordered. For every product p and every plant $s \in \{1, 2, \dots, n\}$, the value $a_{ps} \in \mathbb{Z}_{\geq 0}$ is the number of units available. The company wants a shipment plan that minimises the total number of boxes used.

The problem in words. We *decide* how many units of each product to ship from each plant to each customer, and how many boxes are needed on each route. *The objective*: the minimum number of boxes. *The constraints*: every order met exactly, no plant beyond its stock, and enough boxes on every route for the units shipped.

For brevity write $P = \{1, 2, \dots, k\}$ for the set of products, $S = \{1, 2, \dots, n\}$ for the set of plants and $C = \{1, 2, \dots, m\}$ for the set of customers.

Decision variables

$k n m$ non-negative integer variables for the flow and $n m$ for the boxes:

$$x_{psc} = \text{units of product } p \text{ shipped from } s \text{ to } c, \quad y_{sc} = \text{boxes shipped from } s \text{ to } c.$$

Model 10.5 — shipments in boxes

$$\min \sum_{s=1}^n \sum_{c=1}^m y_{sc} \quad (10.9a)$$

$$\text{subject to } \sum_{s=1}^n x_{psc} = d_{pc}, \quad \forall p \in P, \forall c \in C, \quad (10.9b)$$

$$\sum_{c=1}^m x_{psc} \leq a_{ps}, \quad \forall p \in P, \forall s \in S, \quad (10.9c)$$

$$-\sum_{p=1}^k x_{psc} + w y_{sc} \geq 0, \quad \forall s \in S, \forall c \in C, \quad (10.9d)$$

$$x_{psc} \in \mathbb{Z}_{\geq 0}, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.9e)$$

$$y_{sc} \in \mathbb{Z}_{\geq 0}, \quad \forall s \in S, \forall c \in C. \quad (10.9f)$$

Description. The objective (10.9a) counts the boxes used on all the legs. Constraints (10.9b), one per product–customer pair, say that every order is met exactly. Constraints (10.9c), one per product–plant pair, do not allow shipping more than the plant has. Constraints (10.9d), one per leg, say that the boxes shipped from s to c are enough to hold the units loaded.

The link is a rounding up

Constraint (10.9d) reads

$$\sum_{p=1}^k x_{psc} \leq w y_{sc} \quad \Longleftrightarrow \quad y_{sc} \geq \frac{\sum_{p=1}^k x_{psc}}{w},$$

and since y_{sc} is a non-negative integer this is equivalent to

$$y_{sc} \geq \left\lceil \frac{\sum_{p=1}^k x_{psc}}{w} \right\rceil.$$

The ceiling never appears in the model: integrality produces it. The same constraint also imposes the implication

$$\sum_{p=1}^k x_{psc} > 0 \implies y_{sc} \geq 1,$$

that is: if even a single unit leaves a plant towards a customer, then at least one box must travel on that route. The contrapositive is easier to read: if $y_{sc} = 0$ then $\sum_{p=1}^k x_{psc} = 0$. The other direction — if nothing is shipped then no box — is not imposed by the constraint but follows from optimality, because every box costs 1 and the objective is a minimisation.

The model in gurobipy

```
1 m = gp.Model("shipments")
2 x = m.addVars(nk, nn, nm, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(nn, nm, vtype=GRB.INTEGER, name="y")
```

```

4 m.setObjective(y.sum(), GRB.MINIMIZE)
5 m.addConstrs((x.sum(p, "*", c) == d[p][c] for p in range(nk) for c in
    range(nm)),
6              name="demand")
7 m.addConstrs((x.sum(p, s, "*") <= a[p][s] for p in range(nk) for s in
    range(nn)),
8              name="availability")
9 m.addConstrs((w * y[s, c] - x.sum("*", s, c) >= 0 for s in range(nn)
    for c in range(nm)),
10             name="capacity")

```

The instance

$n = 2$ plants, $m = 2$ customers, $k = 2$ products, $w = 10$:

d_{pc}	$c = 1$	$c = 2$	a_{ps}	$s = 1$	$s = 2$
$p = 1$	5	0	$p = 1$	8	6
$p = 2$	2	4	$p = 2$	5	7

The units to ship are $5 + 0 + 2 + 4 = 11$ in total.

Constructive heuristic: the primal bound

Customer by customer: one tries to serve them from a single plant, the one that has everything they need; if no plant is enough, the order is split among several plants. At the end one counts $\lceil \cdot / w \rceil$ boxes per route.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance customer 1 asks for 5 units of product 1 and 2 of product 2: plant 1 has 8 and 5, so it is enough on its own. Customer 2 asks for 4 units of product 2: after the first shipment plant 1 has 3 left, plant 2 has 7, so the shipment leaves from there. There are two routes, with 7 and 4 units: one box each.

$$z(MILP) \leq UB = 2.$$

LP relaxation and dual: the dual bound

We associate α_{pc} free with the demand constraints (10.9b), $\beta_{ps} \leq 0$ with the availability ones (10.9c) and $\gamma_{sc} \geq 0$ with the link (10.9d). The dual is:

Dual of the LP relaxation of model 10.5

$$\max \sum_{p=1}^k \sum_{c=1}^m d_{pc} \alpha_{pc} + \sum_{p=1}^k \sum_{s=1}^n a_{ps} \beta_{ps} \quad (10.10a)$$

$$\text{subject to } \alpha_{pc} + \beta_{ps} - \gamma_{sc} \leq 0, \quad \forall p \in P, \forall s \in S, \forall c \in C, \quad (10.10b)$$

$$w \gamma_{sc} \leq 1, \quad \forall s \in S, \forall c \in C, \quad (10.10c)$$

$$\alpha_{pc} \geq 0, \quad \forall p \in P, \forall c \in C, \quad (10.10d)$$

$$\beta_{ps} \leq 0, \quad \forall p \in P, \forall s \in S, \quad (10.10e)$$

$$\gamma_{sc} \geq 0, \quad \forall s \in S, \forall c \in C. \quad (10.10f)$$

Description. The variable α_{pc} is the value of one unit of product p delivered to customer c , β_{ps} the (non-positive) price of the availability of product p at plant s , and γ_{sc} the price of one unit of space on the leg from s to c . The objective (10.10a) prices orders and availabilities at those values. Constraints (10.10b) are the columns of the x_{psc} : shipping one unit of product p from s to c is worth $\alpha_{pc} + \beta_{ps}$ and takes up one unit of space at price γ_{sc} ; the balance cannot be positive, because those variables do not appear in the objective. Constraints (10.10c) are the columns of the y_{sc} : one box offers w units of space, and their value cannot exceed 1, which is what the box costs.

A hand-built dual solution. Constraint (10.10c) caps $\gamma_{sc} \leq 1/w$: take the maximum, $\bar{\gamma}_{sc} = 1/w$. With $\bar{\beta} = 0$ constraint (10.10b) becomes $\alpha_{pc} \leq \bar{\gamma}_{sc} = 1/w$: take $\bar{\alpha}_{pc} = 1/w$. The value is

$$LB = \frac{1}{w} \sum_{p=1}^k \sum_{c=1}^m d_{pc} = \frac{11}{10}.$$

The reading is that every unit ordered takes $1/w$ of a box: it is the "volumetric" bound, and it coincides with $z(LP)$.

A stronger integer bound

The volumetric bound ignores that a box cannot be split between two customers. Every customer c with at least one unit ordered receives at least

$$\left\lceil \frac{\sum_{p=1}^k d_{pc}}{w} \right\rceil$$

boxes, and these boxes are different from those of the other customers. Summing over all the customers:

Customer	units ordered	minimum boxes
1	7	1
2	4	1
total	11	2

$$z(MILP) \geq LB = 2,$$

almost twice the volumetric bound $11/10$.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. Plant 2 serves both customers: customer 1 with one box containing 5 units of product 1 and 2 of product 2, customer 2 with one box containing 4 units of product 2.

$$z(MILP) = 2, \quad 2 = LB \leq z(MILP) = 2 \leq UB = 2, \quad z(LP) = z(LP^+) = \frac{11}{10}.$$

The heuristic gap is zero, and the integer bound certifies optimality. The difference $2 - 11/10 = 9/10$ is entirely due to integrality: 45% of the optimal value.

10.5: optimal plan with 2 boxes

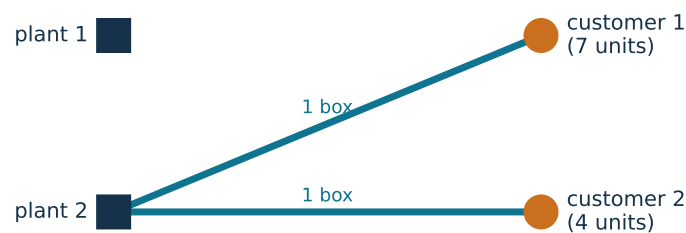


Figure 10.5: The optimal plan: which plants serve which customers and with how many boxes.

Additional considerations

- The model is a multi-product flow with costs on the containers only. If the costs were on the flow (per unit carried) and there were no boxes, the problem would be a pure transportation problem, solvable in polynomial time and with an integral relaxation.
- The variables x_{psc} could be continuous without changing the optimum, because the data are integer and the transportation matrix is totally unimodular *for fixed y*. They stay integer because the problem speaks of units of product.
- Constraint (10.9d) uses the same capacity w on every route. With boxes of different sizes one would need several families $y_{sc}^{(t)}$, one per type, as in problem 10.4.

Additional modelling questions

Modelling question 10.9 — smaller boxes

The boxes hold 4 units instead of 10. What is the new optimum?

Modelling question 10.10 — separate products

Different products cannot travel in the same box. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	2	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	2	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

The sandwich on variant 5a

The capacity of the box changes, 4 instead of 10, not the structure: the recipe stays $\beta = 0$ and $\gamma = \alpha = 1/w$, the dual constraints stay satisfied by construction and the bound of the relaxation is still “units ordered divided by the capacity of the box”, that is $11/4$ instead of $11/10$ — inversely proportional to the capacity, so smaller boxes raise it. As in the base problem, though, the per-customer count is stronger: every customer receives at least $\lceil (\text{its units})/4 \rceil$ boxes and boxes are not shared between customers, so $LB = 2 + 1 = 3$. The heuristic uses 3: the sandwich closes, and the optimum goes from 2 to 3.

	value	what it is
UB	3	heuristic solution
LB	3	count of the boxes per customer
$z(LP)$	$\frac{11}{4}$	relaxation without the bounds
$z(LP^+)$	$\frac{11}{4}$	relaxation with the bounds
$z(MILP)$	3	optimum of the MILP

10.6 Summer camps

Script: `python/fam10_6_camps.py`

[Notebook in Colab](#)

Problem 10.6 — summer camps

A company runs $r \in \mathbb{Z}_{\geq 1}$ summer camps to host children during the holidays. For every camp $j \in \{1, 2, \dots, r\}$, the value $d_j \in \mathbb{Z}_{\geq 1}$ is the maximum number of children that can be hosted. The company has received requests from children of $s \in \mathbb{Z}_{\geq 1}$ different nationalities: for every nationality $i \in \{1, 2, \dots, s\}$ there are $f_i \in \mathbb{Z}_{\geq 0}$ girls and $g_i \in \mathbb{Z}_{\geq 0}$ boys. In every camp the number of girls must be greater than or equal to the number of boys, and the number of children of nationality $c \in \{1, 2, \dots, s\}$ must be greater than or equal to that of every other nationality. The company wants to maximise the total number of children accepted.

The problem in words. We *decide* how many children of each nationality and of each sex to send to each camp. *The objective:* the largest number of children accepted. *The constraints:* neither the availabilities nor the capacities may be exceeded; and in every camp the two composition rules must hold.

Data (input of the model)

Symbol	Type	Meaning
s	$\in \mathbb{Z}_{\geq 1}$	number of nationalities, $i \in \{1, 2, \dots, s\}$
r	$\in \mathbb{Z}_{\geq 1}$	number of camps, $j \in \{1, 2, \dots, r\}$
f_i	$\in \mathbb{Z}_{\geq 0}$	girls available of nationality i
g_i	$\in \mathbb{Z}_{\geq 0}$	boys available of nationality i
d_j	$\in \mathbb{Z}_{\geq 1}$	capacity of camp j
c	$\in \{1, 2, \dots, s\}$	nationality that must be the majority

Decision variables

The variables are not binary but *counts*: $2rs$ non-negative integer variables,

$$x_{ij} = \text{girls of nationality } i \text{ in camp } j, \quad y_{ij} = \text{boys of nationality } i \text{ in camp } j.$$

Model 10.6 — summer camps

$$\max \sum_{i=1}^s \sum_{j=1}^r (x_{ij} + y_{ij}) \quad (10.11a)$$

$$\text{subject to } \sum_{j=1}^r x_{ij} \leq f_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11b)$$

$$\sum_{j=1}^r y_{ij} \leq g_i, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.11c)$$

$$\sum_{i=1}^s (x_{ij} + y_{ij}) \leq d_j, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11d)$$

$$\sum_{i=1}^s (x_{ij} - y_{ij}) \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.11e)$$

$$x_{cj} + y_{cj} - x_{ij} - y_{ij} \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, i \neq c, \forall j \in \{1, 2, \dots, r\}, \quad (10.11f)$$

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}, \quad (10.11g)$$

$$y_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \{1, 2, \dots, s\}, \forall j \in \{1, 2, \dots, r\}. \quad (10.11h)$$

Description. Constraints (10.11b) and (10.11c) cap the availabilities ($2s$ constraints); constraints (10.11d) the capacities (r constraints); (10.11e) impose "girls $\geq$ boys" in every camp (r constraints); (10.11f) impose that nationality c is not fewer than each of the others, one inequality per nationality-camp pair ($(s-1)r$ constraints).

The majority constraint with more than two nationalities

Constraint (10.11f) translates the statement literally: nationality c is not fewer than *each* of the others, one inequality per pair, $(s-1)r$ in all. There is however a second, shorter writing that is easy to mistake for this one:

$$x_{cj} + y_{cj} \geq \sum_{i \neq c} (x_{ij} + y_{ij}) \quad \forall j \in \{1, 2, \dots, r\},$$

that is r inequalities instead of $(s-1)r$. It says something else, and it is *stronger*: that nationality c is not fewer than *all the others together*, that is that it takes at least half of the places in every camp. The two readings coincide for $s = 2$ — with a single "other" nationality the sum has one term — and diverge for $s > 2$, where the aggregated form cuts off solutions the statement allows. The choice is made by reading the text, not for convenience of writing: fewer constraints does not mean a better model if they are not the constraints of the problem.

On the instance of this exercise $s = 2$, so the two forms give the same numbers; and it is precisely because $s = 2$ that the combinatorial argument below can read the majority as "half of the places".

The two bounds built below — the primal one, from the heuristic, and the dual one, from the relaxation — are what the solver works with.

The model in gurobipy

```

1 m = gp.Model("camps")
2 x = m.addVars(s, r, vtype=GRB.INTEGER, name="x")
3 y = m.addVars(s, r, vtype=GRB.INTEGER, name="y")
4 m.setObjective(gp.quicksum(x[i, j] + y[i, j] for i in range(s) for j in
   range(r)),
5               GRB.MAXIMIZE)
6 m.addConstrs((x.sum(i, "*") <= f[i] for i in range(s)), name="girls")
7 m.addConstrs((y.sum(i, "*") <= g[i] for i in range(s)), name="boys")
8 m.addConstrs((gp.quicksum(x[i, j] + y[i, j] for i in range(s)) <= d[j]
9               for j in range(r)), name="capacity")
10 m.addConstrs((gp.quicksum(x[i, j] - y[i, j] for i in range(s)) >= 0
11               for j in range(r)), name="balance")
12 m.addConstrs((x[c, j] + y[c, j] - x[i, j] - y[i, j] >= 0
13               for i in range(s) if i != c for j in range(r)), name="
   majority")

```

The instance

$s = 2$ nationalities, $r = 2$ camps, $c = 1$:

	$i = 1$	$i = 2$		$j = 1$	$j = 2$
f_i	8	10	d_j	15	8
g_i	4	12			

In total there are 34 children available and 23 places.

Constructive heuristic: the primal bound

The problem is a maximisation. One camp is filled at a time, taking first the majority nationality (girls and then boys) and then the others, stopping as soon as one of the three constraints would break.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance:

- **Camp 1** (capacity 15): all 8 girls and all 4 boys of nationality 1 are taken, then 3 girls of nationality 2. The camp is full: 15 places, 12 of nationality 1 against 3 of nationality 2 (majority respected), 11 girls against 4 boys (balance respected).
- **Camp 2** (capacity 8): nationality 1 is exhausted, so any child of nationality 2 would violate the majority. The camp stays empty.

$$z(MILP) \geq LB = 15.$$

LP relaxation and dual: the dual bound

We associate $\alpha_i, \beta_i, \gamma_j \geq 0$ with the three groups of $\leq$ constraints and $\delta_j, \varepsilon_j \geq 0$ with the two groups of composition constraints. The dual of the LP relaxation is:

Dual of the LP relaxation of model 10.6

$$\min \sum_{i=1}^s f_i \alpha_i + \sum_{i=1}^s g_i \beta_i + \sum_{j=1}^r d_j \gamma_j \quad (10.12a)$$

$$\text{subject to } \alpha_c + \gamma_j - \delta_j - \sum_{k \neq c} \varepsilon_{kj} \geq 1, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12b)$$

$$\beta_c + \gamma_j + \delta_j - \sum_{k \neq c} \varepsilon_{kj} \geq 1, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12c)$$

$$\alpha_i + \gamma_j - \delta_j + \varepsilon_{ij} \geq 1, \quad \forall i \neq c, \forall j \in \{1, 2, \dots, r\}, \quad (10.12d)$$

$$\beta_i + \gamma_j + \delta_j + \varepsilon_{ij} \geq 1, \quad \forall i \neq c, \forall j \in \{1, 2, \dots, r\}, \quad (10.12e)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12f)$$

$$\beta_i \geq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.12g)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12h)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.12i)$$

$$\varepsilon_{ij} \geq 0, \quad \forall i \neq c, \forall j \in \{1, 2, \dots, r\}, \quad (10.12j)$$

Description. The variable α_i is the price of a place for the girls of nationality i and β_i that for the boys; γ_j is the price of a place in camp j , δ_j the price of the balance constraint and ε_{ij} that of the comparison between nationality i and the majority one in camp j . The objective (10.12a) prices the availability of each nationality and the capacity of each camp.

The dual constraints are the columns of the primal, and they split into two blocks because nationality c appears in the comparisons differently from the others. Constraints (10.12b) are the columns of the x_{cj} : accepting one girl of the majority nationality uses one place of her nationality and one of the camp, raises the balance by one unit and *loosens* every comparison of that camp, one per other nationality — hence the sum with the minus sign. Constraints (10.12d) are the columns of the x_{ij} with $i \neq c$: that girl tightens one comparison only, her own. In both cases the total value must cover the unit that girl contributes to the primal objective. Constraints (10.12c) and (10.12e) say the same for the boys, with the sign of the balance reversed.

A hand-built dual solution. The simplest recipe prices only the capacity: $\alpha = \beta = \delta = \varepsilon = 0$ and $\gamma_j = 1$ for every camp. All the dual constraints become $\gamma_j \geq 1$ and are satisfied, and

$$UB = \sum_{j=1}^r d_j = 15 + 8 = 23.$$

The reading is obvious: every accepted child takes one place, so no more children can be accepted than there are places. And it is also *optimal*: on the relaxation without the bounds $z(LP) = 23$.

Two more combinatorial arguments

The bound 23 is not the only one that can be read off the data. Constraint (10.11f) says that in every camp nationality c takes at least half of the places; since there are $f_c + g_c = 12$ children of that nationality in total, at most $2 \cdot 12 = 24$ children can be accepted. Likewise constraint (10.11e) says that in every camp the girls are at least half, and there are 18 girls: at most $2 \cdot 18 = 36$ children can be accepted.

Argument	upper bound
capacity of the camps	23
majority nationality	24
girls available	36

On this instance the capacity wins, but that is not a rule: variant 11a below enlarges camp 1 and hands the lead to the majority nationality.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	camp 1		camp 2	
	girls	boys	girls	boys
nationality 1	8	0	0	4
nationality 2	1	6	4	0
total	15 out of 15		8 out of 8	

Both camps are full: $z(MILP) = 23$, and the chain of bounds is

$$15 = LB \leq z(MILP) = 23 = z(LP) = UB.$$

The heuristic gap is 34.8%, and the dual bound settles the problem: the whole gap was on the solution side, not on the certificate side. The mistake of the heuristic is clear: it uses up the majority nationality in the first camp, and in the second one nobody is left who can form the majority.

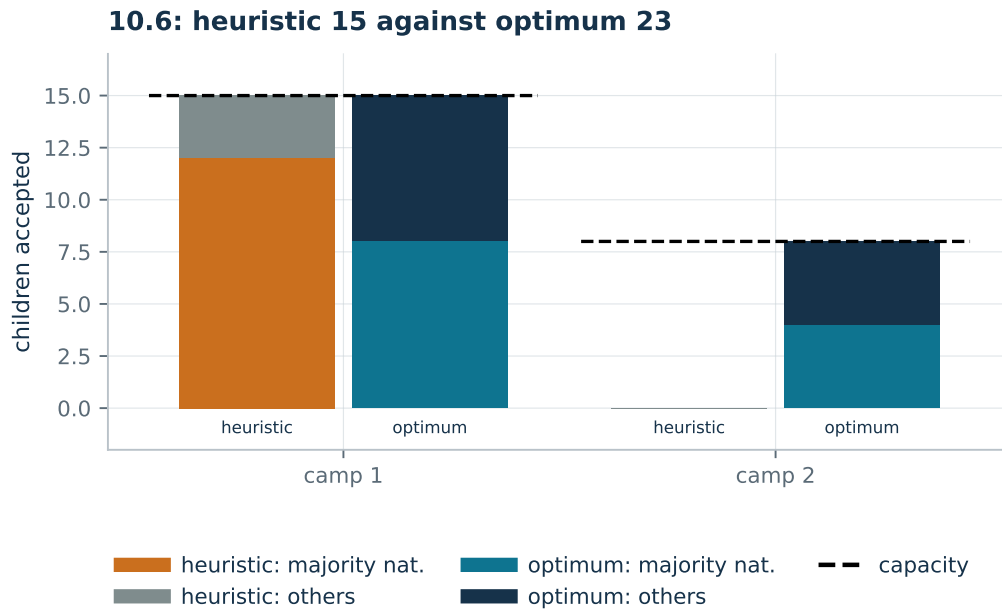


Figure 10.6: Children accepted per camp, split between majority nationality and the others, in the heuristic solution and in the optimal one; the dashed line is the capacity.

Additional considerations

- The variables are integer but not binary: it is the first family of the course in which the counts take large values, and the relaxation stays tight anyway because all the constraints are sums.
- The balance constraint and the majority constraint are *independent*: one can have camps with many girls and few children of nationality c , and vice versa. On the instance it is the second one that bites.
- If a nationality had zero children available, the majority constraint would automatically exclude it from every camp in which anyone else appears: a limit case worth checking on the data.

Additional modelling questions

Modelling question 10.11 — a bigger camp

Camp 1 is enlarged and reaches 20 places. What is the new optimum?

Modelling question 10.12 — a nationality that cannot be split

For organisational reasons the children of nationality 1 must all stay in the same camp. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
LB	15	heuristic solution (LB)
UB	23	certified bound (UB)
$z(MILP)$	23	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|LB - z(MILP)|}{|z(MILP)|} = 34.8\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|LB - UB|}{|z(MILP)|} = 34.8\%.$$

The sandwich on variant 6a

Camp 1 goes from 15 to 20 places: a datum changes, not the structure. The recipe is the one of the base problem — $\gamma_j = 1$ and all the other dual variables at zero, because every child accepted takes a place — and it returns $\sum_j d_j$, which is now 28 instead of 23. The certificate does not change shape: the datum it adds up does. The heuristic fills the camps in order and reaches 20, the optimum is 24.

	value	what it is
UB	28	dual certificate built by hand
LB	20	heuristic solution
$z(LP)$	24	relaxation without the bounds
$z(LP^+)$	24	relaxation with the bounds
$z(MILP)$	24	optimum of the MILP

10.7 Antitrust split

Script: `python/fam10_7_antitrust.py`

[Notebook in Colab](#)

Problem 10.7 — antitrust split

A company has $s \in \mathbb{Z}_{\geq 1}$ branches and sells $r \in \mathbb{Z}_{\geq 1}$ products. For every branch $i \in \{1, 2, \dots, s\}$ and every product $j \in \{1, 2, \dots, r\}$, the value $v_{ij} \in \mathbb{Q}_{\geq 0}$ is the revenue in millions of euros generated by that branch with that product. Because of a new antitrust regulation the company must split into two smaller companies; every branch is indivisible and must be assigned to exactly one of the two. The company wants to partition the branches minimising the maximum difference, over all the products, between the revenues of the two new companies.

The problem in words. We *decide* which of the two companies each branch goes to. *The objective*: to make the two companies as similar as possible, measuring similarity on the product where they do worst. *The constraint*: every branch to a single company.

Decision variables

One family of s binaries is enough:

$$x_i = \begin{cases} 1 & \text{if branch } i \text{ goes to company } A, \\ 0 & \text{if it goes to company } B, \end{cases} \quad \forall i \in \{1, 2, \dots, s\},$$

plus a free variable z for the min-max.

One could also declare $z \geq 0$: the two constraints keep it above an absolute value, hence non-negative, and the solution would not change. It is left free because that is the choice one sees in the dual — the column of a free variable gives an *equality* constraint, and that is where the condition $\sum_j (\lambda_j + \mu_j) = 1$ comes from — and because this way non-negativity stays a consequence of the model, not an assumption added by hand.

One family or two?

The classic statement introduces *two* families, x_i and y_i , with the constraint $x_i + y_i = 1$. The two writings are equivalent, because $y_i = 1 - x_i$: substituting one obtains exactly the model below, with s fewer variables and s fewer constraints. It is better to keep the aggregated form as long as the companies are *two*; with three or more companies the disaggregated form $\sum_{k=1}^2 x_{ik} = 1$ is the only possible one, and it is the one that extends. Recognising when two models are the same thing is part of the craft: here the check is a substitution, in other cases (problem 9.2) it requires a proof.

With $T_j = \sum_{i=1}^s v_{ij}$ the total revenue on product j , the revenue of A is $\sum_{i=1}^s v_{ij} x_i$ and that of B is $T_j - \sum_{i=1}^s v_{ij} x_i$: their difference is $2 \sum_{i=1}^s v_{ij} x_i - T_j$.

Model 10.7 — antitrust split

$$\min \quad z \tag{10.13a}$$

$$\text{subject to} \quad z - 2 \sum_{i=1}^s v_{ij} x_i + T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \tag{10.13b}$$

$$z + 2 \sum_{i=1}^s v_{ij} x_i - T_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \tag{10.13c}$$

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, s\}, \tag{10.13d}$$

$$z \geq 0. \tag{10.13e}$$

Description. The objective (10.13a) is the worst imbalance. Constraints (10.13b) and (10.13c), one per product each, say that z is no smaller than the imbalance of product j nor than its opposite: together they impose $z \geq |2 \sum_i v_{ij} x_i - T_j|$, and the minimisation turns them into an equality on the worst product.

Absolute value and min-max in two lines

The two inequalities (10.13b) and (10.13c) say

$$-z \leq 2 \sum_{i=1}^s v_{ij} x_i - T_j \leq z,$$

that is $z \geq |2 \sum_{i=1}^s v_{ij} x_i - T_j|$ for every product j . Since z appears alone in the minimisation objective, at the optimum it equals exactly the maximum of the absolute values: it is technique 3.6 (min-max) applied to technique 3.7 (absolute value). The direction " z is no larger than necessary" is not imposed by any constraint, but by optimality: if z were larger, it could be lowered while staying feasible.

The model in gurobipy

```

1 m = gp.Model("antitrust")
2 x = m.addVars(s, vtype=GRB.BINARY, name="x")
3 z = m.addVar(lb=-GRB.INFINITY, name="z")
4 m.setObjective(z, GRB.MINIMIZE)
5 for j in range(r):
6     tot = sum(v[i][j] for i in range(s))
7     m.addConstr(z - 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) +
8                 tot >= 0)
9     m.addConstr(z + 2 * gp.quicksum(v[i][j] * x[i] for i in range(s)) -
10                 tot >= 0)

```

The instance

$s = 4$ branches, $r = 3$ products:

v_{ij}	$j = 1$	$j = 2$	$j = 3$	branch total
$i = 1$	3	3	2	8
$i = 2$	6	8	5	19
$i = 3$	3	4	4	11
$i = 4$	2	7	9	18
T_j	14	22	20	

Constructive heuristic: the primal bound

The branches are assigned in decreasing order of total revenue, each one to the company that currently has the smaller total. It is the analogue of LPT for scheduling.

It is a constructive heuristic of the family seen in chapter "Constructive heuristics": a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

On the instance the totals are 8, 19, 11, 18, so the order is 2, 4, 3, 1: branch 2 to A ($A = 19$), branch 4 to B ($B = 18$), branch 3 to B ($B = 29$), branch 1 to A ($A = 27$). One gets $A = \{1, 2\}$, $B = \{3, 4\}$, with differences 4, 0 and 6 on the three products:

$$z(MILP) \leq UB = 6.$$

LP relaxation and dual: the bound is zero

We associate $\lambda_j \geq 0$ with constraints (10.13b) and $\mu_j \geq 0$ with those (10.13c). The column of z (a free variable) gives an equality constraint:

Dual of the LP relaxation of model 10.7

$$\max \sum_{j=1}^r T_j (\mu_j - \lambda_j) \quad (10.14a)$$

$$\text{subject to } \sum_{j=1}^r (\lambda_j + \mu_j) = 1, \quad (10.14b)$$

$$2 \sum_{j=1}^r v_{ij} (\mu_j - \lambda_j) \leq 0, \quad \forall i \in \{1, 2, \dots, s\}, \quad (10.14c)$$

$$\lambda_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}, \quad (10.14d)$$

$$\mu_j \geq 0, \quad \forall j \in \{1, 2, \dots, r\}. \quad (10.14e)$$

Description. The variables λ_j and μ_j are the prices of the two constraints that bound the imbalance of product j , one from above and one from below. The objective (10.14a) prices the total T_j of each product at those values. Constraint (10.14b) is the column of z : the variable appears in every constraint with coefficient 1 and in the primal objective with cost 1, so the prices of the $2r$ constraints must sum to exactly one — the dual distributes a single unit of weight among the products. Constraints (10.14c) are the columns of the x_i : moving branch i from one company to the other changes the imbalance of product j by $2v_{ij}$, and the weighted sum of these moves cannot be positive, because the x_i do not appear in the objective.

A hand-built dual solution. Take $\bar{\lambda}_1 = \bar{\mu}_1 = 1/2$ and everything else zero: constraint (10.14b) is satisfied, $\mu_j - \lambda_j = 0$ for every j and therefore so are constraints (10.14c). The value is $LB = 0$.

No feasible dual solution is worth more than zero. Indeed, setting $\theta_j = \mu_j - \lambda_j$, the objective is $\sum_{j=1}^r T_j \theta_j$ and constraints (10.14c) impose $\sum_{j=1}^r v_{ij} \theta_j \leq 0$ for every branch; summing over all the branches one gets $\sum_{j=1}^r T_j \theta_j \leq 0$. The dual is therefore worth at most 0, and by strong duality $z(LP) = 0$.

The primal certificate is even simpler: the fractional solution $x_i = 1/2$ for every branch with $z = 0$ is feasible for the relaxation and balances every product exactly. Half a branch to each company: legitimate for the LP, meaningless for the problem.

A combinatorial bound, product by product

If the relaxation says nothing, one must look elsewhere. For every product j one can compute, looking at *that product alone*, the smallest imbalance achievable:

$$g_j = \min_{S \subseteq \{1, 2, \dots, s\}} \left| 2 \sum_{i \in S} v_{ij} - T_j \right|.$$

It is the classic partition problem on a single column, and with a small s it is solved by enumeration (2^s subsets). Every feasible partition of the complete problem is in particular a partition for product j , so its imbalance on product j is at least g_j ; since the objective is the maximum over all products,

$$z(MILP) \geq \max_j g_j.$$

On the instance:

Product	total T_j	g_j
1	14	2
2	22	0
3	20	2

(on product 1 the values are 3, 6, 3, 2: the sum 7 is not reachable by any subset, and the best is 6 against 8, that is $g_1 = 2$). Hence

$$z(MILP) \geq LB = 2,$$

a bound that the LP relaxation cannot see, because it comes from integrality and not from the constraints.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there. The optimal partition is $A = \{2, 3\}$ and $B = \{1, 4\}$, with differences 4, 2 and 2 on the three products:

$$z(MILP) = 4,$$

and the chain of bounds is

$$2 = LB \leq z(MILP) = 4 \leq UB = 6, \quad z(LP) = z(LP^+) = 0.$$

The heuristic gap is 50%. Note that LB is not the value of the dual (0) but the combinatorial bound: in the summary table of the chapter the column is called "certified bound" precisely for this reason.

Additional considerations

- The problem with a single product is *number partitioning*, one of Karp's twenty-one NP-complete problems. With several products it is its vector version, and it stays NP-hard.
- The combinatorial bound can be strengthened: instead of the maximum of the g_j one can look, for every pair of products, for the minimum of the maximum of the two imbalances. The cost grows but the bound improves, and it is the same idea as *surrogate relaxation*.
- The min-max objective is not the only possible one: minimising the sum of the differences is another legitimate choice and gives a model with r auxiliary variables instead of one (see question 10.9).

Additional modelling questions

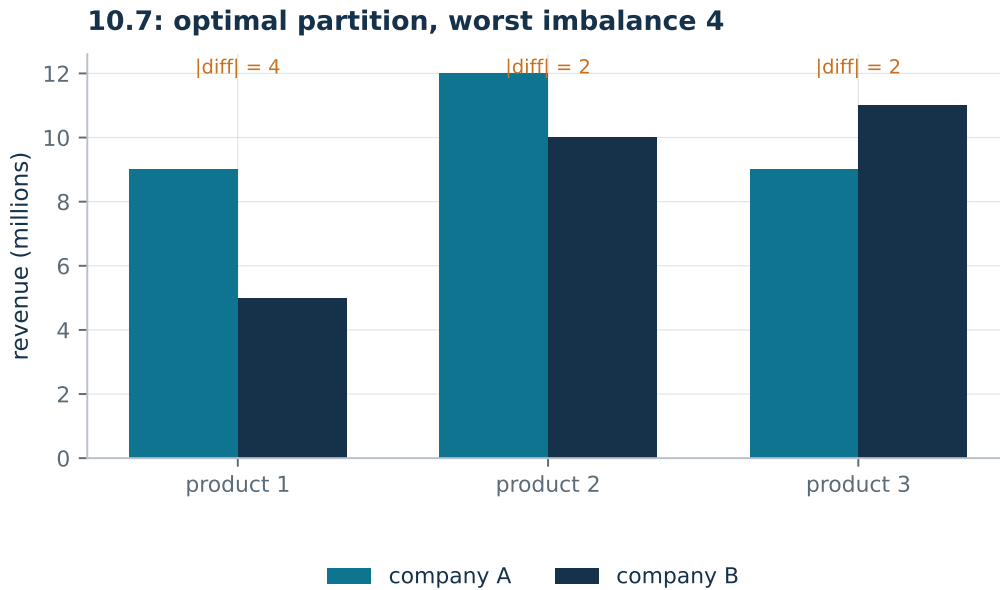


Figure 10.7: Revenue of the two optimal companies, product by product, with the difference highlighted.

Modelling question 10.13 — two inseparable branches

Branches 1 and 2 share their premises and must stay in the same company. How does the model change? What is the new optimum?

Modelling question 10.14 — min-sum instead of min-max

One wants to minimise the *sum* of the differences over all the products instead of the worst difference. How does the model change? Is the optimal partition the same?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	6	heuristic solution (UB)
LB	2	certified bound (LB)
$z(MILP)$	4	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 50.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 100.0\%.$$

The sandwich on variant 7a

The equality $x_1 - x_2 = 0$ adds to the dual a free σ , which appears with a plus sign in the column of branch 1 and with a minus in that of branch 2. Its right-hand side is zero, so the objective does not change: the relaxation is worth 0 as in the base problem, and for the same reason. The useful bound is combinatorial again: keeping branches 1 and 2 together, the best possible imbalance on the three products is 4, 0 and 2, and every feasible partition must respect all of them, so $z(MILP) \geq 4$. With the branches free it was 2: tying two branches raises the bound.

	value	what it is
UB	6	heuristic solution
LB	4	smallest imbalance with branches 1 and 2 tied
$z(LP)$	0	relaxation without the bounds
$z(LP^+)$	0	relaxation with the bounds
$z(MILP)$	6	optimum of the MILP

10.8 Songs on CDs

Script: `python/fam10_8_cds.py`

[Notebook in Colab](#)

Problem 10.8 — songs on CDs

A music production company publishes a collection of $n \in \mathbb{Z}_{\geq 1}$ songs. For every song $i \in \{1, 2, \dots, n\}$, the value $d_i \in \mathbb{Q}_{\geq 1}$ is the duration in minutes. The collection must be distributed over $m \in \mathbb{Z}_{\geq 1}$ compact discs; every song must be assigned to exactly one CD, and every CD $j \in \{1, 2, \dots, m\}$ must contain at least $w_j \in \mathbb{Z}_{\geq 1}$ songs. The company wants to minimise the difference between the total duration of the longest CD and that of the shortest one.

The problem in words. We *decide* which CD every song ends up on. *The objective*: CDs as homogeneous as possible in duration. *The constraints*: every song on one CD only, and no CD too sparse.

Decision variables

$n m$ binary variables and two non-negative continuous ones:

$$x_{ij} = \begin{cases} 1 & \text{if song } i \text{ goes on CD } j, \\ 0 & \text{otherwise,} \end{cases}$$

$$y = \text{duration of the longest CD}, \quad z = \text{duration of the shortest CD}.$$

Model 10.8 — songs on CDs

$$\min \quad y - z \quad (10.15a)$$

$$\text{subject to} \quad \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.15b)$$

$$\sum_{i=1}^n x_{ij} \geq w_j, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15c)$$

$$-\sum_{i=1}^n d_i x_{ij} + y \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15d)$$

$$\sum_{i=1}^n d_i x_{ij} - z \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.15e)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (10.15f)$$

$$y, z \geq 0. \quad (10.15g)$$

Description. The objective (10.15a) is the difference between the longest and the shortest duration. Constraints (10.15b), one per song, say that every song ends up on exactly one CD. Constraints (10.15c), one per CD, impose the minimum number of songs. Constraints (10.15d), one per CD, push y above every duration, and constraints (10.15e), again one per CD, push z below every duration: at the optimum the first becomes the maximum of the durations and the second the minimum.

Two auxiliary variables, two opposite directions

Constraints (10.15d) say $y \geq \sum_{i=1}^n d_i x_{ij}$ for every CD: y is *at least* the maximum. Constraints (10.15e) say $z \leq \sum_{i=1}^n d_i x_{ij}$ for every CD: z is *at most* the minimum. The objective $\min(y - z)$ pushes y down and z up, so at the optimum y equals exactly the largest duration and z exactly the smallest one. It is technique 3.5 used twice, in two opposite directions: note that one needs an objective that pushes *both* variables in the right direction, otherwise one of the two would stay undetermined.

The model in gurobipy

```

1 mod = gp.Model("cbs")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVar(name="y")
4 z = mod.addVar(name="z")
5 mod.setObjective(y - z, GRB.MINIMIZE)
6 mod.addConstrs((x.sum(i, "*") == 1 for i in range(n)), name="song")
7 mod.addConstrs((x.sum("j", j) >= w[j] for j in range(m)), name="minimum")
8 mod.addConstrs((y - gp.quicksum(d[i] * x[i, j] for i in range(n)) >= 0
9                  for j in range(m)), name="maximum")
10 mod.addConstrs((gp.quicksum(d[i] * x[i, j] for i in range(n)) - z >= 0
11                 for j in range(m)), name="minimum_duration")

```

The instance

$n = 6$ songs, $m = 2$ CDs, $w_j = 1$:

	$i = 1$	$i = 2$	$i = 3$	$i = 4$	$i = 5$	$i = 6$
d_i	5	6	7	3	4	10

The total duration is 35 minutes.

Two heuristics compared: the primal bound

The insertion rule is always the same — every song on the currently shortest CD — but the order in which the songs are scanned changes the result a lot.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

(a) Natural order. $5 \rightarrow \text{CD1}$, $6 \rightarrow \text{CD2}$, $7 \rightarrow \text{CD1}$, $3 \rightarrow \text{CD2}$, $4 \rightarrow \text{CD2}$, $10 \rightarrow \text{CD1}$. Final durations 22 and 13: difference 9.

(b) Decreasing duration (LPT). $10 \rightarrow \text{CD1}$, $7 \rightarrow \text{CD2}$, $6 \rightarrow \text{CD2}$, $5 \rightarrow \text{CD1}$, $4 \rightarrow \text{CD2}$, $3 \rightarrow \text{CD1}$. Final durations 18 and 17: difference 1.

The same rule, with two different orders, gives 9 and 1. The better one is kept:

$$z(\text{MILP}) \leq UB = 1.$$

LP relaxation and dual: zero again

We associate α_i free with constraints (10.15b) (equalities), $\beta_j \geq 0$ with (10.15c), $\gamma_j \geq 0$ with (10.15d) and $\delta_j \geq 0$ with (10.15e). The columns of y and z give $\sum_{j=1}^m \gamma_j = 1$ and $\sum_{j=1}^m \delta_j = 1$.

Dual of the LP relaxation of model 10.8

$$\max \sum_{i=1}^n \alpha_i + \sum_{j=1}^m w_j \beta_j \quad (10.16a)$$

$$\text{subject to } \sum_{j=1}^m \gamma_j = 1, \quad (10.16b)$$

$$\sum_{j=1}^m \delta_j = 1, \quad (10.16c)$$

$$\alpha_i + \beta_j - d_i \gamma_j + d_i \delta_j \leq 0, \quad \forall i \in \{1, 2, \dots, n\}, \forall j \in \{1, 2, \dots, m\}, \quad (10.16d)$$

$$\alpha_i \geq 0, \quad \forall i \in \{1, 2, \dots, n\}, \quad (10.16e)$$

$$\beta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16f)$$

$$\gamma_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}, \quad (10.16g)$$

$$\delta_j \geq 0, \quad \forall j \in \{1, 2, \dots, m\}. \quad (10.16h)$$

Description. The variable α_i is the value of song i , β_j the price of the minimum number of songs on CD j , while γ_j and δ_j are the weights with which CD j enters the maximum and the minimum duration respectively. The objective (10.16a) prices the songs and the thresholds w_j . Constraints (10.16b) and (10.16c) are the columns of y and z : each of the two variables appears in one constraint per CD and in the primal objective with coefficient ± 1 , so the respective weights must sum to one. Constraints (10.16d) are the columns of the x_{ij} : putting song i on CD j satisfies its assignment constraint (α_i), contributes to the minimum number of songs (β_j) and moves both durations by d_i ; the balance cannot be positive, because the x_{ij} do not appear in the objective.

A hand-built dual solution. With $\bar{\gamma}_j = \bar{\delta}_j = 1/m$ and $\alpha = \beta = 0$ the two equality constraints are satisfied and constraints (10.16d) become $-d_i/m + d_i/m = 0 \leq 0$. The value is $LB = 0$.

That the relaxation is worth zero is best seen from the primal side: putting $x_{ij} = 1/m$ for every song and every CD, every CD "lasts" $35/2 = 17.5$ minutes and the difference is zero. It is feasible for the relaxation and meaningless for the problem, because a song cannot be split. The LP relaxation of balancing models is almost always like this: it levels everything and is worth zero.

A parity argument that settles the problem

The durations are integers and the CDs are two: their durations q_1 and q_2 are integers adding up to $q = 35$, which is *odd*. Two integers adding up to an odd number cannot be equal, and their difference $|q_1 - q_2|$ has the same parity as q , that is it is odd. An odd non-negative difference is at least 1:

$$z(MILP) \geq LB = 1.$$

The LPT heuristic reaches exactly 1: the two bounds coincide, and the heuristic solution is optimal. One knows this *before* calling the solver, and it is the clearest case in the course in which a two-line combinatorial argument does the job that the LP relaxation cannot do.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	songs	duration
CD 1	1, 2, 3	18
CD 2	4, 5, 6	17

$$1 = LB \leq z(MILP) = 1 \leq UB = 1, \quad z(LP) = z(LP^+) = 0.$$

The heuristic gap is zero. Note that the solution found by the solver is not the one of the LPT heuristic (which put songs 6, 1, 4 on one CD and 3, 2, 5 on the other), but has the same value: the problem has several optima.

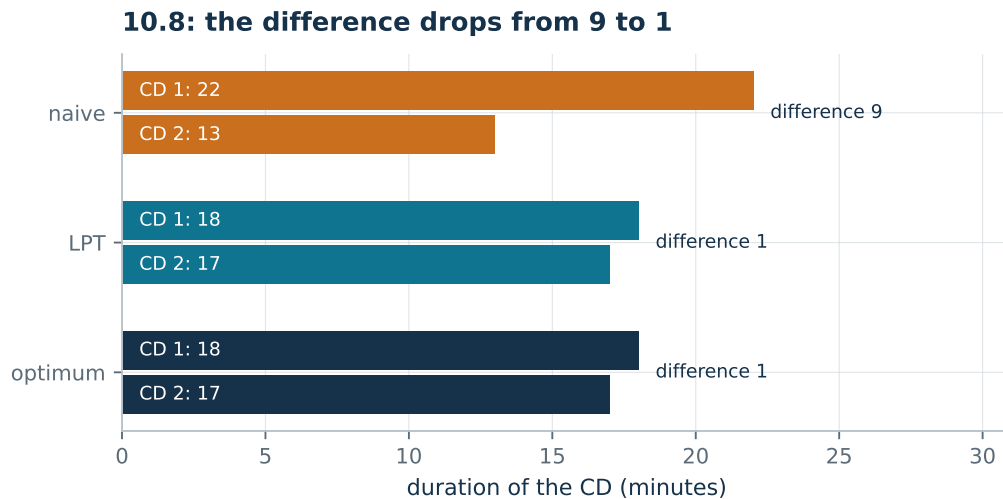


Figure 10.8: Duration of the two CDs in the three solutions: naive heuristic, LPT and optimum.

Additional considerations

- The parity argument holds as written for $m = 2$. With $m = 3$ it must be redone: 35 is not divisible by 3, so the three durations cannot all be equal, but the minimum spread is no longer immediate. On the instance with three CDs the optimum is 2 (durations 13, 11, 11).
- The constraint $\sum_{i=1}^n x_{ij} \geq w_j$ with $w_j = 1$ never bites at the optimum: a solution leaving a CD empty would have $z = 0$ and therefore a difference equal to the total duration. It is needed in the model, though, because without it the relaxation would have degenerate solutions.
- Minimising $y - z$ is not the same as minimising y (the makespan): the latter is the classic formulation of scheduling on parallel machines, and it has different optima. It is the same difference as between "balancing" and "finishing early".

Additional modelling questions

Modelling question 10.15 — a smaller medium

CD 1 is a reduced medium and cannot exceed 15 minutes. How does the model change? What is the new optimum?

Modelling question 10.16 — three CDs

The collection is distributed over three CDs instead of two. How does the model change? What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	1	heuristic solution (UB)
LB	1	certified bound (LB)
$z(MILP)$	1	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 0.0\%.$$

The sandwich on variant 8b

With three CDs instead of two the structure does not change: the columns of y and z ask the γ and the δ to sum to one, and spreading them evenly, $\gamma_j = \delta_j = 1/3$, is the choice that favours no CD. The dual is worth 0, like the relaxation, for the usual reason: a fractional solution splits every track into three equal parts and levels the durations. The useful bound comes from integrality, as in the base problem: the durations are integers summing to 35, which is not a multiple of 3, so they cannot all be equal and the difference is at least 1. The LPT heuristic leaves 2, which is the optimum: the certified gap stays open by one unit.

	value	what it is
UB	2	heuristic solution
LB	1	parity of the durations on three CDs
$z(LP)$	0	relaxation without the bounds
$z(LP^+)$	0	relaxation with the bounds
$z(MILP)$	2	optimum of the MILP

10.9 Books on shelves

Script: `python/fam10_9_shelves.py`

[Notebook in Colab](#)

Problem 10.9 — books on shelves

A library needs to arrange $n \in \mathbb{Z}_{\geq 1}$ books on shelves. Every book $b \in \{1, 2, \dots, n\}$ has width $w_b \in \mathbb{Z}_{\geq 1}$ and height $h_b \in \mathbb{Z}_{\geq 1}$. There are $m \in \mathbb{Z}_{\geq 1}$ shelves available, each of maximum width $c \in \mathbb{Q}_{>0}$. Every book must be assigned to exactly one shelf, and the total width of the books on a shelf cannot exceed c . The height of a shelf is that of the tallest book placed on it. The library wants to minimise the sum of the heights of the shelves.

The problem in words. We *decide* which shelf every book goes on. *The objective*: minimum sum of the heights (it is the timber that is saved). *The constraints*: every book on one shelf only, and no shelf wider than c .

Decision variables

n m binary variables and m non-negative continuous ones:

$$x_{bs} = \begin{cases} 1 & \text{if book } b \text{ goes on shelf } s, \\ 0 & \text{otherwise,} \end{cases} \quad y_s = \text{height of shelf } s.$$

Model 10.9 — books on shelves

$$\min \sum_{s=1}^m y_s \quad (10.17a)$$

$$\text{subject to } \sum_{s=1}^m x_{bs} = 1, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.17b)$$

$$\sum_{b=1}^n w_b x_{bs} \leq c, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.17c)$$

$$-h_b x_{bs} + y_s \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17d)$$

$$x_{bs} \in \{0, 1\}, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.17e)$$

$$y_s \geq 0, \quad \forall s \in \{1, 2, \dots, m\}. \quad (10.17f)$$

Description. The objective (10.17a) is the total height of the shelves. Constraints (10.17b), one per book, say that every book sits on exactly one shelf. Constraints (10.17c), one per shelf, are the available width. Constraints (10.17d), one per book–shelf pair, push the height y_s above that of every book placed on that shelf.

The link between the variables. Constraints (10.17d) say $y_s \geq h_b x_{bs}$: if book b is on shelf s then $y_s \geq h_b$, otherwise the constraint is empty ($y_s \geq 0$). Since y_s appears in the minimisation objective with a positive coefficient, at the optimum it equals exactly the largest height among the books present, and it is 0 if the shelf is empty. It is technique 3.5, the maximum variable, in its disaggregated form: one constraint per book instead of a single one with a big-M.

An empty shelf costs nothing

With $y_s \geq 0$ and no other constraint, an empty shelf has $y_s = 0$ and does not contribute to the objective. It follows that giving the library more shelves can never worsen the optimum, but it need not improve it: splitting the books over more shelves means paying more heights. On the instance, with three shelves instead of two, the optimum stays 15.

The model in gurobipy

```
1 mod = gp.Model("shelves")
2 x = mod.addVars(n, m, vtype=GRB.BINARY, name="x")
3 y = mod.addVars(m, name="y")
4 mod.setObjective(y.sum(), GRB.MINIMIZE)
5 mod.addConstrs((x.sum(b, "*") == 1 for b in range(n)), name="book")
6 mod.addConstrs((gp.quicksum(w[b] * x[b, s] for b in range(n)) <= c
```

```

7         for s in range(m)), name="width")
8 mod.addConstrs((y[s] - h[b] * x[b, s] >= 0 for b in range(n) for s in
           range(m)),
9               name="height")

```

The instance

$n = 4$ books, $m = 2$ shelves, $c = 10$:

	$b = 1$	$b = 2$	$b = 3$	$b = 4$
w_b	3	5	4	6
h_b	8	5	7	4

The total width of the books is 18, the overall capacity $2 \cdot 10 = 20$.

Constructive heuristic: two orders, two outcomes

The rule is first-fit: every book on the first shelf where it fits. As in problem 10.8 the order changes everything — but here, in one order, the heuristic *fails*.

It is a constructive heuristic of the family seen in chapter “Constructive heuristics”: a single solution is built, one element at a time, never backtracking. Below is its run on the instance.

(a) Decreasing height (1, 3, 2, 4). Book 1 (width 3) on shelf 1; book 3 (width 4) on shelf 1, which reaches 7; book 2 (width 5) on shelf 2. Book 4, of width 6, is left: shelf 1 has 3 of residual width, shelf 2 has 5. **The heuristic fails.**

(b) Decreasing width (4, 2, 3, 1). Book 4 (width 6) on shelf 1; book 2 (width 5) on shelf 2; book 3 (width 4) on shelf 1, which reaches 10; book 1 (width 3) on shelf 2, which reaches 8. Heights: $\max(4, 7) = 7$ and $\max(5, 8) = 8$, sum 15.

$$z(MILP) \leq UB = 15.$$

The decreasing-height order is the one suggested by the *objective*, but the constraint that can make an insertion infeasible is the *width*: that is what one must sort on. It is the same logic as first-fit decreasing for bin packing, where one sorts by size and not by value. A constructive heuristic that may fail is not wrong in itself — one only has to foresee the case and change the order — but it must be said explicitly, because a heuristic that “sometimes returns nothing” provides no bound at all.

LP relaxation and dual: the dual bound

We associate α_b free with constraints (10.17b) (equalities), $\beta_s \leq 0$ with (10.17c) ($\leq$ direction in a minimisation) and $\gamma_{bs} \geq 0$ with (10.17d). The dual is:

Dual of the LP relaxation of model 10.9

$$\max \sum_{b=1}^n \alpha_b + c \sum_{s=1}^m \beta_s \quad (10.18a)$$

$$\text{subject to } \sum_{b=1}^n \gamma_{bs} \leq 1, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18b)$$

$$\alpha_b + w_b \beta_s - h_b \gamma_{bs} \leq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}, \quad (10.18c)$$

$$\alpha_b \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \quad (10.18d)$$

$$\beta_s \leq 0, \quad \forall s \in \{1, 2, \dots, m\}, \quad (10.18e)$$

$$\gamma_{bs} \geq 0, \quad \forall b \in \{1, 2, \dots, n\}, \forall s \in \{1, 2, \dots, m\}. \quad (10.18f)$$

Description. The variable α_b is the value of book b , β_s the (non-positive) price of the width of shelf s , and γ_{bs} the price of the link "the height of shelf s covers book b ". The objective (10.18a) prices the books and the width available on each shelf. Constraints (10.18b) are the columns of the y_s : the height of shelf s costs 1 in the primal objective, and the prices of the links that push it up cannot be worth more. Constraints (10.18c) are the columns of the x_{bs} : putting book b on shelf s satisfies its assignment constraint (α_b), takes up w_b of width and forces the height up to h_b ; the balance cannot be positive, because the x_{bs} do not appear in the objective.

A hand-built dual solution. Set $\beta = 0$ (the width is not priced) and concentrate all the "weight" γ on the tallest book, number 1 with $h_1 = 8$: $\bar{\gamma}_{1s} = 1$ for every s and $\bar{\gamma}_{bs} = 0$ for $b \neq 1$. Constraints (10.18b) become $1 \leq 1$. Constraints (10.18c) become $\alpha_1 \leq 8$ for $b = 1$ and $\alpha_b \leq 0$ for $b \neq 1$: take $\bar{\alpha}_1 = 8$ and $\bar{\alpha}_b = 0$. The value is

$$LB = 8.$$

It is the obvious remark: the shelf holding the tallest book is at least as tall as that book.

A stronger combinatorial bound

The total width of the books is 18 and every shelf holds 10: at least

$$\left\lceil \frac{18}{10} \right\rceil = 2$$

shelves must be non-empty. One of them holds the tallest book and measures at least 8; the other one holds at least one book, so it measures at least $\min_{b \neq 1} h_b = 4$. Summing up,

$$z(MILP) \geq LB = 8 + 4 = 12,$$

better than the dual bound 8. Here too the jump comes from integrality: the relaxation can put half a book on each shelf and pay half the height twice, that is 8 in total again.

Optimal solution

The optimum is obtained by solving the model: the two bounds just built, the primal and the dual one, are what helps the solver get there.

	books	width	height
shelf 1	1, 2	8 out of 10	8
shelf 2	3, 4	10 out of 10	7

$$z(MILP) = 15, \quad 12 = LB \leq z(MILP) = 15 \leq UB = 15, \quad z(LP) = z(LP^+) = 8.$$

The heuristic gap is zero: first-fit by decreasing width finds the optimum. The certified gap, before solving the MILP, is $(15 - 12)/15 = 20\%$.

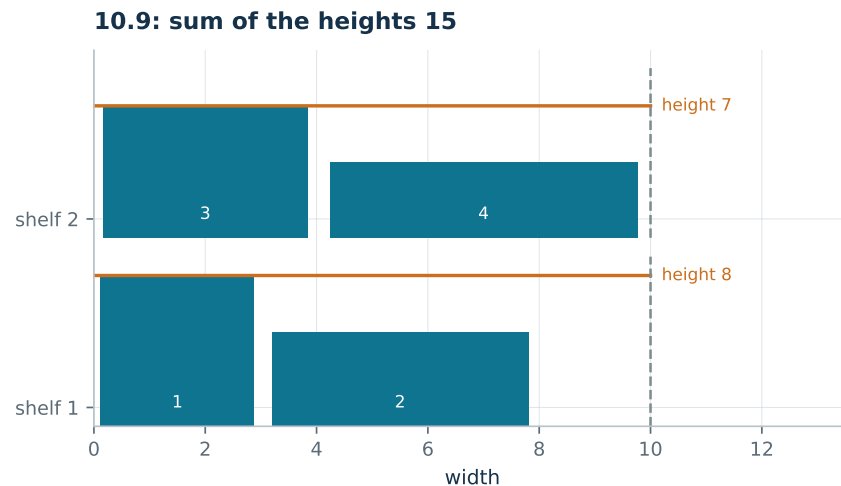


Figure 10.9: Optimal arrangement: the books on each shelf, with the shelf height marked in orange and the maximum width dashed.

Additional considerations

- The problem is a variant of bin packing in which the cost of a container is not fixed but depends on its contents. It is known as *bin packing with item fragmentation* or, in the warehousing literature, as *shelf space allocation*.
- The model has an annoying symmetry: swapping the two shelves gives the same solution with the same value. On larger instances it is worth breaking it, for instance by imposing $y_1 \geq y_2 \geq \dots \geq y_m$.
- The disaggregated form (10.17d) uses $n m$ constraints. The aggregated form with a big-M, $y_s \geq h_b - M(1 - x_{bs})$, would use just as many and is weaker: here the disaggregated one costs nothing and is preferable.

Additional modelling questions

Modelling question 10.17 — a third shelf

The library buys a third shelf, as wide as the others. What is the new optimum?

Modelling question 10.18 — wider shelves

The shelves are 12 wide instead of 10. What is the new optimum?

The final comparison

The three numbers that every problem of the course produces, side by side:

	value	what it is
UB	15	heuristic solution (UB)
LB	12	certified bound (LB)
$z(MILP)$	15	MILP optimum

- Gap of the heuristic with respect to the optimum:

$$\frac{|UB - z(MILP)|}{|z(MILP)|} = 0.0\%.$$

- Certified gap, that is the one known *before* solving the MILP:

$$\frac{|UB - LB|}{|z(MILP)|} = 20.0\%.$$

The sandwich on variant 9b

The width does not enter the dual certificate: the recipe puts the whole γ weight on the tallest book — number 1, 8 tall — because whatever shelf holds it must be at least as tall, and the dual bound stays 8. The combinatorial argument of the base problem is needed here too: the total width is 18 and one shelf holds 12, so the non-empty shelves are still at least two, one at least 8 tall and the other at least 4, that is $LB = 12$. What changes is the heuristic: first-fit by decreasing width now fits the four books on two shelves 5 and 8 tall and drops from 15 to 13; the optimum from 15 to 12, and the bound certifies it.

	value	what it is
UB	13	heuristic solution
LB	12	shelves used and minimum heights
$z(LP)$	8	relaxation without the bounds
$z(LP^+)$	8	relaxation with the bounds
$z(MILP)$	12	optimum of the MILP

Index

- absolute value, 130
- activation, 3, 10, 28, 29, 47, 50, 55, 64, 66, 69, 107, 113
- big-M, 3, 27, 40, 41, 44, 45, 74, 90, 92, 113, 140, 143
- certified gap, 77, 98, 143
- constructive heuristic, 3, 6, 13, 20, 26, 31, 32, 37, 43, 47, 50, 71, 76, 84, 90, 97, 102, 108, 114, 119, 125, 130, 136, 141
- dual bound, 91, 94, 98, 115, 127, 142
- fixed cost, 3, 9, 14, 21, 28, 52, 71, 72, 77
- inventory balance, 71, 73
- knapsack, 4, 28, 32
- LP relaxation, 3, 7, 13, 44, 47, 51, 57, 62, 67, 71, 74, 76, 78, 84, 94, 97, 103, 108, 125, 132, 137
- maximum variable, 3, 47, 140
- minimum lot, 71, 88, 90, 92, 94, 107–111
- p-median, 54
- piecewise linear function, 98
- precedences, 44
- primal bound, 94
- semi-continuous variable, 107
- semi-integer variable, 71, 88
- set packing, 101, 104
- strong duality, 8, 52, 131
- valid inequalities, 16, 22, 27, 33, 45, 69, 99, 104
- weak duality, 8, 21, 52, 57, 68